BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐH CÔNG NGHỆ KỸ THUẬT TP.HCM

BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN

MÁY BAY KHÔNG NGƯỜI LÁI TỰ ĐỘNG THEO DÕI CON NGƯỜI BẰNG TRÍ TUỆ NHÂN TẠO

Mã số: SV2026-474

Chủ nhiệm đề tài: Huỳnh Lê Thành Nhân

TP Hồ Chí Minh, 09/2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐH CÔNG NGHỆ KỸ THUẬT TP.HCM

BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN

MÁY BAY KHÔNG NGƯỜI LÁI TỰ ĐỘNG THEO DÕI CON NGƯỜI BẰNG TRÍ TUỆ NHÂN TẠO

Mã số: SV2026-474

Thuộc nhóm ngành khoa học: Kỹ thuật và Công nghệ

SV thực hiện: Huỳnh Lê Thành Nhân **Nam, Nữ:** Nam

Dân tộc: Kinh

Lớp, khoa: 24119FIE4, Khoa Đào tạo tiên tiến

Năm thứ: 3 / **Số năm đào tạo:** 4

Ngành học: Công nghệ kỹ thuật máy tính

(SV chịu trách nhiệm chính thực hiện đề tài: Huỳnh Lê Thành Nhân)

Thành viên: Trịnh Trần Quốc Bảo (MSSV 24119016)

Người hướng dẫn: Phó Giáo sư, Tiến sĩ Lê Mỹ Hà

TP Hồ Chí Minh, 09/2026

MỤC LỤC

DANH MỤC BẢNG BIỂU . . . . . **v**
DANH MỤC HÌNH ẢNH . . . . . **vii**
DANH MỤC CÁC TỪ VIẾT TẮT . . . . . **ix**
THÔNG TIN KẾT QUẢ NGHIÊN CỨU CỦA ĐỀ TÀI . . . . . **xi**
MỞ ĐẦU **1**
1. Tổng quan tình hình nghiên cứu thuộc lĩnh vực đề tài 1
2. Lý do chọn đề tài 2
3. Mục tiêu đề tài 3
4. Phương pháp nghiên cứu 3
5. Đối tượng và phạm vi nghiên cứu 4
6. Nội dung nghiên cứu và mức độ hoàn thành 4
7. Bố cục báo cáo 5
CHƯƠNG 1 Cơ sở lý thuyết **6**
1.1 Bài toán phát hiện đối tượng 6
1.1.1 Định nghĩa 6
1.1.2 Loại bỏ hộp không cực đại 6
1.2 Mạng nơ-ron tích chập cho phát hiện đối tượng 7
1.2.1 Bộ phát hiện hai giai đoạn và một giai đoạn 7
1.2.2 Kim tự tháp đặc trưng 7
1.2.3 Head anchor-free 7
1.2.4 Họ YOLO và bộ phát hiện dựa trên Transformer 7
1.3 Các khối xây dựng mạng hiệu quả 8
1.3.1 Tích chập depthwise separable 8
1.3.2 Kết nối từng phần liên tầng và tái tham số hoá 8
1.3.3 Self-attention theo cửa sổ 8
1.4 Phát hiện người có kích thước nhỏ từ UAV 9
1.4.1 Giữ mức phân giải cao 9
1.4.2 Khoảng cách Wasserstein chuẩn hoá 9
1.4.3 Hàm mất mát hồi quy CIoU và phân loại theo chất lượng 9
1.5 Suy giảm ảnh do rung động của UAV 10
1.5.1 Nguồn gốc vật lý 10
1.5.2 Độ bền của mạng nơ-ron trước ảnh suy giảm 10
1.6 Lượng tử hoá mô hình và bộ tăng tốc nơ-ron 11
1.6.1 Lượng tử hoá INT8 11
1.6.2 Bộ tăng tốc Hailo-8L 11
1.7 Theo vết đa đối tượng 11
1.7.1 Bộ lọc Kalman 11
1.7.2 Từ SORT tới OC-SORT 12
1.8 Mô hình camera và ước lượng khoảng cách 12

1.9 Điều khiển servo thị giác và bộ điều khiển PID . . . 13
1.9.1 Servo thị giác . . . 13
1.9.2 Bộ điều khiển PID . . . 13
1.10 Bộ điều khiển bay ArduPilot và giao thức MAVLink . . . 13
1.11 Các chỉ số đánh giá . . . 14
1.12 Kết luận chương . . . 14
CHƯƠNG 2 Dữ liệu và phương pháp đánh giá . . . 16
2.1 Yêu cầu đối với dữ liệu . . . 16
2.2 Các bộ dữ liệu . . . 16
2.2.1 VisDrone-person . . . 16
2.2.2 COCO-person . . . 17
2.2.3 Tập bay thật flight_val . . . 17
2.3 Hai nhánh mô hình . . . 18
2.4 Tiền xử lý và data augmentation . . . 18
2.5 Giao thức mô phỏng rung VisDrone-Shake . . . 19
2.5.1 Mô hình suy giảm . . . 19
2.5.2 Giao thức phải được cố định seed ngẫu nhiên . . . 20
2.6 Một chương trình đo duy nhất cho mọi mô hình . . . 20
2.7 Quy tắc chọn checkpoint . . . 21
2.8 Đánh giá trên miền triển khai . . . 21
2.9 Môi trường thực nghiệm . . . 22
2.10 Kết luận chương . . . 22
CHƯƠNG 3 Mô hình SteadyDet-N và phương pháp huấn luyện nhận biết rung . . . 23
3.1 Yêu cầu thiết kế . . . 23
3.2 Quá trình phát triển kiến trúc . . . 23
3.3 Kiến trúc SteadyDet-N . . . 24
3.3.1 Backbone . . . 25
3.3.2 FPN neck mỏng . . . 26
3.3.3 Head LiteHead . . . 26
3.3.4 So sánh hai phiên bản . . . 27
3.4 Hàm mất mát . . . 27
3.5 Quy trình huấn luyện . . . 28
3.6 Huấn luyện nhận biết rung . . . 29
3.6.1 Ý tưởng . . . 29
3.6.2 Phương pháp . . . 29
3.6.3 Biến thể ràng buộc nhất quán . . . 30
3.6.4 Thiết lập tinh chỉnh . . . 31
3.6.5 Biến thể không dùng NMS . . . 31
3.7 Kết luận chương . . . 31
CHƯƠNG 4 Kết quả thực nghiệm của mô hình . . . 33
4.1 Kết quả chính và các thí nghiệm đối chứng . . . 33
4.1.1 Mọi bộ phát hiện nhỏ gọn đều sụp đổ khi rung . . . 33
4.1.2 Huấn luyện nhận biết rung tăng gấp đôi tỷ lệ giữ độ chính xác . 34

4.1.3 Đối chứng 1: tác dụng không đến từ việc huấn luyện thêm . . . 34
4.1.4 Đối chứng 2: ràng buộc nhất quán kém hơn phương pháp đơn giản 34
4.2 Tính tổng quát và giới hạn của phương pháp . . . 35
4.2.1 Lặp lại trên một kiến trúc khác . . . 35
4.2.2 Không có tác dụng trên YOLO11-n . . . 35
4.3 So sánh với các mô hình phát hiện nhẹ . . . 37
4.3.1 Trên ảnh sạch, SteadyDet-N không dẫn đầu . . . 37
4.3.2 Khi rung, thứ hạng đảo ngược . . . 38
4.3.3 Độ bền không phải là sản phẩm phụ của độ chính xác . . . 39
4.4 Kiểm chứng trên dữ liệu bay thật . . . 39
4.4.1 Cả hai cách xử lý rung đều có tác dụng trên rung thật . . . 40
4.4.2 Chọn dữ liệu theo miền triển khai, không theo benchmark . . . 41
4.5 Thảo luận . . . 41
CHƯƠNG 5 Triển khai mô hình trên thiết bị biên . . . 43
5.1 Nền tảng phần cứng . . . 43
5.2 Chuẩn bị mô hình cho trình biên dịch . . . 44
5.2.1 Loại bỏ khối self-attention . . . 44
5.2.2 Một lỗi cấu hình và cách phát hiện . . . 44
5.2.3 Xuất ONNX . . . 44
5.3 Lượng tử hoá và biên dịch . . . 45
5.4 Độ chính xác sau lượng tử hoá . . . 45
5.4.1 Chọn tệp HEF triển khai . . . 45
5.4.2 Tổn thất INT8 phụ thuộc kích thước đối tượng . . . 45
5.4.3 Chọn ngưỡng tin cậy vận hành . . . 47
5.5 Tốc độ suy luận trên NPU . . . 47
5.5.1 Tốc độ theo từng khung và theo lô . . . 47
5.5.2 So sánh với YOLO26-n trên cùng một chip . . . 48
5.6 Độ trễ đầu-cuối với camera thật . . . 49
5.6.1 Hai lỗi phía camera làm hệ thống kẹt ở 15 khung hình/giây . . . 49
5.6.2 Phân tích độ trễ từng chặng . . . 50
5.7 Kết luận chương . . . 51
CHƯƠNG 6 Hệ thống điều khiển máy bay bám theo người . . . 52
6.1 Kiến trúc tổng thể . . . 52
6.2 Vòng lặp điều khiển . . . 53
6.3 Theo vết đa đối tượng . . . 54
6.4 Chọn và khoá mục tiêu . . . 55
6.5 Ước lượng khoảng cách . . . 56
6.6 Servo thị giác và bộ điều khiển PID . . . 57
6.7 Lớp an toàn . . . 58
6.8 Giao tiếp MAVLink và lựa chọn nhịp gửi lệnh . . . 59
6.8.1 Kiểm tra liên lạc . . . 59
6.8.2 Chọn nhịp gửi lệnh 30 Hz . . . 60
6.9 Bảng giám sát và ghi nhật ký . . . 61
6.10 Kết luận chương . . . 61

CHƯƠNG 7 Tích hợp và kiểm thử toàn hệ thống **62**
7.1 Cấu hình hệ thống tích hợp 62
7.2 Quy trình kiểm thử phân tầng 63
7.3 Kiểm thử tự động 63
7.4 Bay thử 64
7.4.1 Chuyến bay thu dữ liệu và video thử nghiệm 64
7.4.2 Vòng bám người trên video bay thử 65
7.4.3 Quy trình bay bám tự động 66
7.5 Đánh giá mức độ đạt mục tiêu 67
7.6 Hạn chế 67
7.7 Kết luận chương 68
KẾT LUẬN VÀ KIẾN NGHỊ **69**
1. Kết luận 69
2. Kiến nghị 70
TÀI LIỆU THAM KHẢO **72**
PHỤ LỤC A Cấu hình phần cứng và tham số hệ thống **77**
A.1 Tham số an toàn trên bộ điều khiển bay (ArduPilot Copter 4.7.0) 77
A.2 Tham số chính của chương trình bám người (`config.yaml`) 79
A.3 Siêu tham số huấn luyện 80
PHỤ LỤC B Chương trình máy tính **81**
B.1 Cấu trúc mã nguồn 81
B.2 Các lệnh tái lập kết quả 82
B.3 Trích đoạn mã nguồn tiêu biểu 82
PHỤ LỤC C Quy trình vận hành bay bám người **84**
PHỤ LỤC D Minh chứng sản phẩm đề tài **87**
D.1 Danh mục sản phẩm nộp kèm 87
D.2 Hình ảnh sản phẩm 88
PHỤ LỤC E Poster và standee của đề tài **89**
PHỤ LỤC F Bản sao thuyết minh và hợp đồng đã được phê duyệt **91**

DANH MỤC BẢNG BIỂU

Bảng 1 Đối chiếu nội dung thuyết minh với kết quả thực hiện 5

Bảng 2.1 Thống kê ba bộ dữ liệu sau chuẩn hoá (lớp duy nhất *person*) . . . 18
Bảng 2.2 Tham số ba mức rung của giao thức VisDrone-Shake 19
Bảng 2.3 Môi trường phần cứng và phần mềm dùng trong đề tài 22

Bảng 3.1 So sánh cấu trúc và chi phí của SteadyDet-W và SteadyDet-N (ảnh vào 640×640) 27

Bảng 4.1 Kết quả trên VisDrone-person, ảnh 640×640. Trong mỗi khối, các dòng chỉ khác nhau đúng một yếu tố và dùng chung lịch trình, learning rate và phép tăng cường. Cột có rung: trung bình $\pm$ độ lệch chuẩn qua 3 hạt giống 33
Bảng 4.2 So sánh với năm mô hình công khai trên VisDrone-person, mọi dòng đo bằng cùng một chương trình. mCP: AP@[.50:.95] trung bình ba mức rung, tức độ chính xác *khi bay*. Các mô hình YOLO và RT-DETR khởi tạo từ trọng số huấn luyện sẵn trên COCO, còn SteadyDet huấn luyện từ đầu, điều này có lợi cho mô hình đối chứng 37
Bảng 4.3 Kết quả trên 92 khung hình bay thật ($\approx$6 m), cùng chương trình đo. Trong mỗi nhóm, ba mô hình giống hệt kiến trúc và chỉ khác cách xử lý rung 40

Bảng 5.1 So sánh các tệp HEF trên 92 khung bay thật; mỗi tệp dùng ngưỡng tin cậy cho F1 cao nhất của chính nó 45
Bảng 5.2 Độ chính xác FP32 và INT8 của mô hình triển khai, cùng một chương trình chấm điểm 46
Bảng 5.3 Thông lượng và độ trễ theo batch size trên Hailo-8L (mô hình 320×320) 48
Bảng 5.4 SteadyDet-N và YOLO26-n trên cùng Hailo-8L, ảnh 640×640, lô 1 48
Bảng 5.5 Độ trễ từng chặng của chuỗi xử lý trên Raspberry Pi 5 + Hailo-8L (ms), 3.594 khung 50

Bảng 6.1 Tham số bộ theo vết OC-SORT 55
Bảng 6.2 Diễn biến hành vi khi mất mục tiêu (tính từ lần cuối nhìn thấy) . . 56
Bảng 6.3 Tham số ba bộ điều khiển PID 57
Bảng 6.4 Ngân sách độ trễ từ ảnh tới lệnh theo nhịp gửi (ms) 60

Bảng 7.1 Các thành phần của hệ thống tích hợp 62
Bảng 7.2 Các tầng kiểm thử và kết quả 63
Bảng 7.3 Kết quả vòng bám người trên video bay thử 21/07/2026 (3.961 khung) 66
Bảng 7.4 Đối chiếu kết quả với mục tiêu và yêu cầu khoa học trong thuyết minh 67

Bảng A.1 Tham số ArduPilot cần chỉnh trước khi bay tự động 77
Bảng A.2 Một số tham số ArduPilot đã đổi tên và đơn vị ở phiên bản 4.7 . . 78
Bảng A.3 Tham số cấu hình chính của chương trình 79
Bảng A.4 Siêu tham số huấn luyện SteadyDet-N 80

Bảng C.1 Điều kiện bắt buộc trước khi arm 85
Bảng C.2 Sự cố thường gặp và cách xử lý 86

Bảng D.1 Danh mục sản phẩm nộp kèm hồ sơ nghiệm thu 87

DANH MỤC HÌNH ẢNH

Hình 2.1 Ba nguồn dữ liệu của đề tài và nhãn bounding box tương ứng: VisDrone-person chụp từ trên cao (trái), COCO-person ở cự ly gần (giữa) và khung hình bay thật ở độ cao khoảng 6 m (phải) . . 16

Hình 2.2 Minh hoạ ba mức rung VisDrone-Shake trên một ảnh VisDrone (hàng trên) và một khung bay thật (hàng dưới); hướng nhoè cố định 35° để dễ so sánh 19

Hình 3.1 Kiến trúc SteadyDet-N; độ phân giải giảm dần từ trên xuống. Chi phí được phân bổ không đối xứng: nhánh P_2 (nét đứt) chỉ có một khối, số kênh bằng một nửa và head tách được, còn các khối CSP hai tầng và phép self-attention nằm ở những mức có bản đồ nhỏ . 25

Hình 3.2 Hàm mất mát huấn luyện (trái, thang log) và AP@0,5 trên tập kiểm định ảnh sạch (phải) của SteadyDet-N nhánh chuẩn: 300 epoch huấn luyện gốc (xám) và 120 epoch tinh chỉnh nhận biết rung (xanh). Hàm mất mát tăng vọt khi bắt đầu tinh chỉnh vì một nửa số lô là ảnh đã bị làm rung 29

Hình 3.3 (a) Huấn luyện nhận biết rung: một nửa số lô được thay bằng bản đã làm rung và tính hàm mất mát phát hiện thông thường. (b) Biến thể ràng buộc nhất quán: truyền xuôi cả hai bản và kéo dự đoán trên ảnh rung về phía dự đoán trên ảnh sạch đã dừng gradient. Cả hai chỉ tồn tại khi huấn luyện 30

Hình 4.1 AP@[.50:.95] theo mức rung. Mô hình gốc (nét đứt) có AP sạch cao hơn nhưng giảm nhanh nhất; sau huấn luyện nhận biết rung (nét liền xanh), SteadyDet-N vượt YOLO11-n và RT-DETR-l từ mức rung 2. Thanh sai số: độ lệch chuẩn qua 3 hạt giống 35

Hình 4.2 Tỷ lệ giữ độ chính xác khi rung của mọi mô hình. Bốn thế hệ YOLO bản nano nằm sát nhau ở 19–20% dù khác nhau về kiến trúc; hai mô hình của đề tài sau huấn luyện nhận biết rung đứng đầu 38

Hình 4.3 AP trên ảnh sạch và mCP khi rung, mỗi điểm là một mô hình. Mũi tên xanh: tác dụng của huấn luyện nhận biết rung trên hai mô hình của đề tài 39

Hình 4.4 Kết quả trên dữ liệu bay thật theo nhóm dữ liệu huấn luyện. Nhóm có COCO-person (trái) chính xác gấp nhiều lần nhóm chỉ VisDrone (phải); trong nhóm có COCO, cả hai cách xử lý rung làm AP@0,75 tăng gần gấp đôi 40

Hình 5.1 Quy trình đưa mô hình lên NPU: biên dịch một lần trên máy tính x86 (trên) và chuỗi xử lý mỗi khung hình trên Raspberry Pi 5 (dưới) 43

Hình 5.2 Độ chính xác FP32 và INT8 trên hai miền dữ liệu. Trên người rất nhỏ, INT8 làm hộp kém khít (AP@0,75 giảm 25%); trên người ở độ cao bay thực tế, hai bản gần như trùng nhau 46

Hình 5.3 F1 của mô hình triển khai theo ngưỡng tin cậy trên 92 khung bay thật 47
Hình 5.4 Trái: tốc độ SteadyDet-N và YOLO26-n trên cùng Hailo-8L. Phải: độ trễ phần cứng tăng theo batch size dù thông lượng tăng 49
Hình 5.5 Tốc độ khung thực tế của chuỗi xử lý với bốn cấu hình camera . . 50
Hình 5.6 Phân bổ độ trễ trung vị theo chặng so với ngân sách 33,3 ms của vòng lặp 30 Hz 51

Hình 6.1 Kiến trúc tổng thể hệ thống. Toàn bộ khối xử lý nằm trên Raspberry Pi 5; bảng giám sát chỉ đọc (trừ nút đổi mục tiêu); tay điều khiển RC luôn có quyền ưu tiên cao nhất 52
Hình 6.2 Lưu đồ một vòng lặp điều khiển 30 Hz trong `main.py` 54
Hình 6.3 Máy trạng thái của thang mất mục tiêu; phần mềm không bao giờ tự đổi chế độ bay 59
Hình 6.4 Độ trễ từ ảnh tới lệnh theo nhịp gửi 60

Hình 7.1 Ba khung hình trích từ video bay thử (hộp xanh: phát hiện của SteadyDet-N; dấu thập: tâm khung hình). Giây 15 và 45: phát hiện đúng. Giây 80: người bị cắt ở mép trái bị bỏ sót và có một hộp sai nhỏ 65
Hình 7.2 Vòng bám người trên video bay thử: hộp xanh lá là mục tiêu đang khoá kèm mã định danh, mũi tên nối tâm khung hình với mục tiêu, vệt vàng là quỹ đạo; bảng góc trái hiển thị trạng thái, sai số hướng và lệnh quay hướng tính được 65

Hình D.1 Phần 1 của video bay thử: SteadyDet-N phát hiện người trên các khung hình của chuyến bay 21/07/2026 88
Hình D.2 Phần 2 của video bay thử: vòng bám người khoá mục tiêu, hiển thị trạng thái, sai số hướng và lệnh quay hướng 88

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Nghĩa tiếng Việt
AP	Average Precision	Độ chính xác trung bình
AR	Average Recall	Độ phủ trung bình
CIoU	Complete Intersection over Union	Chỉ số chồng lấp đầy đủ
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
COCO	Common Objects in Context	Bộ dữ liệu COCO
CPU	Central Processing Unit	Bộ xử lý trung tâm
CSP	Cross Stage Partial	Kết nối từng phần liên tầng
DFC	Dataflow Compiler	Trình biên dịch luồng dữ liệu (Hailo)
EKF	Extended Kalman Filter	Bộ lọc Kalman mở rộng
FCU	Flight Control Unit	Bộ điều khiển bay
FLOPs	Floating-point Operations	Số phép tính dấu phẩy động
FP32	32-bit Floating Point	Số thực dấu phẩy động 32 bit
FPN	Feature Pyramid Network	Mạng kim tự tháp đặc trưng
FPS	Frames Per Second	Số khung hình trên giây
GCS	Ground Control Station	Trạm điều khiển mặt đất
GFLOPs	Giga FLOPs	Tỷ phép tính dấu phẩy động
GPS	Global Positioning System	Hệ thống định vị toàn cầu
GPU	Graphics Processing Unit	Bộ xử lý đồ hoạ
HEF	Hailo Executable Format	Tệp mô hình thực thi trên NPU Hailo
IMU	Inertial Measurement Unit	Khối đo lường quán tính
INT8	8-bit Integer	Số nguyên 8 bit
IoU	Intersection over Union	Tỷ lệ giao trên hợp
MAVLink	Micro Air Vehicle Link	Giao thức truyền thông cho phương tiện bay nhỏ
mCP	mean Corrupted Precision	AP trung bình trên ảnh bị suy giảm
NMS	Non-Maximum Suppression	Loại bỏ hộp không cực đại
NPU	Neural Processing Unit	Bộ xử lý nơ-ron chuyên dụng
NWD	Normalized Wasserstein Distance	Khoảng cách Wasserstein chuẩn hoá

Viết tắt	Tiếng Anh	Nghĩa tiếng Việt
OC-SORT	Observation-Centric SORT	Thuật toán theo vết đa đối tượng lấy quan sát làm trung tâm
ONNX	Open Neural Network Exchange	Định dạng trao đổi mạng nơ-ron mở
PID	Proportional–Integral–Derivative	Bộ điều khiển tỷ lệ – tích phân – vi phân
PTQ	Post-Training Quantization	Lượng tử hoá sau huấn luyện
QFL	Quality Focal Loss	Hàm mất mát tiêu điểm theo chất lượng
RTL	Return To Launch	Chế độ tự bay về điểm cất cánh
SITL	Software In The Loop	Mô phỏng phần mềm trong vòng lặp
SoC	System on Chip	Hệ thống trên một vi mạch
SORT	Simple Online and Realtime Tracking	Theo vết trực tuyến thời gian thực đơn giản
STAL	Small-Target-Aware Label Assignment	Gán nhãn nhận biết mục tiêu nhỏ
TOPS	Tera Operations Per Second	Nghìn tỷ phép tính mỗi giây
UAV	Unmanned Aerial Vehicle	Máy bay không người lái
UIB	Universal Inverted Bottleneck	Khối nút cổ chai đảo ngược tổng quát
USB	Universal Serial Bus	Chuẩn giao tiếp nối tiếp đa năng
YOLO	You Only Look Once	Họ mô hình phát hiện đối tượng một giai đoạn

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐH CÔNG NGHỆ KỸ THUẬT TP.HCM

THÔNG TIN KẾT QUẢ NGHIÊN CỨU CỦA ĐỀ TÀI

1. Thông tin chung:

– Tên đề tài: Máy bay không người lái tự động theo dõi con người bằng trí tuệ nhân tạo.

– Mã số: SV2026-474.

– Chủ nhiệm đề tài: Huỳnh Lê Thành Nhân Mã số SV: 24119068

– Lớp: 24119FIE4 Khoa: Đào tạo tiên tiến

– Thành viên đề tài:

STT	Họ và tên	MSSV	Lớp	Khoa
1	Huỳnh Lê Thành Nhân	24119068	24119FIE4	Đào tạo tiên tiến
2	Trịnh Trần Quốc Bảo	24119016	24119FIE4	Đào tạo tiên tiến

– Người hướng dẫn: PGS.TS. Lê Mỹ Hà (halm@hcmute.edu.vn).

2. Mục tiêu đề tài:

- Xây dựng mô hình trí tuệ nhân tạo phát hiện và nhận dạng con người theo thời gian thực từ camera gắn trên máy bay không người lái, đạt độ chính xác từ 70% trở lên.
- Thiết kế thuật toán theo dõi chuyển động để máy bay giữ khoảng cách và hướng bay phù hợp với người được theo.
- Tích hợp camera – khối xử lý ảnh – bộ điều khiển bay thành một hệ thống hoạt động ổn định trong môi trường thực tế.

3. Tính mới và sáng tạo:

- Đề xuất mạng phát hiện người SteadyDet-N chỉ **1,17 triệu tham số và 4,6 GFLOPs**, giữ nhánh độ phân giải cao P2 (bước 4) để thấy được người ở xa, nhưng làm mỏng nhánh này để chạy được trên thiết bị biên.
- Chỉ ra bằng đo đạc có đối chứng rằng rung động của khung máy bay làm mọi bộ phát hiện nhỏ mất 74–83% độ chính xác, và đề xuất **huấn luyện nhận biết rung**: đưa mô hình vật lý của motion blur, nhiễu cảm biến và dao động phơi sáng vào dữ liệu huấn luyện. Tỷ lệ giữ độ chính xác khi rung tăng từ **16,8% lên 36,2%** mà **không tốn thêm chi phí suy luận**; khi rung, SteadyDet-N vượt cả bốn mô hình YOLO bản nano.

- Xây dựng giao thức đánh giá rung VisDrone-Shake **cố định seed ngẫu nhiên, lặp 3 lần và báo cáo độ lệch chuẩn**, dùng một chương trình chấm điểm duy nhất cho mọi mô hình.
- Chứng minh trên phần cứng thật rằng tốc độ trên NPU Hailo-8L phụ thuộc vào **số context** của mô hình chứ không chỉ số phép tính: SteadyDet-N chạy **nhanh hơn YOLO26-n 1,70 lần** trên cùng một chip.
- Thiết kế lớp an toàn theo nguyên tắc **người lái luôn giữ quyền chọn chế độ bay**: phần mềm chỉ đề xuất vận tốc khi máy bay đang ở chế độ GUIDED, mất mục tiêu thì đứng yên chờ, và nguyên tắc được khoá bằng kiểm thử tự động.

4. Kết quả nghiên cứu:

- Mô hình chạy INT8 trên NPU Hailo-8L đạt **AP@0,5 = 0,917** (tức 91,7%, vượt chỉ tiêu 70%) và **F1 = 0,880** (recall 0,837, precision 0,928) trên 92 khung hình bay thật ở độ cao khoảng 6 m.
- Toàn bộ chuỗi xử lý camera – tiền xử lý – suy luận – hậu xử lý chạy **30,0 khung hình/giây** trên Raspberry Pi 5 (trung vị 32,04 ms; p99 36,15 ms), nhiệt độ 48,5 °C, không giảm xung.
- Chương trình máy tính điều khiển bám người hoàn chỉnh: theo vết OC-SORT, khoá mục tiêu, ước lượng khoảng cách, bộ điều khiển PID ba kênh, thang an toàn khi mất mục tiêu, gửi lệnh MAVLink tới ArduPilot ở chế độ GUIDED với nhịp 30 Hz (khứ hồi lệnh p50 4,75 ms, 0% mất gói), cùng bảng giám sát web; 2 bộ kiểm thử tự động với 25/25 phép kiểm tra đều đạt. Chạy trên video bay thử, vòng bám người giữ được mục tiêu trong 83,5% thời gian, sai số hướng trung vị 5,5°.
- Báo cáo phân tích gồm các bảng so sánh với 5 mô hình đối chứng (YOLOv8-n, YOLOv10-n, YOLO11-n, YOLO26-n, RT-DETR-l) dưới một giao thức đo thống nhất.

5. Đóng góp về mặt giáo dục và đào tạo, kinh tế – xã hội, an ninh, quốc phòng và khả năng áp dụng của đề tài:

Hệ thống xử lý hoàn toàn trên máy bay, không cần kết nối Internet, nên dùng được cho tìm kiếm cứu nạn ở vùng mất sóng, giám sát an ninh khu vực, hỗ trợ tuần tra và quay phim tự động. Kỹ thuật huấn luyện nhận biết rung chỉ thay đổi dữ liệu huấn luyện, áp dụng được cho các mô hình phát hiện khác trên thiết bị bay mà không cần gimbal chống rung đắt tiền. Về đào tạo, đề tài cung cấp một quy trình trọn vẹn và tái lập được từ dữ liệu, huấn luyện, lượng tử hoá, triển khai trên NPU tới điều khiển bay, dùng làm học liệu

cho các học phần thị giác máy tính, hệ thống nhúng và điều khiển tự động. Sản phẩm được thử nghiệm tại Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh.

6. Công bố khoa học của SV từ kết quả nghiên cứu của đề tài:

Không có.

Ngày . . . tháng . . . năm 2026
SV chịu trách nhiệm chính thực hiện đề tài
(ký, họ và tên)

Huỳnh Lê Thành Nhân

Nhận xét của người hướng dẫn về những đóng góp khoa học của SV thực hiện đề tài *(phần này do người hướng dẫn ghi)*:

. . . . . .

. . . . . .

. . . . . .

. . . . . .

Ngày . . . tháng . . . năm 2026
Người hướng dẫn
(ký, họ và tên)

PGS.TS. Lê Mỹ Hà

MỞ ĐẦU

1. Tổng quan tình hình nghiên cứu thuộc lĩnh vực đề tài

1.1. Tình hình nghiên cứu ngoài nước

Máy bay không người lái (UAV) loại nhiều cánh quạt đã trở thành một nền tảng cảm biến phổ biến nhờ giá thành giảm nhanh và khả năng bay treo tại chỗ. Khi gắn thêm camera và một bộ xử lý đủ mạnh, UAV có thể tự nhận biết đối tượng trong ảnh và phản ứng mà không cần người điều khiển. Bài tổng quan của Wu và cộng sự [1] cho thấy phát hiện và theo vết đối tượng bằng học sâu là hướng được quan tâm nhiều nhất trong thị giác máy tính cho UAV, đồng thời chỉ ra hai khó khăn đặc thù: đối tượng thường rất nhỏ trong khung hình và tài nguyên tính toán trên máy bay rất hạn chế. Các bộ dữ liệu chuẩn như VisDrone [2] và UAVDT [3] ra đời để đo lường đúng những khó khăn này.

Về bài toán bám theo người, nhiều hệ thống đã được công bố với các lựa chọn khác nhau. Yang và cộng sự [4] kết hợp một mạng YOLOv4 đã được tỉa bớt để phát hiện nhanh với mô hình SiamMask để theo vết liên tục, rồi dùng bộ điều khiển PID để điều chỉnh tư thế bay trên nền tảng ROS, thử nghiệm cả trong nhà và ngoài trời. Lo và cộng sự [5] xây dựng hệ thống giám sát tự động dùng YOLOv4-Tiny, ước lượng tư thế ba chiều của đối tượng và bộ lọc Kalman để UAV bám theo mục tiêu chuyển động. Liu và Szirányi [6] dùng YOLOv3-tiny chạy ngay trên máy bay để phát hiện người và nhận dạng cử chỉ phục vụ cứu nạn. Gần đây hơn, Kang và Kim [7] đề xuất một UAV bám theo người dựa trên phát hiện điểm mốc cơ thể ba chiều, dùng bộ điều khiển PI để giữ vị trí tương đối và nhận dạng cử chỉ để điều khiển thiết bị IoT. Ở mảng thương mại, các hãng như DJI đã tích hợp tính năng tự động bám theo đối tượng (ActiveTrack) vào sản phẩm tiêu dùng [8], cho thấy nhu cầu thực tế của chức năng này.

Song song với các hệ thống ứng dụng, nhiều nhóm nghiên cứu tại châu Âu tập trung vào việc đưa mạng nơ-ron lên máy bay có tài nguyên cực nhỏ. Nhóm của Đại học Zurich và ETH Zurich đã trình diễn mạng DroNet học lái UAV từ dữ liệu xe đạp và ô tô [9], rồi thu gọn nó để chạy với công suất 64 mW trên máy bay cỡ nano [10]; cũng nhóm này đã huấn luyện được UAV đua thắng các tay đua vô địch bằng học tăng cường [11]. Về mô hình phát hiện, các họ YOLO [12, 13, 14, 15] và RT-DETR [16] liên tục cải thiện

độ chính xác trên mỗi đơn vị tính toán, còn RemDet [17] thiết kế riêng cho ảnh UAV với nhiều vật thể nhỏ.

Tuy nhiên, gần như toàn bộ các công trình trên đánh giá mô hình trên ảnh tĩnh, sạch, thu từ nền tảng ổn định. Trong khi đó, khung máy bay nhiều cánh quạt rung liên tục khi động cơ quay; rung truyền tới camera tạo ra motion blur, tăng nhiễu và làm độ sáng dao động. Các nghiên cứu về độ bền của mạng nơ-ron trước ảnh bị suy giảm [18, 19, 20] đã chứng minh mạng học sâu rất nhạy với nhoè và nhiễu, nhưng chưa có giao thức nào mô phỏng riêng loại suy giảm do rung của UAV để đánh giá các bộ phát hiện nhỏ gọn. Đây là khoảng trống mà đề tài hướng tới.

1.2. Tình hình nghiên cứu trong nước

Tại Việt Nam, các trường đại học và viện nghiên cứu đã có nhiều đề tài về UAV và xử lý ảnh bằng trí tuệ nhân tạo. Nhóm nghiên cứu tại Học viện Kỹ thuật Quân sự đã công bố hệ thống phát hiện và theo vết UAV trong nền phức tạp bằng kết hợp nhiều mô hình học sâu [21]. Tại Trường Đại học Công nghệ Kỹ thuật TP.HCM, nhóm của PGS.TS. Lê Mỹ Hà nghiên cứu nâng cao độ bền của khối phát hiện đối tượng cho hệ thống hỗ trợ lái xe [22], một bài toán có cùng bản chất với đề tài này là giữ độ chính xác của bộ phát hiện khi điều kiện ảnh xấu đi. Ở mảng ứng dụng cứu nạn, Ngô và cộng sự [23] xây dựng khung UAV phát hiện, theo vết và phân loại tư thế người gặp nạn trên biển, có thử nghiệm tại tỉnh Quảng Ninh.

Nhìn chung, phần lớn nghiên cứu trong nước mới dừng ở phát hiện đối tượng trên video thu sẵn, định vị bằng GPS hoặc ổn định tư thế bay. Hướng nghiên cứu UAV tự động bám theo người, với toàn bộ khâu nhận dạng chạy ngay trên máy bay trong thời gian thực và khép kín vòng điều khiển, vẫn còn ở giai đoạn thử nghiệm. Việc triển khai mô hình học sâu lên thiết bị biên còn hạn chế vì yêu cầu cao về năng lực tính toán, độ trễ và độ ổn định của hệ thống.

2. Lý do chọn đề tài

UAV ngày càng được dùng rộng rãi trong giám sát, cứu hộ, quay phim và giao hàng. Trong nhiều ứng dụng này, người điều khiển vừa phải lái máy bay vừa phải giữ đối tượng trong khung hình, một yêu cầu khó đáp ứng khi đối tượng di chuyển liên tục. Một UAV tự nhận dạng và bám theo người sẽ giải phóng người vận hành khỏi thao tác lái, tăng tính

linh hoạt, an toàn và tiện ích của hệ thống.

Đề tài được lựa chọn vì ba lý do cụ thể. *Thứ nhất*, bài toán đòi hỏi kết hợp nhiều lĩnh vực: học sâu, tối ưu mô hình cho phần cứng nhúng, theo vết đa đối tượng và điều khiển bay; đây là cơ hội để sinh viên ngành Công nghệ kỹ thuật máy tính vận dụng kiến thức liên ngành. *Thứ hai*, qua khảo sát ban đầu, nhóm nhận thấy các bộ phát hiện nhẹ đạt điểm cao trên bộ dữ liệu chuẩn nhưng mất phần lớn độ chính xác khi ảnh bị nhoè do rung, một vấn đề thực tế nhưng ít được đo lường. *Thứ ba*, các bộ tăng tốc nơ-ron giá rẻ như Hailo-8L gắn trên Raspberry Pi 5 đã xuất hiện, mở ra khả năng xây dựng hệ thống xử lý hoàn toàn trên máy bay với chi phí thấp, phù hợp điều kiện nghiên cứu của sinh viên.

3. Mục tiêu đề tài

Theo thuyết minh đã được phê duyệt, đề tài có ba mục tiêu:

1. Xây dựng mô hình máy bay không người lái có khả năng phát hiện và nhận dạng con người theo thời gian thực bằng trí tuệ nhân tạo, với độ chính xác nhận dạng từ 70% trở lên.
2. Thiết kế thuật toán theo dõi chuyển động để máy bay duy trì khoảng cách và hướng bay phù hợp với người được theo.
3. Tích hợp hệ thống máy ảnh – xử lý ảnh – điều khiển bay tự động thành một hệ thống hoạt động ổn định trong môi trường thực tế.

4. Phương pháp nghiên cứu

Cách tiếp cận. Ứng dụng học sâu và thị giác máy tính để nhận dạng người, kết hợp điều khiển tự động để di chuyển máy bay theo mục tiêu. Mọi quyết định thiết kế được đưa ra dựa trên số đo trên phần cứng thật thay vì ước đoán.

Các phương pháp cụ thể.

- *Nghiên cứu tài liệu:* tổng hợp các công trình về phát hiện đối tượng nhỏ, độ bền của mạng nơ-ron trước ảnh suy giảm, lượng tử hoá mô hình, theo vết đa đối tượng và điều khiển UAV bằng thị giác.
- *Thực nghiệm có đối chứng:* mỗi thay đổi trong mô hình được so với một cấu hình đối chứng chỉ khác đúng một biến. Mọi mô hình, gồm cả 5 mô hình đối chứng công

khai, được chấm bằng cùng một chương trình đánh giá, với 3 hạt giống ngẫu nhiên và báo cáo trung bình $\pm$ độ lệch chuẩn.

- *Đo đạc trên phần cứng đích:* tốc độ, độ trễ từng chặng, nhiệt độ và độ chính xác sau lượng tử hoá được đo trực tiếp trên Raspberry Pi 5 gắn NPU Hailo-8L, không suy ra từ máy tính huấn luyện.
- *Kiểm thử phân tầng:* kiểm thử đơn vị cho các khối logic, kiểm thử trên bàn không cất cánh (chế độ `-no-fly`), kiểm tra liên lạc với bộ điều khiển bay thật, rồi mới bay thử ngoài bãi.

5. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: (i) các mô hình học sâu phát hiện người từ ảnh chụp trên cao; (ii) ảnh hưởng của rung động khung máy bay tới độ chính xác phát hiện; (iii) thuật toán theo vết và điều khiển bám theo người cho UAV nhiều cánh quạt chạy firmware ArduPilot.

Phạm vi nghiên cứu: theo thuyết minh, đề tài giới hạn trong không gian thoáng, tầm nhìn rõ, mục tiêu chính là một người di chuyển với tốc độ trung bình, độ cao bay thấp (khoảng 3–6 m). Đề tài không xét trường hợp nhiều người cần bám đồng thời hoặc điều kiện môi trường phức tạp như ánh sáng mạnh, gió lớn hay che khuất kéo dài. Khi có nhiều người trong khung hình, hệ thống chỉ khoá và bám một người.

6. Nội dung nghiên cứu và mức độ hoàn thành

Bảng 1 đối chiếu năm nội dung trong thuyết minh với sản phẩm thực tế và vị trí trình bày trong báo cáo.

Bảng 1. Đối chiếu nội dung thuyết minh với kết quả thực hiện

STT	Nội dung theo thuyết minh	Sản phẩm đạt được	Trình bày tại
1	Nghiên cứu tổng quan và thu thập dữ liệu huấn luyện	Tổng quan tài liệu; bộ dữ liệu chuẩn hoá một lớp *person*: 6.471 ảnh huấn luyện và 548 ảnh kiểm định VisDrone, 2.654 ảnh COCO-person, 92 khung bay thật gán nhãn tay	Mở đầu, Chương 1, 2
2	Huấn luyện và đánh giá mô hình nhận dạng người	Mô hình SteadyDet-N; AP@0,5 = 0,917 trên dữ liệu bay thật (chỉ tiêu $\geq 70\%$); 30 khung hình/giây đầu-cuối trên máy bay	Chương 3, 4, 5
3	Thiết kế hệ thống điều khiển máy bay tự động đi theo người	Chương trình điều khiển: OC-SORT, khoá mục tiêu, ước lượng khoảng cách, PID ba kênh, thang an toàn, MAVLink 30 Hz	Chương 6
4	Tích hợp và kiểm thử toàn hệ thống	Máy bay tích hợp Raspberry Pi 5 + Hailo-8L + camera + Pixhawk 6C; kiểm thử phân tầng, bay thử, chạy vòng bám người trên video bay thử	Chương 7
5	Viết báo cáo tổng kết	Báo cáo tổng kết, bài tóm tắt, poster, video bay thử	Toàn bộ hồ sơ

7. Bố cục báo cáo

Ngoài phần Mở đầu và Kết luận, báo cáo gồm bảy chương. Chương 1 trình bày cơ sở lý thuyết về phát hiện đối tượng, suy giảm ảnh do rung, lượng tử hoá, theo vết và điều khiển UAV. Chương 2 mô tả dữ liệu và giao thức đánh giá. Chương 3 trình bày kiến trúc SteadyDet-N và phương pháp huấn luyện nhận biết rung. Chương 4 báo cáo kết quả thực nghiệm của mô hình. Chương 5 trình bày quá trình triển khai lên thiết bị biên. Chương 6 trình bày hệ thống điều khiển bám người. Chương 7 trình bày tích hợp và kiểm thử toàn hệ thống.

CHƯƠNG 1

CƠ SỞ LÝ THUYẾT

Chương này trình bày các kiến thức nền tảng mà hệ thống dựa vào, đi theo đúng thứ tự dữ liệu chảy qua hệ thống: từ ảnh đầu vào, qua mô hình phát hiện, bộ theo vết, khối ước lượng khoảng cách, bộ điều khiển, tới bộ điều khiển bay. Mỗi mục chỉ giữ lại những gì cần để hiểu các lựa chọn thiết kế ở các chương sau.

1.1 Bài toán phát hiện đối tượng

1.1.1 Định nghĩa

Cho một ảnh $I \in \mathbb{R}^{H\times W\times 3}$, bài toán phát hiện đối tượng yêu cầu xuất ra một tập bounding box $\{(b_i, c_i, s_i)\}$, trong đó $b_i = (x_1, y_1, x_2, y_2)$ là toạ độ hộp, c_i là lớp và $s_i \in [0,1]$ là điểm tin cậy. Đề tài chỉ quan tâm một lớp là *người*, nên bài toán thu về việc định vị chính xác từng người trong ảnh.

Mức độ trùng khớp giữa hộp dự đoán B_p và hộp thật B_g được đo bằng tỷ lệ giao trên hợp:

$$\mathrm{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \tag{1.1}$$

Một dự đoán được xem là đúng khi IoU với một hộp thật chưa được ghép vượt ngưỡng τ, thường là 0,5.

1.1.2 Loại bỏ hộp không cực đại

Các bộ phát hiện dày đặc sinh ra nhiều hộp chồng lên nhau cho cùng một đối tượng. Thuật toán loại bỏ hộp không cực đại (NMS) sắp xếp các hộp theo điểm tin cậy, giữ hộp cao nhất và xoá các hộp có IoU với nó vượt ngưỡng, rồi lặp lại. Biến thể DIoU-NMS [24] bổ sung khoảng cách tâm vào tiêu chí xoá, giúp giữ lại hai người đứng sát nhau. Chi phí NMS phụ thuộc số hộp, nên trong cảnh đông người, NMS là chặng có độ trễ biến động nhất của chuỗi xử lý.

1.2 Mạng nơ-ron tích chập cho phát hiện đối tượng

1.2.1 Bộ phát hiện hai giai đoạn và một giai đoạn

Học sâu thay thế các đặc trưng thiết kế thủ công bằng đặc trưng học từ dữ liệu [25, 26]. Mạng tích chập (CNN) với kết nối tắt [27] là backbone của phần lớn bộ phát hiện hiện nay. Họ bộ phát hiện hai giai đoạn, tiêu biểu là Faster R-CNN [28], trước tiên đề xuất các vùng có khả năng chứa đối tượng rồi mới phân loại và tinh chỉnh từng vùng; cách này chính xác nhưng chậm. Họ một giai đoạn như SSD [29] và YOLO [12] dự đoán trực tiếp lớp và hộp trên một lưới đặc trưng trong một lần truyền, phù hợp với ứng dụng thời gian thực. RetinaNet [30] chỉ ra rằng sự mất cân bằng giữa rất nhiều ô nền và rất ít ô chứa đối tượng là nguyên nhân chính khiến bộ phát hiện một giai đoạn kém chính xác, và đề xuất hàm mất mát tiêu điểm để khắc phục.

1.2.2 Kim tự tháp đặc trưng

Đối tượng xuất hiện ở nhiều kích thước. Mạng kim tự tháp đặc trưng (FPN) [31] kết hợp feature map sâu (giàu ngữ nghĩa, độ phân giải thấp) với bản đồ nông (chi tiết, độ phân giải cao) qua một đường từ trên xuống, rồi dự đoán ở nhiều mức. Ký hiệu P_k chỉ feature map có bước (stride) 2^k so với ảnh vào; với ảnh 640×640, P_3 có kích thước 80×80, còn P_2 có kích thước 160×160. EfficientDet [32] mở rộng FPN thành cấu trúc hai chiều có trọng số.

1.2.3 Head anchor-free

Các bộ phát hiện đời đầu dựa vào tập anchor box định sẵn. FCOS [33] và các thế hệ YOLO gần đây dự đoán trực tiếp khoảng cách từ mỗi grid cell tới bốn cạnh hộp, bỏ hẳn anchor box và giảm số siêu tham số phải chỉnh. Việc ghép grid cell với hộp thật trong huấn luyện được thực hiện bằng các chiến lược gán nhãn động như TOOD [34], trong đó mức độ phù hợp của một ô được đo bằng tích giữa điểm phân loại và IoU của hộp nó dự đoán.

1.2.4 Họ YOLO và bộ phát hiện dựa trên Transformer

Họ YOLO liên tục được cải tiến qua các phiên bản YOLOv7 [13], YOLOv8, YOLO11 và YOLO26 của Ultralytics [15]. YOLOv10 [14] huấn luyện song song hai head, một head "one-to-many" cho tín hiệu học phong phú và một head "one-to-one" cho phép bỏ NMS

khi suy luận. DETR [35] coi phát hiện là bài toán dự đoán tập hợp bằng Transformer; RT-DETR [16] là phiên bản thời gian thực đầu tiên của hướng này. Đề tài dùng bốn mô hình YOLO bản nano và RT-DETR-l làm đối chứng, phủ ba trường phái: CNN có NMS, CNN không NMS và Transformer.

1.3 Các khối xây dựng mạng hiệu quả

1.3.1 Tích chập depthwise separable

Một lớp tích chập chuẩn kích thước $k \times k$ từ C_{in} sang C_{out} kênh trên bản đồ $H \times W$ tốn khoảng $k^2 C_{in} C_{out} HW$ phép nhân. Tích chập depthwise separable tách lớp này thành một tích chập $k \times k$ riêng cho từng kênh và một tích chập 1×1 trộn kênh, tổng chi phí khoảng $(k^2 C_{in} + C_{in} C_{out})HW$, tức giảm gần k^2 lần khi C_{out} lớn. MobileNetV2 [36] xây dựng khối nút cổ chai đảo ngược (mở rộng kênh, tích chập depthwise, thu hẹp kênh). MobileNetV4 [37] tổng quát hoá thành khối UIB, cho phép thêm hoặc bỏ tích chập depthwise ở đầu và giữa khối.

Công thức chi phí trên cho thấy một điểm quan trọng: *chi phí tỷ lệ thuận với diện tích feature map*. Một lớp đặt ở P_2 (160×160) tốn gấp 16 lần cùng lớp đó đặt ở P_3 (80×80) và gấp 64 lần ở P_4. Đây là cơ sở của thiết kế "nhánh P_2 mỏng" ở Chương 3.

1.3.2 Kết nối từng phần liên tầng và tái tham số hoá

CSPNet [38] chia feature map thành hai phần, chỉ cho một phần đi qua chuỗi khối tính toán rồi ghép lại, nhờ đó giảm chi phí mà vẫn giữ luồng gradient phong phú. RepVGG [39] dùng nhiều nhánh khi huấn luyện và gộp chúng thành một tích chập duy nhất khi suy luận; phép gộp lớp tích chập với lớp batch norm cũng được dùng khi xuất mô hình để triển khai.

1.3.3 Self-attention theo cửa sổ

Cơ chế self-attention cho phép mỗi vị trí tổng hợp thông tin từ mọi vị trí khác, nhưng chi phí tăng theo bình phương số vị trí. Swin Transformer [40] giới hạn phép chú ý trong các cửa sổ cục bộ để chi phí tăng tuyến tính. Đặt khối này ở mức phân giải thấp nhất (P_5, 20×20 với ảnh 640) giúp mô hình có ngữ cảnh rộng với chi phí rất nhỏ. LSKNet [41] là một hướng khác, dùng nhân tích chập lớn có chọn lọc cho ảnh viễn thám.

1.4 Phát hiện người có kích thước nhỏ từ UAV

Trong ảnh chụp từ độ cao vài chục mét, một người thường chiếm ít hơn 32×32 điểm ảnh, thuộc nhóm "vật thể nhỏ" theo quy ước của COCO [42]. Các khảo sát [43, 44] chỉ ra ba khó khăn chính: đặc trưng của vật nhỏ bị mất sau các lớp giảm mẫu, IoU rất nhạy với sai lệch vài điểm ảnh, và mất cân bằng mẫu dương trong gán nhãn.

1.4.1 Giữ mức phân giải cao

Với bước 8, một người rộng 8 điểm ảnh chỉ còn đúng một ô trên bản đồ P_3, và thông tin bị trộn với nền xung quanh. Thêm mức dự đoán P_2 (bước 4) giúp giữ lại tín hiệu này nhưng làm tăng mạnh chi phí tính toán như đã phân tích ở mục trên. Phần lớn các mô hình YOLO bản nano bắt head từ P_3.

1.4.2 Khoảng cách Wasserstein chuẩn hoá

Với hộp nhỏ, lệch một hai điểm ảnh làm IoU giảm rất mạnh, khiến gradient không ổn định. Wang và cộng sự [45] mô hình hoá mỗi hộp (c_x, c_y, w, h) như một phân phối Gauss hai chiều và đo khoảng cách Wasserstein bậc hai giữa chúng:

$$W_2^2 = (c_x^p - c_x^g)^2 + (c_y^p - c_y^g)^2 + \left(\tfrac{w^p - w^g}{2}\right)^2 + \left(\tfrac{h^p - h^g}{2}\right)^2,$$

$$\mathrm{NWD} = \exp\left(-\frac{\sqrt{W_2^2}}{C}\right), \tag{1.2}$$

trong đó C là hằng số chuẩn hoá. NWD thay đổi trơn theo sai lệch vị trí kể cả khi hai hộp không giao nhau, nên phù hợp với vật nhỏ.

1.4.3 Hàm mất mát hồi quy CIoU và phân loại theo chất lượng

CIoU [24] bổ sung vào IoU hai thành phần phạt khoảng cách tâm và sai khác tỷ lệ khung hình:

$$\mathscr{L}_{\mathrm{CIoU}} = 1 - \mathrm{IoU} + \frac{\rho^2(\mathbf{c}_p, \mathbf{c}_g)}{d^2} + \alpha v, \tag{1.3}$$

với ρ là khoảng cách Euclid giữa hai tâm, d là đường chéo hộp nhỏ nhất bao cả hai hộp, v đo sai khác tỷ lệ khung hình và α là hệ số cân bằng. Ở nhánh phân loại, hàm mất mát tiêu điểm theo chất lượng (QFL) [46] dùng IoU của hộp dự đoán làm nhãn mềm thay

cho nhãn cứng 0/1:

$$\mathscr{L}_{\mathrm{QFL}}(p,q) = -\,|q-p|^{\beta}\,\big[(1-q)\log(1-p) + q\log p\big], \tag{1.4}$$

trong đó p là xác suất dự đoán, q là IoU mục tiêu và β điều chỉnh mức tập trung vào mẫu khó. Nhờ vậy điểm tin cậy phản ánh cả khả năng có người lẫn độ khít của hộp.

1.5 Suy giảm ảnh do rung động của UAV

1.5.1 Nguồn gốc vật lý

Động cơ và cánh quạt mất cân bằng, cùng các tần số cộng hưởng của khung, truyền dao động tần số cao tới camera. Cảm biến ảnh tích luỹ ánh sáng trong suốt thời gian phơi sáng t_e; nếu camera dịch chuyển trong khoảng thời gian này, mỗi điểm của cảnh bị trải dài theo quỹ đạo chuyển động. Với chuyển động gần như thẳng trong thời gian phơi sáng ngắn, ảnh thu được có thể mô hình hoá bằng phép tích chập:

$$I_{\mathrm{rung}} = \gamma \cdot \big(I_{\mathrm{sạch}} * k_{L,\theta}\big) + n, \qquad n \sim \mathscr{N}(0,\sigma^2), \tag{1.5}$$

trong đó $k_{L,\theta}$ là nhân nhoè dạng đoạn thẳng dài L điểm ảnh theo hướng θ, được chuẩn hoá để tổng bằng 1; n là nhiễu cảm biến; γ là hệ số độ sáng. Ba hiệu ứng xuất hiện cùng lúc: *nhoè có hướng* do dịch chuyển trong lúc phơi sáng; *nhiễu tăng* vì khi ảnh tối và nhoè, bộ tự phơi sáng tăng độ khuếch đại; *dao động độ sáng giữa các khung* do tần số cánh quạt giao thoa với màn trập cuộn. Độ dài nhoè L tỷ lệ với biên độ vận tốc góc và thời gian phơi sáng, nên rút ngắn t_e làm giảm nhoè nhưng làm tăng nhiễu.

1.5.2 Độ bền của mạng nơ-ron trước ảnh suy giảm

Dodge và Karam [20] cho thấy các mạng phân loại giảm mạnh độ chính xác khi ảnh bị nhoè và nhiễu dù con người vẫn nhận ra nội dung. Hendrycks và Dietterich [18] đề xuất bộ ImageNet-C gồm 15 loại suy giảm ở 5 mức và chỉ số lỗi trung bình trên các mức; Michaelis và cộng sự [19] mở rộng cho bài toán phát hiện đối tượng. Các benchmark này bao quát nhiều loại suy giảm chung, còn trên UAV phân phối suy giảm tập trung hơn, do động học cánh quạt và thời gian phơi sáng quyết định. Có hai hướng xử lý chính: khôi phục ảnh bằng mạng khử nhoè [47, 48], vốn thêm một mạng nữa vào đường xử lý và làm tăng độ trễ; hoặc làm cho chính bộ phát hiện bền hơn qua dữ liệu huấn luyện [49]. Đề tài

chọn hướng thứ hai vì nó không tốn chi phí khi suy luận.

1.6 Lượng tử hoá mô hình và bộ tăng tốc nơ-ron

1.6.1 Lượng tử hoá INT8

Lượng tử hoá biểu diễn trọng số và giá trị kích hoạt bằng số nguyên 8 bit thay cho số thực 32 bit, giảm 4 lần bộ nhớ và cho phép dùng phép tính nguyên trên phần cứng chuyên dụng. Phép lượng tử hoá affine [50] ánh xạ số thực r sang số nguyên q:

$$q = \mathrm{clamp}\Big(\mathrm{round}\Big(\frac{r}{S}\Big) + Z,\, 0,\, 255\Big), \qquad r \approx S\,(q - Z), \tag{1.6}$$

với S là hệ số tỷ lệ và Z là điểm không. Lượng tử hoá sau huấn luyện (PTQ) [51] ước lượng S và Z từ một tập ảnh hiệu chuẩn nhỏ; các kỹ thuật như làm tròn thích nghi [52] giảm sai số làm tròn. Sai số lượng tử $S/2$ trên toạ độ hộp ảnh hưởng lớn tới vật nhỏ, vì với vật chỉ vài điểm ảnh, sai lệch này chiếm tỷ lệ đáng kể so với kích thước vật.

1.6.2 Bộ tăng tốc Hailo-8L

Hailo-8L là bộ xử lý nơ-ron dạng M.2, công suất tính toán danh định 13 TOPS, được Raspberry Pi phân phối kèm bộ AI Kit cho Raspberry Pi 5 [53]. Mô hình ở định dạng ONNX được trình biên dịch luồng dữ liệu (DFC) của Hailo phân tích, lượng tử hoá với tập hiệu chuẩn rồi biên dịch thành tệp HEF. Khi mô hình không vừa bộ nhớ trên chip, trình biên dịch chia nó thành nhiều *context*; mỗi lần chuyển context, trọng số phải được nạp lại qua đường PCIe. Vì vậy, trên loại chip này, số context là một yếu tố quyết định tốc độ bên cạnh số phép tính.

1.7 Theo vết đa đối tượng

1.7.1 Bộ lọc Kalman

Bộ lọc Kalman [54] ước lượng trạng thái $\mathbf{x}$ của một hệ tuyến tính có nhiễu qua hai bước. Bước dự đoán:

$$\hat{\mathbf{x}}_{k|k-1} = F\,\hat{\mathbf{x}}_{k-1}, \qquad P_{k|k-1} = F P_{k-1} F^{\top} + Q. \tag{1.7}$$

Bước cập nhật khi có quan sát $\mathbf{z}_k$:

$$\begin{aligned} K_k &= P_{k|k-1}H^\top \big(HP_{k|k-1}H^\top + R\big)^{-1}, \\ \hat{\mathbf{x}}_k &= \hat{\mathbf{x}}_{k|k-1} + K_k\big(\mathbf{z}_k - H\hat{\mathbf{x}}_{k|k-1}\big), \qquad P_k = (I - K_k H)P_{k|k-1}. \end{aligned} \tag{1.8}$$

Trong đó F là ma trận chuyển trạng thái, H là ma trận quan sát, Q và R là hiệp phương sai nhiễu quá trình và nhiễu đo. Khi không có quan sát, chỉ bước dự đoán được thực hiện, cho phép "đi tiếp" theo quỹ đạo trong thời gian ngắn.

1.7.2 Từ SORT tới OC-SORT

SORT [55] theo vết mỗi đối tượng bằng một bộ lọc Kalman với trạng thái gồm tâm, diện tích, tỷ lệ khung hình và vận tốc, rồi ghép dự đoán với phát hiện mới bằng thuật toán Hungary trên ma trận chi phí $1 - \text{IoU}$. DeepSORT [56] thêm đặc trưng ngoại hình; ByteTrack [57] tận dụng cả các phát hiện có điểm thấp. OC-SORT [58] chỉ ra rằng khi đối tượng bị mất quan sát vài khung, lỗi của bộ lọc Kalman tích luỹ theo thời gian; nó khắc phục bằng cách dùng lại các quan sát thật để hiệu chỉnh trạng thái khi đối tượng xuất hiện trở lại, và thêm tiêu chí nhất quán hướng chuyển động khi ghép. OC-SORT không cần mạng trích đặc trưng ngoại hình, nên chạy nhẹ trên CPU, phù hợp với máy tính nhúng.

1.8 Mô hình camera và ước lượng khoảng cách

Mô hình camera lỗ kim ánh xạ điểm (X, Y, Z) trong hệ toạ độ camera thành điểm ảnh (u, v):

$$u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y, \tag{1.9}$$

với f_x, f_y là tiêu cự tính theo điểm ảnh và (c_x, c_y) là tâm ảnh, xác định bằng hiệu chuẩn bàn cờ [59]. Nếu một người có chiều cao thực H_r và chiều cao bounding box trên ảnh là h điểm ảnh, khoảng cách tới người đó xấp xỉ:

$$Z \approx \frac{f_y H_r}{h}. \tag{1.10}$$

Đạo hàm theo h cho $|\partial Z/\partial h| = Z^2/(f_y H_r)$: sai số tăng theo bình phương khoảng cách, vì ở xa mỗi điểm ảnh sai lệch tương ứng nhiều mét hơn. Tương tự, độ lệch góc phương vị của người so với trục quang học được tính từ độ lệch ngang của tâm hộp: $\psi \approx (u - c_x)/f_x$.

1.9 Điều khiển servo thị giác và bộ điều khiển PID

1.9.1 Servo thị giác

Điều khiển servo thị giác dùng thông tin từ ảnh để tạo lệnh chuyển động [60]. Có hai trường phái: servo dựa trên ảnh (IBVS) điều khiển trực tiếp sai số trong mặt phẳng ảnh; servo dựa trên vị trí (PBVS) ước lượng vị trí ba chiều của mục tiêu rồi điều khiển trong không gian. Các hệ thống lai kết hợp cả hai, ví dụ dùng sai số ngang trong ảnh để quay hướng và khoảng cách ước lượng để tiến lùi.

1.9.2 Bộ điều khiển PID

Bộ điều khiển PID [61] tính tín hiệu điều khiển từ sai số $e(t)$ giữa giá trị đặt và giá trị đo:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}. \tag{1.11}$$

Thành phần tỷ lệ phản ứng với sai số hiện tại, tích phân triệt tiêu sai số xác lập, vi phân giảm vọt lố. Khi tín hiệu ra bị giới hạn, cần chặn thành phần tích phân (chống bão hoà tích phân) để tránh vọt lố lớn khi sai số đổi dấu. Trong hệ rời rạc, tích phân và vi phân được xấp xỉ bằng tổng và sai phân theo bước thời gian thực đo được giữa hai lần gọi.

1.10 Bộ điều khiển bay ArduPilot và giao thức MAVLink

ArduPilot là phần mềm điều khiển bay mã nguồn mở, hỗ trợ nhiều loại phương tiện, trong đó ArduCopter dành cho máy bay nhiều cánh quạt [62]. ArduCopter có nhiều chế độ bay: `STABILIZE` (người lái điều khiển tư thế), `LOITER` (giữ vị trí bằng GPS), `RTL` (tự bay về điểm cất cánh, trước tiên leo lên độ cao an toàn) và `GUIDED`, chế độ nhận lệnh vị trí hoặc vận tốc từ máy tính bên ngoài. Bộ điều khiển bay tự đảm nhận vòng ổn định tư thế và vận tốc bên trong với tần số cao, còn máy tính đồng hành chỉ cần gửi vận tốc mong muốn.

MAVLink là giao thức thông điệp gọn nhẹ dùng phổ biến giữa UAV, trạm mặt đất và máy tính đồng hành [63]. Mỗi gói có tiêu đề, định danh hệ thống/thành phần, mã thông điệp, dữ liệu và mã kiểm tra. Các thông điệp dùng trong đề tài gồm `HEARTBEAT` (trạng thái sống và chế độ), `ATTITUDE`, `GLOBAL_POSITION_INT`, `SYS_STATUS`, `VIBRATION` (mức rung ba trục và số lần bão hoà cảm biến gia tốc) và `SET_POSITION_TARGET_LOCAL_NED` để gửi vận tốc đặt. ArduPilot có tham số `GUID_TIMEOUT`: nếu không nhận lệnh mới trong

khoảng thời gian này, máy bay tự dừng lại, là một lớp bảo vệ khi máy tính đồng hành bị treo.

1.11 Các chỉ số đánh giá

Độ chuẩn xác, recall và F1. Với TP, FP, FN là số dự đoán đúng, sai và số người bị bỏ sót:

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}. \quad (1.12)$$

Độ chính xác trung bình (AP). Khi thay đổi ngưỡng điểm tin cậy, ta được đường cong precision theo recall; AP là diện tích dưới đường cong này (nội suy 101 điểm theo chuẩn COCO) [42, 64]. AP@0,5 dùng ngưỡng IoU 0,5; AP@0,75 dùng ngưỡng 0,75 và phản ánh độ khít của hộp; AP@[.50:.95] là trung bình trên 10 ngưỡng IoU từ 0,50 tới 0,95. AR@100 là recall trung bình khi giữ tối đa 100 dự đoán mỗi ảnh. AP_s là AP trên vật có diện tích dưới 32^2 điểm ảnh.

Độ chính xác khi bị suy giảm. Theo cách tiếp cận của [18], đề tài định nghĩa mCP là AP@[.50:.95] trung bình trên M mức suy giảm và tỷ lệ giữ độ chính xác:

$$\text{mCP} = \frac{1}{M}\sum_{m=1}^{M}\text{AP}_m, \qquad \text{retain} = 100\times\frac{\text{mCP}}{\text{AP}_{\text{sạch}}}\ (\%). \quad (1.13)$$

mCP cho biết mô hình chính xác bao nhiêu *khi đang bay*, còn retain cho biết mô hình giữ lại được bao nhiêu phần độ chính xác so với khi đứng yên. Hai mô hình có cùng AP sạch có thể khác nhau rất xa về retain.

Khoảng tin cậy bootstrap. Khi tập đánh giá nhỏ, chênh lệch giữa hai mô hình có thể chỉ do ngẫu nhiên. Phương pháp bootstrap [65] lấy mẫu lại có hoàn lại các khung hình nhiều lần, tính chỉ số trên mỗi mẫu và lấy phân vị 2,5% và 97,5% làm khoảng tin cậy 95%. Khi lấy mẫu ghép cặp (cùng một tập khung cho cả hai mô hình), khoảng tin cậy của *hiệu* hai chỉ số không chứa 0 nghĩa là chênh lệch có ý nghĩa trên tập dữ liệu đó.

1.12 Kết luận chương

Chương này đã trình bày nền tảng về bộ phát hiện một giai đoạn, các khối mạng hiệu quả và các khó khăn khi phát hiện người nhỏ; mô hình vật lý của suy giảm ảnh do rung;

lượng tử hoá INT8 và đặc điểm context của NPU Hailo; bộ lọc Kalman và OC-SORT; mô hình camera lỗ kim; bộ điều khiển PID; ArduPilot và MAVLink; cùng các chỉ số đánh giá. Hai nhận xét định hướng cho các chương sau: chi phí tính toán tỷ lệ với diện tích feature map, và các benchmark phát hiện thông thường không đo độ chính xác trong điều kiện rung mà UAV luôn gặp phải.

CHƯƠNG 2

DỮ LIỆU VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

Một kết quả thực nghiệm chỉ có giá trị khi dữ liệu phản ánh đúng điều kiện triển khai và cách đo không thiên vị mô hình nào. Chương này mô tả ba nguồn dữ liệu của đề tài, quy trình chuẩn hoá, giao thức mô phỏng rung VisDrone-Shake và các quy tắc đo mà mọi thí nghiệm ở Chương 4 tuân theo.

2.1 Yêu cầu đối với dữ liệu

Bài toán có hai miền dữ liệu khác nhau rõ rệt. Miền thứ nhất là ảnh chụp từ độ cao lớn, nơi người rất nhỏ; đây là miền của các benchmark công khai, dùng để so sánh công bằng với các công trình khác. Miền thứ hai là miền triển khai thật của đề tài: máy bay bay thấp khoảng 3–6 m và bám theo một người ở cự ly vài mét, nên người chiếm một phần đáng kể của khung hình. Thống kê trên dữ liệu thực tế (Bảng 2.1) cho thấy trung vị diện tích bounding box trên ảnh 640×640 là khoảng 122 điểm ảnh vuông ở VisDrone nhưng khoảng 7.874 điểm ảnh vuông ở dữ liệu bay thật, tức lớn hơn khoảng 64 lần. Vì vậy, đề tài dùng đồng thời dữ liệu của cả hai miền và luôn đánh giá riêng trên từng miền.



Hình 2.1. Ba nguồn dữ liệu của đề tài và nhãn bounding box tương ứng: VisDrone-person chụp từ trên cao (trái), COCO-person ở cự ly gần (giữa) và khung hình bay thật ở độ cao khoảng 6 m (phải)

2.2 Các bộ dữ liệu

2.2.1 VisDrone-person

VisDrone2019-DET [2] là bộ dữ liệu phát hiện đối tượng từ UAV do Đại học Thiên Tân (Trung Quốc) công bố, chụp ở nhiều thành phố, độ cao và điều kiện ánh sáng, gồm

mười lớp đối tượng. Đề tài chuyển bộ dữ liệu về bài toán một lớp bằng cách gộp hai lớp `pedestrian` (người đang đi hoặc đứng) và `people` (người ở tư thế khác như ngồi) thành lớp *person*, vì hệ thống bám theo phải nhận ra người ở mọi tư thế. Các hộp có cạnh nhỏ hơn 10 điểm ảnh bị loại vì gần như không thể gán nhãn nhất quán. Nhãn được chuyển sang định dạng YOLO (tâm, rộng, cao chuẩn hoá về $[0, 1]$). Kết quả gồm 6.471 ảnh huấn luyện với 65.523 hộp và 548 ảnh kiểm định với 8.165 hộp. Trong tập kiểm định, 98,7% số hộp có diện tích dưới 32^2 điểm ảnh sau khi đưa ảnh về 640×640, nên AP trên tập này gần như trùng với AP của vật nhỏ.

2.2.2 COCO-person

COCO [42] chứa ảnh đời thường với người ở cự ly gần và nhiều tư thế. Đề tài lấy các chú thích lớp *person* từ tập val2017, bỏ những hộp có diện tích dưới 100 điểm ảnh vuông, được 2.654 ảnh với 9.783 hộp. Tập này bổ sung cho mô hình kinh nghiệm về người kích thước lớn mà VisDrone gần như không có. Khi dùng, mỗi epoch trộn thêm một lượng ảnh COCO bằng 25% số ảnh VisDrone (1.618 ảnh).

2.2.3 Tập bay thật flight_val

Tập này được xây dựng từ chuyến bay thử ngày 21/07/2026 trên một bãi nền bê tông. Máy bay bay ở chế độ giữ vị trí, độ cao khoảng 6 m, camera nhìn xuống, một người đi bộ bên dưới. Video được ghi từ màn hình trạm mặt đất, vùng giao diện được cắt bỏ, và một khung hình được lấy mỗi giây trong đoạn bay ổn định từ giây thứ 18 tới giây thứ 150. Nhãn được khởi tạo bằng mô hình rồi sửa tay từng khung trên công cụ Roboflow theo bộ quy tắc: hộp ôm sát toàn thân nhìn thấy, xoá mọi hộp sai (bóng, vết bẩn nền, biểu tượng giao diện), bổ sung người bị bỏ sót. Sau khi rà soát còn 92 khung hình 1174×654, mỗi khung có đúng một người. Tập này *không* được dùng để huấn luyện, chọn mô hình hay hiệu chuẩn lượng tử hoá; nó chỉ dùng để đánh giá, nhằm tránh rò rỉ dữ liệu.

Bảng 2.1. Thống kê ba bộ dữ liệu sau chuẩn hoá (lớp duy nhất *person*)

Bộ dữ liệu	**Số ảnh**	**Số hộp**	**Trung vị diện tích hộp @640 (px^2)**	**Vai trò**
VisDrone-person huấn luyện	6.471	65.523	–	Huấn luyện
VisDrone-person kiểm định	548	8.165	122	Chọn mô hình, so sánh, đánh giá rung
COCO-person	2.654	9.783	–	Bổ sung người cự ly gần (nhánh triển khai)
flight_val (bay thật ≈6 m)	92	92	7.874	Chỉ đánh giá trên miền triển khai

2.3 Hai nhánh mô hình

Để vừa so sánh công bằng với các công trình khác, vừa phục vụ triển khai thật, đề tài duy trì hai nhánh huấn luyện độc lập dùng chung một kiến trúc:

- **Nhánh chuẩn:** chỉ huấn luyện trên VisDrone-person, không thêm dữ liệu ngoài. Mọi bảng so sánh với mô hình đối chứng dùng nhánh này, để chênh lệch không đến từ dữ liệu bổ sung.
- **Nhánh triển khai:** huấn luyện trên VisDrone-person trộn COCO-person, nhắm tới người ở cự ly gần. Mô hình chạy trên máy bay thuộc nhánh này.

2.4 Tiền xử lý và data augmentation

Ảnh được co giãn giữ nguyên tỷ lệ về cạnh dài 640 điểm ảnh và đệm phần thiếu bằng màu xám (letterbox), toạ độ nhãn được biến đổi tương ứng. Trong huấn luyện, đề tài dùng các phép tăng cường chuẩn: ghép bốn ảnh (xác suất 0,5, tắt ở các epoch cuối để mô hình quen với ảnh đơn), trộn hai ảnh (0,1), lật ngang (0,5), biến đổi affine, thay đổi màu, sao chép dán đối tượng (0,4), một số phép biến đổi đặc thù ảnh UAV (0,3), và huấn luyện đa tỷ lệ với kích thước ảnh ngẫu nhiên từ 50% tới 100% của 640, bước 32 điểm ảnh. Các phép này đều là những biến đổi mà dữ liệu gốc đã phần nào chứa đựng; không

phép nào mô phỏng rung của khung máy bay. Suy giảm do rung được đưa vào riêng ở Chương 3.

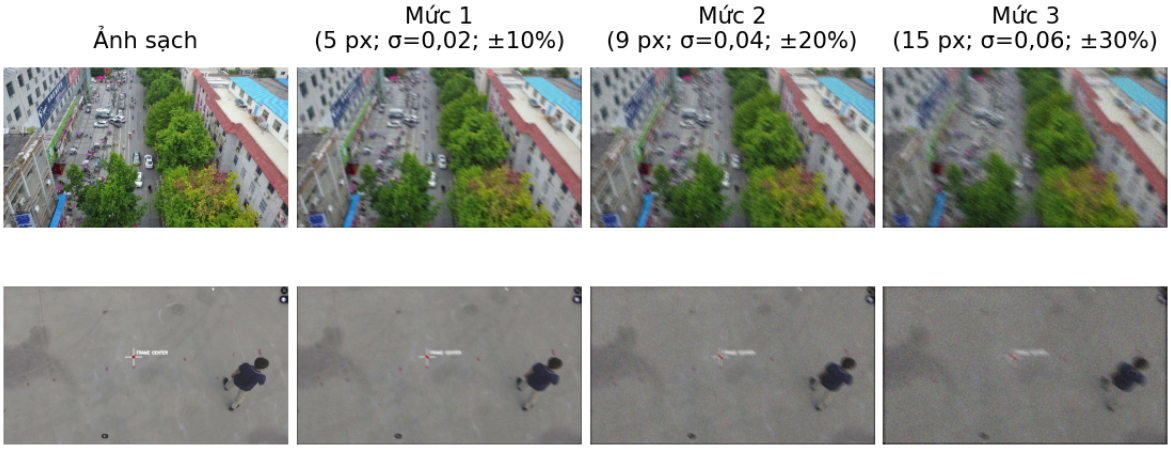
2.5 Giao thức mô phỏng rung VisDrone-Shake

2.5.1 Mô hình suy giảm

Giao thức hiện thực phương trình (1.5) trên ảnh đã chuẩn hoá về $[0,1]$ theo ba bước: (1) tích chập theo từng kênh màu với nhân nhoè dạng đoạn thẳng dài L điểm ảnh, hướng θ lấy ngẫu nhiên đều trong $[0^\circ, 180^\circ)$, nhân được chuẩn hoá tổng bằng 1; (2) cộng nhiễu Gauss độ lệch chuẩn σ; (3) nhân toàn ảnh với hệ số $1+u$, u lấy đều trong $[-b,b]$. Ba mức suy giảm được định nghĩa trong Bảng 2.2 và minh hoạ ở Hình 2.2. Mức 1 và mức 2 tương ứng máy bay cân bằng tốt đang bay ổn định hoặc bay tiến; mức 3 tương ứng kích thích mạnh như cánh quạt mất cân bằng.

Bảng 2.2. Tham số ba mức rung của giao thức VisDrone-Shake

Mức	**Độ dài nhoè L (px)**	**Nhiễu σ**	**Dao động sáng b**	**Tình huống tương ứng**
Mức 1	5	0,02	$\pm 10\%$	Bay treo, khung cân bằng tốt
Mức 2	9	0,04	$\pm 20\%$	Bay tiến thông thường
Mức 3	15	0,06	$\pm 30\%$	Kích thích mạnh (cánh quạt lệch)



Hình 2.2. Minh hoạ ba mức rung VisDrone-Shake trên một ảnh VisDrone (hàng trên) và một khung bay thật (hàng dưới); hướng nhoè cố định 35° để dễ so sánh

Điểm quan trọng là không bước nào trong ba bước làm dịch chuyển đối tượng: nhoè đối xứng quanh tâm, nhiễu và độ sáng không đổi hình học. Vì vậy bounding box gốc vẫn

hợp lệ trên ảnh đã suy giảm, và cùng một toán tử có thể dùng cho cả đánh giá lẫn huấn luyện mà không phải sửa nhãn.

2.5.2 Giao thức phải được cố định seed ngẫu nhiên

Hướng nhoè, nhiễu và độ sáng đều được rút ngẫu nhiên, nên một lần chạy đánh giá chỉ là một lần rút thăm. Trong quá trình làm đề tài, nhóm đã đo cùng một checkpoint ba lần mà không cố định hạt giống và nhận được tỷ lệ retain 17,0%, 17,1% và 17,6%, tức dao động hơn nửa điểm phần trăm, đủ để tạo ra một "cải tiến" giả. Giao thức chính thức vì thế gieo lại hạt giống trước *mỗi* lượt đánh giá ở mỗi mức, chạy với ba hạt giống 0, 1, 2 và báo cáo trung bình $\pm$ độ lệch chuẩn mẫu. Đánh giá trên ảnh sạch là tất định nên chỉ chạy một lần. Chỉ số tổng hợp là mCP và retain theo phương trình (1.13) với $M = 3$.

2.6 Một chương trình đo duy nhất cho mọi mô hình

Mỗi họ mô hình đi kèm chương trình đánh giá riêng với ngưỡng tin cậy, số hộp tối đa và quy ước diện tích khác nhau. Khi đo lại YOLOv8-n bằng chương trình của Ultralytics, nhóm được AP@[.50:.95] = 0,2787; khi đo bằng chương trình chung của đề tài, được 0,2684. Chênh lệch 0,01 này cùng cỡ với khoảng cách giữa các mô hình đối chứng với nhau, nên nếu trộn hai chương trình đo trong cùng một bảng, người đọc sẽ nhầm khác biệt do cách đo thành khác biệt giữa các mô hình. Với RT-DETR-l chênh lệch còn lớn hơn (0,3340 so với 0,2502), vì bộ tiền xử lý gốc của nó kéo giãn ảnh cho vuông thay vì đệm xám.

Do đó, **mọi mô hình trong mọi bảng của Chương 4** được đo bằng một chương trình duy nhất (`paper_eval_unified.py`) với cùng cấu hình:

- cùng bộ nạp ảnh letterbox 640×640, mọi mô hình nhận đúng cùng một tensor đầu vào;
- ngưỡng tin cậy 0,01, NMS với ngưỡng IoU 0,5, tối đa 300 hộp mỗi ảnh;
- chấm điểm bằng thư viện `pycocotools` trên cùng một tệp nhãn chuẩn;
- ba hạt giống cho mọi lượt đánh giá có rung.

Chỉ số retain không bị ảnh hưởng bởi cách tiền xử lý của từng mô hình vì nó là tỷ số giữa hai lượt đo trong cùng một đường xử lý; tuy vậy, đề tài vẫn ghi chú rõ trường hợp RT-DETR-l trong bảng kết quả.

2.7 Quy tắc chọn checkpoint

Mỗi lần huấn luyện sinh ra checkpoint tốt nhất theo AP kiểm định trên ảnh sạch (`best.pt`) và checkpoint cuối (`last.pt`). Với SteadyDet-W được huấn luyện có ràng buộc nhất quán, `last.pt` cho retain 33,4% còn `best.pt` cho 30,5%. Nếu lấy `last.pt` cho mô hình của mình và `best.pt` cho mô hình đối chứng thì đó là hai quy tắc khác nhau và kết quả bị thiên vị. Đề tài áp dụng một quy tắc duy nhất: **mọi mô hình dùng checkpoint tốt nhất theo AP sạch**, không checkpoint nào được chọn dựa trên chính giao thức rung dùng để đánh giá nó.

2.8 Đánh giá trên miền triển khai

Trên tập flight_val, các mô hình được đánh giá bằng hai cách bổ trợ. Cách thứ nhất dùng cùng hàm `pycocotools` để tính AP@0,5, AP@[.50:.95], AP@0,75 và AR@100. Vì chỉ có 92 khung, đề tài lấy mẫu lại khung hình 1.000 lần theo phương pháp bootstrap ghép cặp để tính khoảng tin cậy 95% cho hiệu giữa hai mô hình. Cách thứ hai sát với vận hành hơn: chạy mô hình một lần ở ngưỡng tin cậy thấp 0,02 rồi quét ngưỡng ngoại tuyến, tại mỗi ngưỡng ghép dự đoán với nhãn bằng IoU $\geq 0{,}5$ để tính precision, recall, F1 và số hộp sai mỗi khung. Ngưỡng cho F1 cao nhất được chọn làm ngưỡng vận hành trên máy bay.

Khi so sánh mô hình FP32 và INT8, cả hai được xuất dự đoán ra cùng một định dạng và chấm bằng cùng một hàm, để chênh lệch chỉ phản ánh tác động của lượng tử hoá.

2.9 Môi trường thực nghiệm

Bảng 2.3. Môi trường phần cứng và phần mềm dùng trong đề tài

Hạng mục	**Cấu hình**
Máy huấn luyện	Máy tính xách tay GPU NVIDIA RTX 4060 Laptop 8 GB, RAM 14 GB, CUDA; PyTorch; Ultralytics 8.4 cho các mô hình đối chứng
Máy tính trên UAV	Raspberry Pi 5 (Debian 12, nhân 6.12, aarch64), Python 3.11
Bộ tăng tốc	Hailo-8L qua PCIe, HailoRT 4.20.0; biên dịch bằng Hailo Dataflow Compiler 3.34 trên máy x86
Camera	Logitech C270 HD, USB, MJPG, tối đa 1280×720 ở 30 khung hình/giây
Bộ điều khiển bay	Holybro Pixhawk 6C, ArduPilot Copter 4.7.0, kết nối USB, MAVLink v2
Phần mềm hỗ trợ	Mission Planner (cấu hình tham số), pymavlink, OpenCV, NumPy, SciPy

2.10 Kết luận chương

Đề tài sử dụng ba nguồn dữ liệu bao phủ hai miền: VisDrone-person cho so sánh chuẩn, COCO-person bổ sung người cự ly gần, và 92 khung bay thật chỉ dùng để đánh giá trên miền triển khai. Giao thức VisDrone-Shake mô phỏng ba hiệu ứng vật lý của rung mà không làm sai nhãn, được cố định seed và lặp ba lần. Mọi mô hình được đo bằng một chương trình duy nhất và chọn checkpoint theo một quy tắc duy nhất. Những ràng buộc này đảm bảo các so sánh ở Chương 4 phản ánh khác biệt thật giữa các mô hình.

CHƯƠNG 3

MÔ HÌNH STEADYDET-N VÀ PHƯƠNG PHÁP HUẤN LUYỆN NHẬN BIẾT RUNG

Chương này trình bày mô hình phát hiện người do đề tài xây dựng. Phần đầu mô tả quá trình đi tới kiến trúc cuối cùng và lý do của từng lựa chọn; phần sau trình bày hàm mất mát, quy trình huấn luyện và phương pháp huấn luyện nhận biết rung, là đóng góp chính về mặt mô hình của đề tài.

3.1 Yêu cầu thiết kế

Mô hình phải thoả mãn đồng thời bốn yêu cầu, mỗi yêu cầu xuất phát từ một ràng buộc cụ thể của hệ thống:

1. **Phát hiện được người nhỏ**: khi máy bay lên cao hoặc người đi xa, bounding box chỉ còn vài chục điểm ảnh vuông, nên mô hình cần một mức dự đoán độ phân giải cao.
2. **Chi phí tính toán thấp**: một khung hình phải đi qua camera, mô hình, bộ theo vết và bộ điều khiển trong ngân sách 33,3 ms của vòng lặp 30 Hz trên một máy tính nhúng tản nhiệt thụ động.
3. **Chỉ dùng các phép toán mà NPU hỗ trợ**: mô hình phải biên dịch được bằng Hailo DFC và lượng tử hoá INT8 được.
4. **Bền trước rung**: độ chính xác phải giữ được khi ảnh bị nhoè và nhiễu do rung của khung máy bay.

3.2 Quá trình phát triển kiến trúc

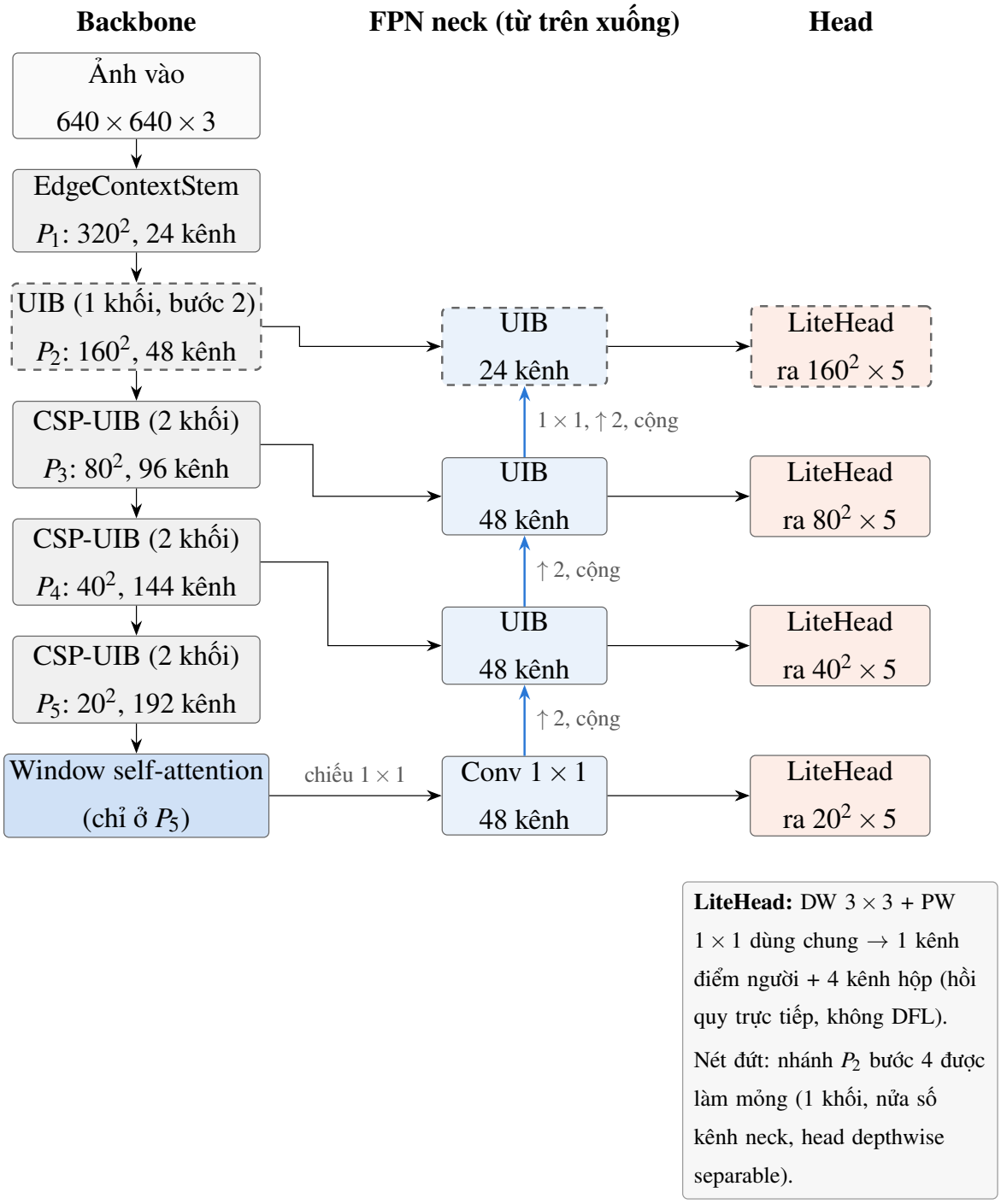
Kiến trúc được phát triển qua nhiều phiên bản, mỗi phiên bản được đo đạc trước khi quyết định hướng tiếp theo. Hai phiên bản quan trọng nhất được đặt tên trong báo cáo là **SteadyDet-W** (bản rộng) và **SteadyDet-N** (bản nano, bản cuối cùng). Trong mã nguồn, hai mô hình này tương ứng các tệp `pfdet_nano_v15.py` và `pfdet_nano_v17.py`.

SteadyDet-W đã có ít tham số (1,00 triệu) và chính xác nhất trong các phiên bản của nhóm, nhưng chi phí tính toán tới 12,9 GFLOPs ở độ phân giải 640. Phân tích chi phí theo từng lớp cho thấy nguyên nhân: riêng head ở mức P_2 (160×160) đã tiêu tốn

1,82 GMACs, tức 28% tổng chi phí, vì nó dùng năm lớp tích chập đầy đủ trên bản đồ lớn nhất. Đây là minh hoạ trực tiếp cho nhận xét ở Chương 1: số tham số ít không đồng nghĩa với chạy nhanh, vì chi phí tỷ lệ với diện tích feature map mà lớp đó xử lý. SteadyDet-N ra đời để giữ mức P_2 nhưng làm cho nó rẻ.

3.3 Kiến trúc SteadyDet-N

Hình 3.1 mô tả kiến trúc SteadyDet-N với cấu hình *light* dùng trong toàn bộ đề tài. Mô hình có 1,17 triệu tham số và 4,6 GFLOPs với ảnh vào 640×640, là mô hình nhỏ nhất và rẻ nhất trong mọi bảng so sánh của báo cáo.



Hình 3.1. Kiến trúc SteadyDet-N; độ phân giải giảm dần từ trên xuống. Chi phí được phân bổ không đối xứng: nhánh P_2 (nét đứt) chỉ có một khối, số kênh bằng một nửa và head tách được, còn các khối CSP hai tầng và phép self-attention nằm ở những mức có bản đồ nhỏ

3.3.1 Backbone

Khối đầu vào EdgeContextStem. Ảnh vào được xử lý song song bởi hai nhánh: nhánh cục bộ gồm hai tích chập 3×3 (lớp đầu có bước 2) giữ chi tiết cạnh, và nhánh ngữ cảnh gồm phép lấy trung bình 2×2 rồi một tích chập 3×3. Hai nhánh được ghép kênh và

trộn bằng tích chập 1×1, cho ra bản đồ P_1 kích thước 320×320 với 24 kênh. Việc giảm độ phân giải bằng hai con đường bổ trợ giúp giữ thông tin của các vật rất nhỏ ngay từ lớp đầu tiên.

Mức P_2 chỉ một khối. Mức bước 4 dùng đúng một khối UIB [37] có bước 2, gồm tích chập depthwise 3×3, mở rộng kênh 1×1 với hệ số 4, rồi thu hẹp 1×1 về 48 kênh. SteadyDet-W dùng ba khối ở mức này; giảm xuống một khối cắt phần lớn chi phí của mức đắt nhất.

Các mức P_3–P_5. Ba mức sâu hơn dùng khối CSP-UIB: một khối UIB giảm mẫu, sau đó bản đồ được tách thành hai nửa theo kênh, một nửa đi qua hai khối UIB, nửa kia đi tắt, rồi hai nửa được ghép lại và trộn bằng tích chập 1×1 theo ý tưởng của CSPNet [38]. Số kênh lần lượt là 96, 144 và 192. Ở P_3 và P_4, khối UIB có thêm một tích chập depthwise để mở rộng receptive field với chi phí thấp. Chi phí tính toán được dồn vào các mức này vì bản đồ ở đây nhỏ: 80×80, 40×40 và 20×20.

Self-attention chỉ ở P_5. Sau mức P_5, một khối self-attention theo cửa sổ [40] với window size 3 và 2 attention head cho mô hình ngữ cảnh toàn cục (ví dụ phân biệt người với cột điện hay bóng đổ). Đặt ở 20×20 nên chi phí của nó không đáng kể.

3.3.2 FPN neck mỏng

Neck của mạng là một FPN [31] chỉ có đường từ trên xuống. Mỗi mức của backbone được chiếu về số kênh chung bằng tích chập 1×1: 48 kênh cho P_3–P_5 và chỉ **24 kênh cho P_2**. Bản đồ mức trên được phóng to gấp đôi bằng nội suy nearest-neighbor, cộng với bản đồ mức dưới, rồi đi qua một khối UIB. Riêng khi đi từ P_3 xuống P_2, bản đồ được chiếu từ 48 về 24 kênh trước khi phóng to. Việc giảm một nửa số kênh ở mức 160×160 làm chi phí mọi lớp ở mức này giảm khoảng bốn lần.

3.3.3 Head LiteHead

Mỗi mức có một head LiteHead gồm một tích chập depthwise 3×3 và một tích chập 1×1 dùng chung, sau đó tách thành hai tích chập 1×1: một cho điểm có người (1 kênh), một cho bounding box (4 kênh). Như vậy mỗi head xuất một tensor $5 \times H \times W$ ở bốn bước 4, 8, 16 và 32. So với head tách rời năm lớp tích chập đầy đủ của SteadyDet-W, LiteHead rẻ hơn nhiều mà vẫn tách hai nhiệm vụ phân loại và hồi quy. Hộp được hồi quy trực tiếp (không dùng phân phối rời rạc), tương tự hướng của YOLO26 [15]. Độ lệch ban

đầu của kênh phân loại được đặt bằng $-5{,}5$ để xác suất ban đầu xấp xỉ 0,004, tránh việc hàng chục nghìn ô nền sinh hộp sai trong những epoch đầu.

Giải mã. Với grid cell (i, j) ở bước s và đầu ra (t_x, t_y, t_w, t_h), hộp dự đoán được giải mã như sau:

$$c_x = \big(j + \mathrm{sigmoid}(t_x) - 0{,}5\big)\, s, \quad c_y = \big(i + \mathrm{sigmoid}(t_y) - 0{,}5\big)\, s, \quad w = e^{t_w} s, \quad h = e^{t_h} s. \tag{3.1}$$

Mọi phép toán của mô hình (tích chập, batch norm được gộp vào tích chập, SiLU, cộng, ghép kênh, nội suy) đều được Hailo DFC hỗ trợ, trừ khối self-attention; Chương 5 trình bày cách xử lý khối này khi triển khai.

3.3.4 So sánh hai phiên bản

Bảng 3.1. So sánh cấu trúc và chi phí của SteadyDet-W và SteadyDet-N (ảnh vào 640×640)

Thành phần	**SteadyDet-W**	**SteadyDet-N**
Số khối ở mức P_2	3	1
Số kênh cổ ở mức P_2	bằng các mức khác	một nửa (24)
Head	tách rời 5 tích chập đầy đủ	LiteHead depthwise separable
Vị trí self-attention	P_4 và P_5	chỉ P_5
Số tham số	1,00 triệu	1,17 triệu
Chi phí tính toán	12,9 GFLOPs	**4,6 GFLOPs** (giảm 2,8 lần)

SteadyDet-N có nhiều tham số hơn SteadyDet-W một chút vì phần tính toán được dồn về các mức sâu, nơi số kênh lớn; nhưng chính các mức này có bản đồ nhỏ nên tổng chi phí giảm 2,8 lần.

3.4 Hàm mất mát

Tổng hàm mất mát gồm hai thành phần:

$$\mathscr{L}_{\mathrm{det}} = \lambda_{\mathrm{obj}}(e)\, \mathscr{L}_{\mathrm{obj}} + \lambda_{\mathrm{box}}(e)\, \mathscr{L}_{\mathrm{box}}, \tag{3.2}$$

trong đó e là chỉ số epoch.

Thành phần phân loại $\mathscr{L}_{\text{obj}}$ là hàm QFL [46] theo phương trình (1.4) với $\beta = 2$, nhãn mềm là IoU giữa hộp dự đoán và hộp thật tại các ô dương.

Thành phần hồi quy kết hợp NWD và CIoU với trọng số phụ thuộc diện tích hộp thật A (điểm ảnh vuông):

$$\mathscr{L}_{\text{box}} = w_{\text{ASL}}(A)\Big[\,\omega(A)\,\big(1-\text{NWD}\big) + \big(1-\omega(A)\big)\,\mathscr{L}_{\text{CIoU}}\Big], \qquad \omega(A) = \exp\!\left(-\frac{A}{1024}\right). \tag{3.3}$$

Với người rất nhỏ ($A \ll 32^2$), $\omega \approx 1$ và NWD chiếm ưu thế nhờ gradient ổn định; với người lớn, CIoU chiếm ưu thế vì nó phạt chính xác hình dạng hộp. Hằng số chuẩn hoá NWD là $C = 0{,}019$ theo toạ độ chuẩn hoá. Hệ số $w_{\text{ASL}}(A) = \text{clip}\big(\sqrt{A_{\text{ref}}/A},\, 1,\, 4\big)$ tăng trọng số cho vật nhỏ, với A_{ref} tương ứng 0,4% diện tích ảnh.

Gán nhãn nhận biết mục tiêu nhỏ. Việc chọn ô dương dựa trên mức độ phù hợp theo TOOD [34], $t = s^{\alpha} \cdot \text{IoU}^{\beta}$ với $\alpha = 1$, $\beta = 8$. Vì một người nhỏ chỉ phủ vài grid cell, chiến lược mặc định dễ cấp cho họ quá ít ô dương. Đề tài áp dụng hai điều chỉnh: người có diện tích dưới 576 điểm ảnh vuông (24^2) được cấp tới 6 ô dương và vùng tìm kiếm bán kính 3 ô, còn người lớn hơn được cấp 3 ô với bán kính 2; đồng thời số ô dương tối thiểu cho mục tiêu nhỏ không dưới 4.

Cân bằng tiến triển. Ở đầu quá trình huấn luyện, mô hình cần học "có người hay không" trước khi học "hộp khít tới đâu". Trọng số hai thành phần được nội suy tuyến tính theo tiến độ huấn luyện: λ_{obj} giảm từ 2×8 về 8 và λ_{box} tăng từ $4/2$ lên 4.

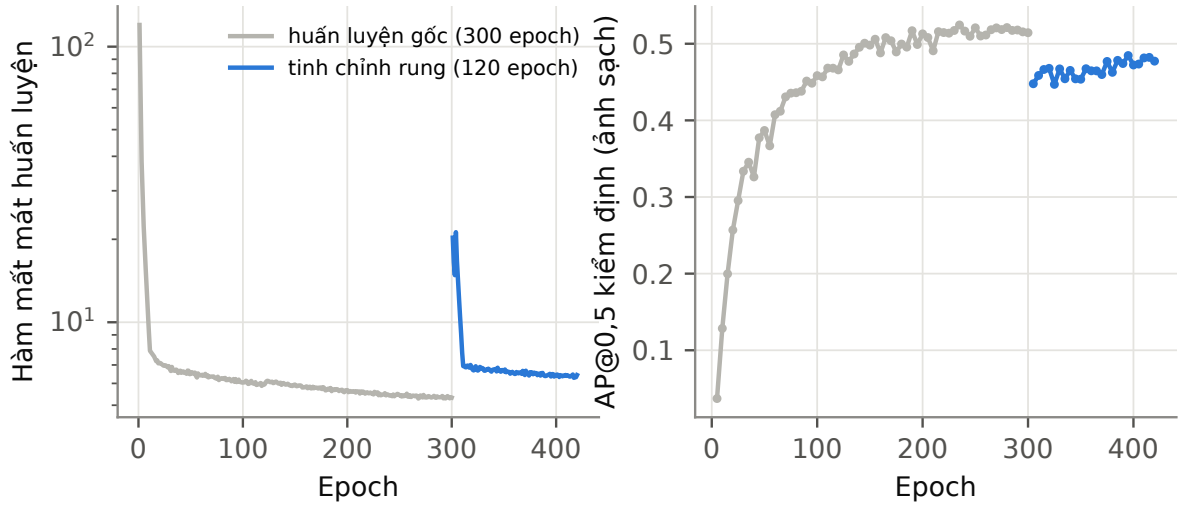
3.5 Quy trình huấn luyện

Bộ tối ưu. Đề tài dùng MuSGD, bộ tối ưu lai được Ultralytics giới thiệu cùng YOLO26 [15], kết hợp Muon [66] với SGD có quán tính Nesterov. Với các tensor trọng số nhiều chiều, bước cập nhật là tổng của hai phần: phần Muon (hệ số 0,2), trong đó gradient được trực giao hoá xấp xỉ bằng 5 vòng lặp Newton–Schulz, và phần SGD (hệ số 1,0); các tham số một chiều (độ lệch, batch norm) chỉ dùng SGD. Trong các thử nghiệm của nhóm trên SteadyDet-W, MuSGD cho AP@0,5 kiểm định 0,5932, cao nhất trong các bộ tối ưu đã thử.

Huấn luyện gốc (nhánh chuẩn). SteadyDet-N được huấn luyện từ khởi tạo ngẫu nhiên trong 300 epoch, batch size 16 với tích luỹ gradient 6 bước, learning rate 0,01 giảm theo hàm cosin xuống 10^{-4}, 10 epoch warm-up, weight decay 5×10^{-4}, huấn luyện độ

chính xác hỗn hợp và EMA của trọng số với hệ số 0,9998. Phép ghép bốn ảnh được tắt trong 60 epoch cuối. Mô hình được đánh giá trên tập kiểm định sau mỗi 5 epoch.

Nhánh triển khai. Mô hình được huấn luyện 400 epoch với batch size 12; dữ liệu COCO-person được trộn với tỷ lệ 25% từ epoch 100 tới epoch 360. Khi cần triển khai lên NPU, mô hình này được tinh chỉnh thêm 20 epoch với learning rate 10^{-3} ở cấu hình không có khối self-attention (Chương 5).



Hình 3.2. Hàm mất mát huấn luyện (trái, thang log) và AP@0,5 trên tập kiểm định ảnh sạch (phải) của SteadyDet-N nhánh chuẩn: 300 epoch huấn luyện gốc (xám) và 120 epoch tinh chỉnh nhận biết rung (xanh). Hàm mất mát tăng vọt khi bắt đầu tinh chỉnh vì một nửa số lô là ảnh đã bị làm rung

3.6 Huấn luyện nhận biết rung

3.6.1 Ý tưởng

Nguyên nhân gốc khiến bộ phát hiện sụp đổ khi rung là *sai lệch phân phối*: VisDrone được thu từ các nền tảng ổn định, nên mô hình chưa bao giờ thấy ảnh nhoè do rung trong huấn luyện nhưng lại gặp nó liên tục khi bay. Các phép tăng cường chuẩn ở Chương 2 như lật ảnh, co giãn hay đổi màu chỉ lấy mẫu những biến đổi mà dữ liệu vốn đã chứa; chúng không sinh ra loại suy giảm này. Phương pháp của đề tài vì thế rất trực tiếp: đưa chính toán tử rung vào phân phối huấn luyện.

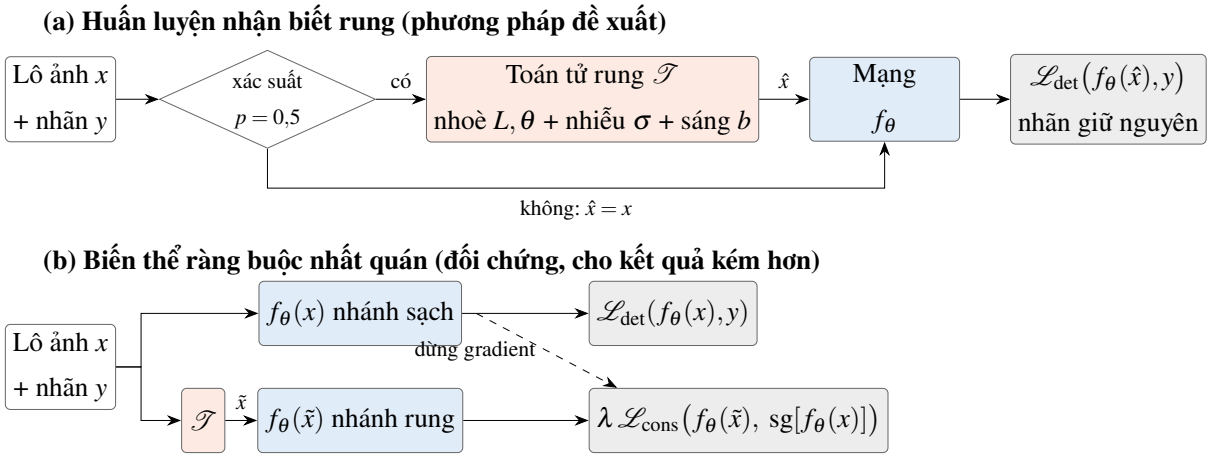
3.6.2 Phương pháp

Gọi $\mathscr{T}$ là toán tử rung ngẫu nhiên của giao thức VisDrone-Shake (mục 2.5), trong đó mức rung được chọn ngẫu nhiên đều trong $\{1, 2, 3\}$ và hướng nhoè được rút lại cho mỗi

batch. Trong huấn luyện, mỗi batch x được thay bằng $\mathscr{T}(x)$ với xác suất $p = 0{,}5$, và hàm mất mát phát hiện thông thường được tính trên ảnh mà mạng nhận được:

$$\mathscr{L} = \mathscr{L}_{\text{det}}\big(f_\theta(\hat{x}), y\big), \qquad \hat{x} = \begin{cases} \mathscr{T}(x) & \text{với xác suất } p, \\ x & \text{ngược lại.} \end{cases} \tag{3.4}$$

Vì $\mathscr{T}$ không làm dịch chuyển đối tượng, nhãn y không cần điều chỉnh; đây chính là điều làm cho phép thay thế hợp lệ. Phương pháp không thêm mô-đun nào, không cần truyền xuôi hai lần, và mô hình xuất ra để triển khai có cấu trúc giống hệt mô hình gốc, nên **chi phí suy luận không thay đổi**. Chi phí huấn luyện tăng thêm chỉ là một phép tích chập depthwise và hai phép toán từng phần tử trên GPU. Hình 3.3(a) minh hoạ phương pháp.



Hình 3.3. (a) Huấn luyện nhận biết rung: một nửa số lô được thay bằng bản đã làm rung và tính hàm mất mát phát hiện thông thường. (b) Biến thể ràng buộc nhất quán: truyền xuôi cả hai bản và kéo dự đoán trên ảnh rung về phía dự đoán trên ảnh sạch đã dừng gradient. Cả hai chỉ tồn tại khi huấn luyện

3.6.3 Biến thể ràng buộc nhất quán

Trước khi có kết quả, nhóm kỳ vọng rằng có thể khai thác nhiều tín hiệu hơn từ ảnh rung so với chỉ dùng nhãn. Biến thể này, lấy cảm hứng từ các phương pháp học bán giám sát dựa trên tính nhất quán, truyền cả x và $\tilde{x} = \mathscr{T}(x)$ qua cùng một mạng, tính hàm mất mát phát hiện trên nhánh sạch, và thêm một thành phần kéo dự đoán trên ảnh rung về phía bản sao đã dừng gradient của dự đoán sạch:

$$\mathscr{L} = \mathscr{L}_{\text{det}}\big(f_\theta(x), y\big) + \lambda\, \mathscr{L}_{\text{cons}}\Big(f_\theta(\tilde{x}),\ \text{sg}\big[f_\theta(x)\big]\Big), \qquad \lambda = 0{,}5. \tag{3.5}$$

$\mathscr{L}_{\text{cons}}$ gồm ba phần cộng trên các mức: entropy chéo có nhiệt độ $T = 2$ giữa điểm tin cậy hai nhánh, sai số L_1 trơn trên toạ độ hộp chỉ tại các ô mà nhánh sạch tin cậy trên 0,1, và sai số bình phương giữa đặc trưng cổ mạng với trọng số 0,5. Phép dừng gradient là bắt buộc: nếu thiếu nó, cách dễ nhất để giảm $\mathscr{L}_{\text{cons}}$ là làm dự đoán sạch kém đi cho giống dự đoán rung, tức phá huỷ bộ phát hiện. Biến thể này cần hai lượt truyền xuôi nên tốn khoảng 1,8 lần thời gian huấn luyện. Chương 4 cho thấy nó thực tế kém hơn phương pháp đơn giản.

3.6.4 Thiết lập tinh chỉnh

Huấn luyện nhận biết rung được áp dụng như một giai đoạn *tinh chỉnh*: xuất phát từ checkpoint tốt nhất của huấn luyện gốc, tinh chỉnh 120 epoch với batch size 8 và tích luỹ gradient 12 bước, MuSGD với learning rate 2×10^{-3}, 3 epoch warm-up trước khi bật toán tử rung. Nhờ vậy phương pháp có thể áp dụng cho một mô hình đã huấn luyện xong mà không phải lặp lại toàn bộ lịch trình. Checkpoint được chọn theo AP kiểm định trên ảnh sạch, đúng quy tắc áp dụng cho mô hình gốc.

Để tách bạch tác dụng của rung khỏi tác dụng của việc huấn luyện thêm 120 epoch, đề tài chạy một **thí nghiệm đối chứng**: tinh chỉnh mô hình gốc đúng 120 epoch với cùng mọi siêu tham số nhưng tắt toán tử rung. Cả ba cấu hình (đối chứng, nhận biết rung, ràng buộc nhất quán) chỉ khác nhau đúng một biến.

3.6.5 Biến thể không dùng NMS

Đề tài cũng thử một biến thể SteadyDet-N không cần NMS theo ý tưởng của YOLOv10 [14]: thêm một head LiteHead "one-to-one" song song dùng chung đặc trưng, huấn luyện đồng thời với head "one-to-many", và khi suy luận chỉ dùng head one-to-one. Biến thể này đạt AP@[.50:.95] 0,1954 trên ảnh sạch và retain 23,4 $\pm$ 0,8%. Tuy nhiên, trên hệ thống thực tế NMS chỉ tốn trung vị 0,84 ms mỗi khung (Chương 5), nên việc bỏ NMS không mang lại lợi ích đáng kể về độ trễ; đề tài giữ head thông thường cho các thí nghiệm chính và triển khai.

3.7 Kết luận chương

SteadyDet-N giữ mức dự đoán bước 4 cho người nhỏ nhưng làm nhánh này mỏng đi, dồn tính toán về các mức sâu, dùng head depthwise separable, nhờ đó giảm chi phí 2,8 lần so với SteadyDet-W xuống 4,6 GFLOPs với 1,17 triệu tham số. Hàm mất mát kết hợp QFL,

NWD và CIoU theo diện tích cùng chiến lược gán nhãn ưu tiên mục tiêu nhỏ. Phương pháp huấn luyện nhận biết rung đưa mô hình vật lý của rung vào phân phối huấn luyện, không đổi nhãn, không đổi mô hình khi suy luận. Chương tiếp theo kiểm chứng các thiết kế này bằng thực nghiệm có đối chứng.

CHƯƠNG 4

KẾT QUẢ THỰC NGHIỆM CỦA MÔ HÌNH

Chương này trình bày kết quả đánh giá mô hình theo đúng giao thức ở Chương 2. Mọi số liệu trên VisDrone-person được đo bằng một chương trình duy nhất; các cột có rung là trung bình ± độ lệch chuẩn qua ba hạt giống. Chương gồm bốn phần: tác dụng của huấn luyện nhận biết rung cùng các đối chứng; tính tổng quát và giới hạn của phương pháp; so sánh với năm mô hình nhẹ công khai; và kiểm chứng trên dữ liệu bay thật.

4.1 Kết quả chính và các thí nghiệm đối chứng

Bảng 4.1. Kết quả trên VisDrone-person, ảnh 640 × 640. Trong mỗi khối, các dòng chỉ khác nhau đúng một yếu tố và dùng chung lịch trình, learning rate và phép tăng cường. Cột có rung: trung bình ± độ lệch chuẩn qua 3 hạt giống

	Ảnh sạch			Khi rung (AP@[.50:.95])			
Mô hình	**AP**	**AP_{50}**	**AP_{75}**	**Mức 1**	**Mức 2**	**Mức 3**	**retain (%)**
SteadyDet-N (1,17 triệu tham số; 4,6 GFLOPs)							
Gốc	0,1892	0,5127	0,0811	0,0792 ±0,0026	0,0123 ±0,0013	0,0040 ±0,0005	16,8 ±0,7
+ 120 epoch không rung	0,1823	0,4984	0,0762	0,0761 ±0,0028	0,0125 ±0,0005	0,0040 ±0,0005	16,9 ±0,7
+ ràng buộc nhất quán	0,1746	0,4837	0,0728	0,1098 ±0,0010	0,0392 ±0,0021	0,0081 ±0,0011	30,0 ±0,6
+ huấn luyện nhận biết rung	0,1708	0,4731	0,0715	**0,1223 ±0,0014**	**0,0518 ±0,0019**	**0,0111 ±0,0002**	**36,2 ±0,7**
SteadyDet-W (1,00 triệu tham số; 12,9 GFLOPs)							
Gốc	0,2349	0,5800	0,1305	0,1260 ±0,0023	0,0258 ±0,0022	0,0046 ±0,0026	22,2 ±0,9
+ ràng buộc nhất quán	0,2207	0,5561	0,1202	0,1309 ±0,0026	0,0580 ±0,0007	0,0132 ±0,0016	30,5 ±0,6
+ huấn luyện nhận biết rung	0,2219	0,5511	0,1279	**0,1583 ±0,0013**	**0,0839 ±0,0001**	**0,0200 ±0,0012**	**39,4 ±0,3**
YOLO11-n huấn luyện từ đầu trên cùng dữ liệu (2,59 triệu tham số; 6,4 GFLOPs)							
Gốc	0,2588	0,5842	0,1839	0,1398 ±0,0057	0,0248 ±0,0022	0,0052 ±0,0021	21,9 ±1,1
+ 120 epoch không rung	0,2641	0,5897	0,1857	0,1410 ±0,0036	0,0226 ±0,0026	0,0062 ±0,0030	21,4 ±0,9
+ huấn luyện nhận biết rung	0,2579	0,5835	0,1806	0,1384 ±0,0037	0,0227 ±0,0017	0,0055 ±0,0026	21,5 ±0,8

4.1.1 Mọi bộ phát hiện nhỏ gọn đều sụp đổ khi rung

Dòng đầu của Bảng 4.1 cho thấy mức độ nghiêm trọng của vấn đề. SteadyDet-N gốc đạt AP 0,1892 trên ảnh sạch nhưng chỉ còn 0,0123 ở mức rung 2, tức giữ lại 6,5% độ chính

xác ở đúng điều kiện bay tiến thông thường; tính trung bình ba mức, nó chỉ giữ được $16{,}8 \pm 0{,}7\%$. Mục 4.3 sẽ cho thấy đây không phải điểm yếu riêng của mô hình này: mọi mô hình đối chứng, từ 2,5 tới 32,8 triệu tham số, đều mất 74–83% độ chính xác.

4.1.2 Huấn luyện nhận biết rung tăng gấp đôi tỷ lệ giữ độ chính xác

Tinh chỉnh với toán tử rung nâng retain của SteadyDet-N từ $16{,}8 \pm 0{,}7\%$ lên $36{,}2 \pm 0{,}7\%$, gấp 2,15 lần. Ở mức rung 2, AP tăng từ 0,0123 lên 0,0518, tức **tăng 4,2 lần**, và mô hình giữ được 30,3% độ chính xác sạch thay vì 6,5%. Cái giá phải trả là AP trên ảnh sạch giảm 9,7% (từ 0,1892 xuống 0,1708). Với một hệ thống bay, đây là trao đổi có lợi: một bộ phát hiện kém hơn một phần mười khi đứng yên nhưng chính xác hơn bốn lần khi cánh quạt đang quay là bộ phát hiện hữu ích hơn. Chênh lệch retain giữa hai mô hình (19,3 điểm) lớn hơn khoảng 20 lần độ lệch chuẩn gộp của hai phép đo, nên không thể do ngẫu nhiên của giao thức đo.

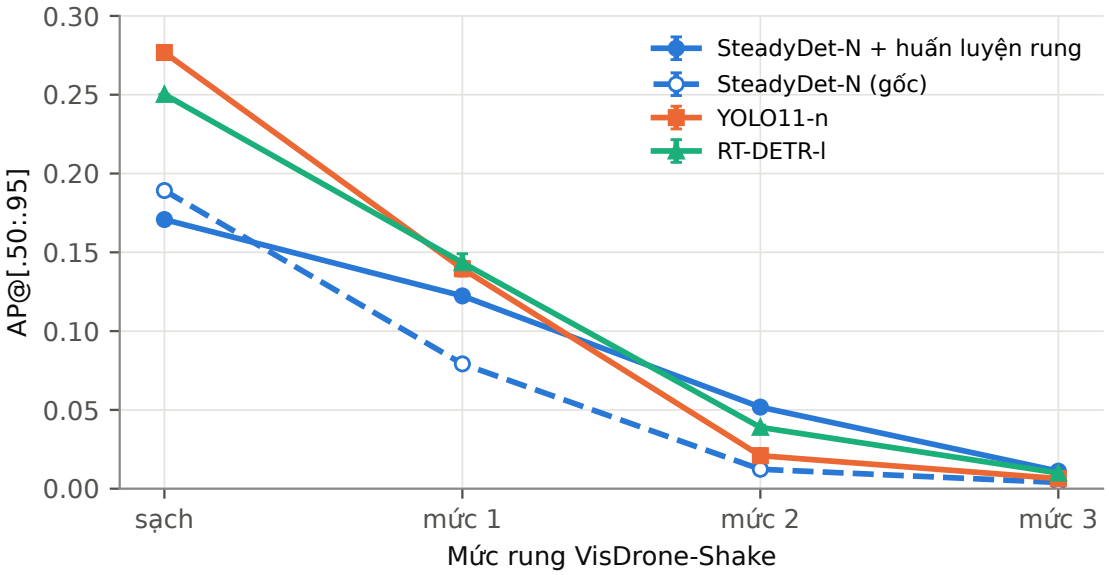
4.1.3 Đối chứng 1: tác dụng không đến từ việc huấn luyện thêm

Vì phương pháp được áp dụng dưới dạng tinh chỉnh, mô hình nhận biết rung đã được huấn luyện thêm 120 epoch mà mô hình gốc không có. Thí nghiệm đối chứng tinh chỉnh đúng 120 epoch với cùng mọi thiết lập nhưng tắt toán tử rung cho retain $16{,}9 \pm 0{,}7\%$, nằm trong một độ lệch chuẩn so với mô hình gốc ($16{,}8 \pm 0{,}7\%$). Huấn luyện thêm không đóng góp gì cho độ bền. Tuy vậy, AP sạch có giảm nhẹ trong 120 epoch đó (0,1892 xuống 0,1823), nên khoảng một phần ba mức giảm AP sạch của mô hình nhận biết rung đến từ bản thân quá trình tinh chỉnh chứ không phải từ toán tử rung.

4.1.4 Đối chứng 2: ràng buộc nhất quán kém hơn phương pháp đơn giản

Kết quả này trái với kỳ vọng ban đầu của nhóm. Biến thể ràng buộc nhất quán theo phương trình (3.5) chỉ đạt retain $30{,}0 \pm 0{,}6\%$, thấp hơn phương pháp đơn giản 6,2 điểm phần trăm, khoảng chín lần độ lệch chuẩn của mỗi phép đo. Nó cũng kém hơn ở *từng* mức rung tính theo AP tuyệt đối chứ không chỉ sau khi chuẩn hoá, trong khi tốn 1,8 lần thời gian huấn luyện. Cách giải thích của nhóm là: ở biến thể này, hàm mất mát phát hiện chỉ được tính trên nhánh sạch, nên mạng không bao giờ bị yêu cầu *phát hiện* trên ảnh nhoè; nó chỉ bị yêu cầu làm cho dự đoán trên ảnh nhoè giống dự đoán trên ảnh sạch. Yêu cầu này yếu hơn và có thể được thoả mãn bằng cách làm cả hai dự đoán bớt chắc chắn. Phương pháp đơn giản đặt ảnh nhoè trực tiếp đối diện với nhãn. Đây là cách giải thích

hợp lý chứ chưa được chứng minh bằng thí nghiệm riêng.



Hình 4.1. AP@[.50:.95] theo mức rung. Mô hình gốc (nét đứt) có AP sạch cao hơn nhưng giảm nhanh nhất; sau huấn luyện nhận biết rung (nét liền xanh), SteadyDet-N vượt YOLO11-n và RT-DETR-l từ mức rung 2. Thanh sai số: độ lệch chuẩn qua 3 hạt giống

4.2 Tính tổng quát và giới hạn của phương pháp

4.2.1 Lặp lại trên một kiến trúc khác

Một phương pháp chỉ có tác dụng trên đúng một kiến trúc thì khó phân biệt với một sự tương tác may mắn. Khối giữa của Bảng 4.1 lặp lại thí nghiệm trên SteadyDet-W, mô hình có nhánh P_2 đầy đủ, head năm lớp tích chập và chi phí gấp 2,8 lần, mà không chỉnh lại siêu tham số nào. Retain tăng từ $22{,}2 \pm 0{,}9\%$ lên $39{,}4 \pm 0{,}3\%$, tức +17,2 điểm so với +19,3 điểm ở SteadyDet-N; biến thể ràng buộc nhất quán lại một lần nữa kém hơn ($30{,}5 \pm 0{,}6\%$). Chiều hướng, độ lớn và thứ tự giữa hai biến thể đều được tái lập trên một mạng có độ rộng, độ sâu và thiết kế head khác.

4.2.2 Không có tác dụng trên YOLO11-n

Khối dưới cùng áp dụng đúng quy trình cho YOLO11-n và kết quả là không có thay đổi: retain từ $21{,}9 \pm 1{,}1\%$ thành $21{,}5 \pm 0{,}8\%$, đối chứng không rung $21{,}4 \pm 0{,}9\%$. Trên checkpoint YOLO11-n được huấn luyện sẵn trên COCO, phương pháp cho 20,1% thành 20,1%, còn biến thể nhất quán cho 19,2%. Nhật ký huấn luyện xác nhận toán tử rung đã thực sự được áp dụng. Nhóm đã kiểm tra ba giả thuyết và cả ba đều bị bác bỏ:

- *Do huấn luyện sẵn trên COCO*: nếu mô hình đã có sẵn đặc trưng chịu được nhoè thì không còn gì để học. Huấn luyện YOLO11-n từ khởi tạo ngẫu nhiên vẫn cho kết quả không đổi, và checkpoint huấn luyện sẵn lại là bản *kém* bền hơn (20,1% so với 21,9%).
- *Do kích thước vật*: có thể chỉ mức bước 4 mới hưởng lợi. Nhưng khi tách riêng vật nhỏ, mức tăng của SteadyDet-N là +18,7 điểm so với +19,3 điểm tổng thể, và gần như mọi người trong tập đều là vật nhỏ với cả hai họ mô hình (AP_s của YOLO11-n là 0,2741 trên AP 0,2768).
- *Do khoảng trống còn lại*: SteadyDet-W bắt đầu từ $22{,}2 \pm 0{,}9\%$ và YOLO11-n từ đầu bắt đầu từ $21{,}9 \pm 1{,}1\%$, gần như cùng điểm xuất phát, nhưng một mô hình tăng 17,2 điểm, mô hình kia không tăng.

Nguyên nhân còn lại nằm ở kiến trúc hoặc hàm mất mát mà hai mô hình của đề tài có chung còn YOLO11-n thì không: mức bước 4, hàm mất mát NWD kết hợp CIoU, QFL, và hồi quy hộp trực tiếp thay vì theo phân phối. Một mô hình không phản hồi không đủ để tách các yếu tố này, nên đề tài phát biểu kết luận đúng phạm vi bằng chứng: phương pháp cho mức tăng lớn và tái lập được trên họ kiến trúc của đề tài, và không có tác dụng trên mô hình ngoài họ đã thử.

Kết quả âm này cũng có một vai trò quan trọng: nó làm cho phép so sánh ở mục sau trở nên công bằng. Vì phương pháp không nâng được YOLO11-n, việc so sánh "SteadyDet-N có xử lý" với "YOLO11-n không xử lý" không phải là so với một đối thủ bị làm yếu đi.

4.3 So sánh với các mô hình phát hiện nhẹ

Bảng 4.2. So sánh với năm mô hình công khai trên VisDrone-person, mọi dòng đo bằng cùng một chương trình. mCP: AP@[.50:.95] trung bình ba mức rung, tức độ chính xác *khi bay*. Các mô hình YOLO và RT-DETR khởi tạo từ trọng số huấn luyện sẵn trên COCO, còn SteadyDet huấn luyện từ đầu, điều này có lợi cho mô hình đối chứng

Mô hình	Tham số (M)	GFLOPs	AP_{50}	AP	mCP	retain (%)
RT-DETR-l [16]	32,81	108,0	0,5909	0,2502[†]	0,0641	25,6 ±1,1
YOLOv10-n [14]	2,71	8,4	0,6043	0,2730	0,0538	19,7 ±1,4
YOLOv8-n [15]	3,01	8,2	0,6033	0,2684	0,0522	19,5 ±0,8
YOLO26-n [15]	2,50	5,8	0,5898	0,2675	0,0507	19,0 ±1,0
YOLO11-n [15]	2,59	6,4	**0,6119**	**0,2768**	0,0557	20,1 ±1,2
+ huấn luyện nhận biết rung	2,59	6,4	0,6057	0,2700	0,0542	20,1 ±1,3
SteadyDet-W (gốc)	1,00	12,9	0,5800	0,2349	0,0521	22,2 ±0,9
+ huấn luyện nhận biết rung	1,00	12,9	0,5511	0,2219	**0,0874**	**39,4 ±0,3**
SteadyDet-N (gốc)	**1,17**	**4,6**	0,5127	0,1892	0,0318	16,8 ±0,7
+ ràng buộc nhất quán	1,17	4,6	0,4837	0,1746	0,0524	30,0 ±0,6
+ huấn luyện nhận biết rung	**1,17**	**4,6**	0,4731	0,1708	**0,0617**	**36,2 ±0,7**

[†] Bộ tiền xử lý gốc của RT-DETR kéo giãn ảnh thay vì đệm xám; trong đường xử lý chung, AP sạch của nó bị kéo xuống (chương trình gốc cho 0,3340). mCP và retain là tỷ số trong cùng một đường xử lý nên không bị ảnh hưởng.

4.3.1 Trên ảnh sạch, SteadyDet-N không dẫn đầu

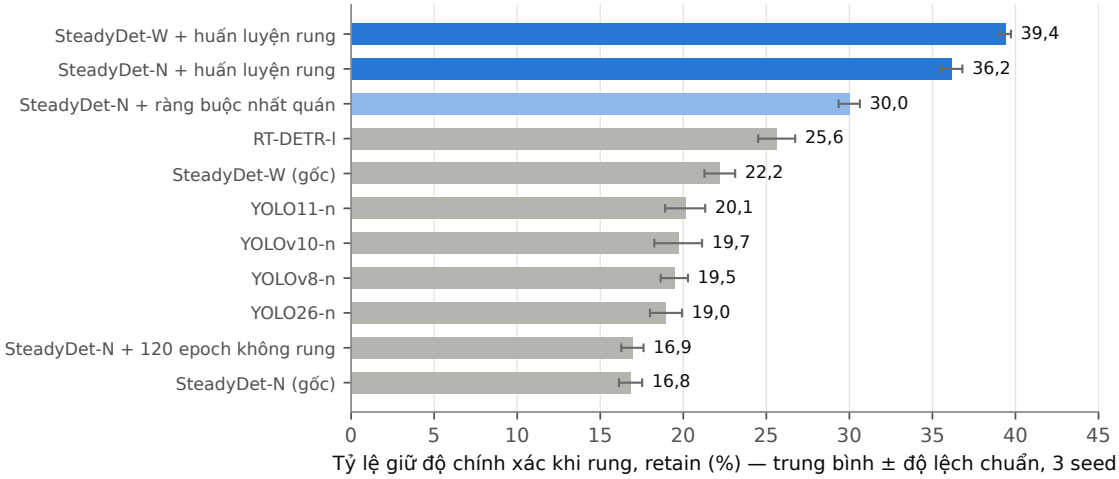
Cả bốn mô hình YOLO bản nano đều có AP sạch cao hơn SteadyDet-N, và đề tài không né tránh điều đó. Việc các mô hình đối chứng được khởi tạo từ trọng số huấn luyện sẵn trên COCO là một yếu tố, nhưng khoảng cách lớn hơn mức mà yếu tố này có thể giải thích. Hiệu quả tính toán cũng không bù được: 4,6 GFLOPs so với 5,8 GFLOPs của mô hình đối chứng rẻ nhất (YOLO26-n) chỉ tiết kiệm 21%. Nếu chỉ xét benchmark trên ảnh sạch, SteadyDet-N thua.

4.3.2 Khi rung, thứ hạng đảo ngược

Cột mCP xếp hạng bảng gần như ngược với cột AP sạch. SteadyDet-N có huấn luyện nhận biết rung đạt mCP 0,0617, cao hơn mô hình đối chứng nhỏ tốt nhất là YOLO11-n (0,0557), trong khi chỉ dùng 45% số tham số và 72% chi phí tính toán của nó. **SteadyDet-N vượt cả bốn mô hình YOLO bản nano về độ chính xác khi bay.**

RT-DETR-l có mCP trung bình cao hơn SteadyDet-N (0,0641 so với 0,0617), nhưng không phải ở điều kiện quan trọng nhất. Ở mức rung 2, tương ứng bay tiến thông thường, RT-DETR-l đạt $0{,}0389 \pm 0{,}0014$ còn SteadyDet-N đạt $0{,}0518 \pm 0{,}0019$, chênh lệch khoảng 5,5 lần độ lệch chuẩn gộp, trong khi RT-DETR-l có số tham số gấp 28 lần và chi phí gấp 23 lần. Ở mức 3, hai mô hình nằm trong phạm vi nhiễu ($0{,}0100 \pm 0{,}0017$ so với $0{,}0111 \pm 0{,}0002$), nên đề tài không kết luận thứ tự ở mức này. So với YOLO11-n ở mức 2, SteadyDet-N hơn 2,5 lần (0,0518 so với 0,0210).

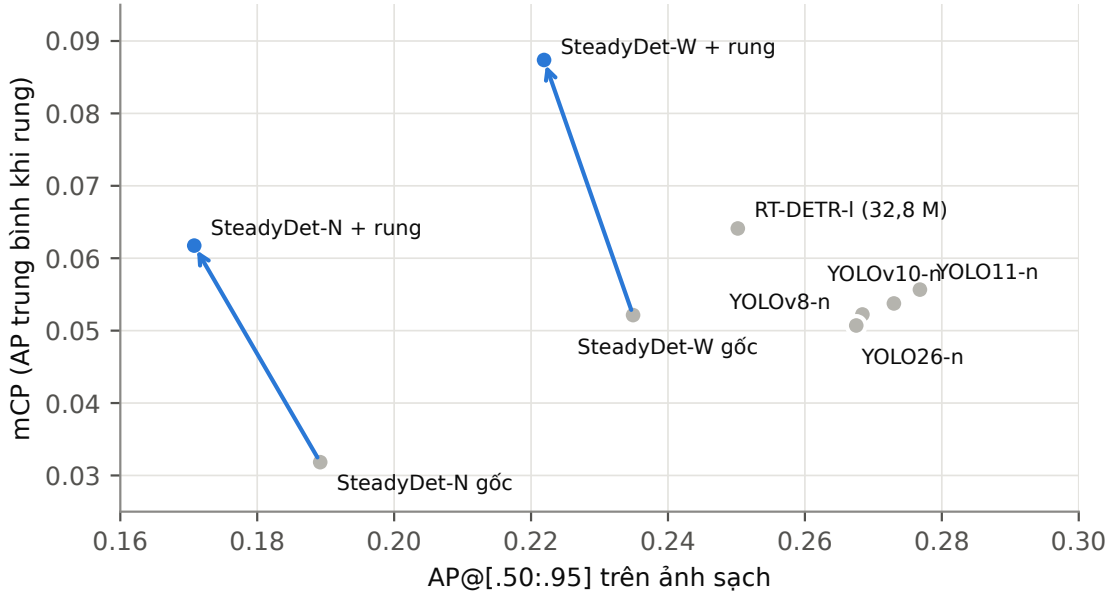
SteadyDet-W có huấn luyện nhận biết rung dẫn đầu toàn bảng với mCP 0,0874. Tuy vậy, đề tài chọn SteadyDet-N làm mô hình chính vì trên nền tảng đích, ràng buộc chặt nhất là chi phí tính toán chứ không phải số tham số: mô hình, bộ theo vết và bộ điều khiển phải cùng vừa một khung thời gian. Tính trên mỗi GFLOP, thứ tự rất rõ ràng: SteadyDet-N đạt 0,0134 mCP/GFLOP, so với 0,0087 của YOLO11-n và 0,0068 của SteadyDet-W.



Hình 4.2. Tỷ lệ giữ độ chính xác khi rung của mọi mô hình. Bốn thế hệ YOLO bản nano nằm sát nhau ở 19–20% dù khác nhau về kiến trúc; hai mô hình của đề tài sau huấn luyện nhận biết rung đứng đầu

4.3.3 Độ bền không phải là sản phẩm phụ của độ chính xác

Hình 4.3 biểu diễn AP sạch theo mCP. Các mô hình công khai tập trung ở góc phải dưới: độ chính xác sạch cao hơn nhưng không bền hơn; bốn thế hệ YOLO trải dài bốn năm và ba trường phái thiết kế nhưng retain chỉ cách nhau khoảng một điểm phần trăm. RT-DETR-l đạt 25,6% nhờ số tham số gấp 28 lần. Hendrycks và Dietterich đã ghi nhận hiện tượng tương tự với mô hình phân loại ImageNet [18]: độ bền tương đối gần như không cải thiện qua nhiều năm tiến bộ kiến trúc. Kết quả của đề tài cho thấy điều này cũng đúng với bộ phát hiện nhẹ trên ảnh UAV. Hệ quả thực tiễn là **độ bền phải được đo và tối ưu riêng**; các mũi tên trong hình cho thấy thay đổi phân phối huấn luyện mang lại mức tăng độ bền lớn với mức giảm độ chính xác sạch nhỏ.



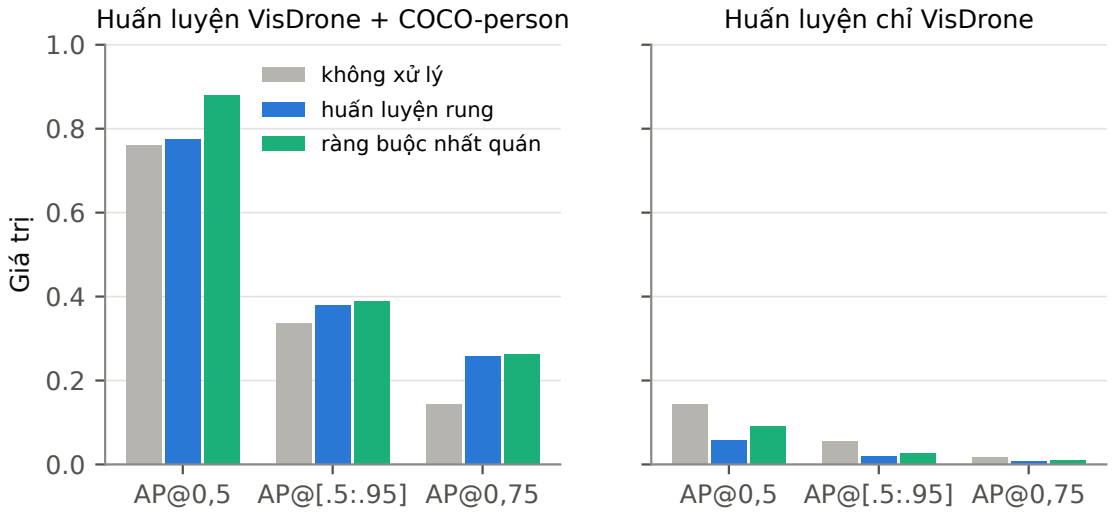
Hình 4.3. AP trên ảnh sạch và mCP khi rung, mỗi điểm là một mô hình. Mũi tên xanh: tác dụng của huấn luyện nhận biết rung trên hai mô hình của đề tài

4.4 Kiểm chứng trên dữ liệu bay thật

Mọi kết quả ở trên dựa trên suy giảm mô phỏng. Mục này dùng 92 khung hình bay thật ở độ cao khoảng 6 m, nơi nhoè do chính máy bay tạo ra. Sáu mô hình SteadyDet-N có cùng kiến trúc được so sánh theo hai trục độc lập: dữ liệu huấn luyện (có hay không có COCO-person) và cách xử lý rung.

Bảng 4.3. Kết quả trên 92 khung hình bay thật (≈6 m), cùng chương trình đo. Trong mỗi nhóm, ba mô hình giống hệt kiến trúc và chỉ khác cách xử lý rung

Dữ liệu huấn luyện	Xử lý rung	AP_{50}	AP	AP_{75}	AR_{100}
VisDrone + COCO-person	không	0,7613	0,3369	0,1448	0,5076
VisDrone + COCO-person	huấn luyện nhận biết rung	0,7747	0,3792	0,2583	**0,5315**
VisDrone + COCO-person	ràng buộc nhất quán	**0,8798**	**0,3885**	**0,2620**	0,5087
Chỉ VisDrone	không	0,1435	0,0547	0,0166	0,4315
Chỉ VisDrone	huấn luyện nhận biết rung	0,0579	0,0207	0,0076	0,4076
Chỉ VisDrone	ràng buộc nhất quán	0,0909	0,0283	0,0093	0,3772



Hình 4.4. Kết quả trên dữ liệu bay thật theo nhóm dữ liệu huấn luyện. Nhóm có COCO-person (trái) chính xác gấp nhiều lần nhóm chỉ VisDrone (phải); trong nhóm có COCO, cả hai cách xử lý rung làm AP@0,75 tăng gần gấp đôi

4.4.1 Cả hai cách xử lý rung đều có tác dụng trên rung thật

Trong nhóm mô hình phù hợp với miền này (có COCO-person), cả hai cách xử lý đều cải thiện theo cùng một kiểu. AP@0,75 tăng từ 0,1448 lên 0,2583 với huấn luyện nhận biết rung và 0,2620 với ràng buộc nhất quán, tức tăng tương đối 78% và 81% ở độ khít hộp, trong khi recall gần như không đổi (0,5076 thành 0,5315 và 0,5087). Mô hình không tìm thấy thêm người; nó đặt hộp khít hơn quanh những người đã tìm thấy, đúng dấu hiệu mong đợi nếu mô hình học được cách nhìn xuyên qua nhoè.

Để kiểm tra ý nghĩa thống kê trên 92 khung, đề tài lấy mẫu lại khung hình 1.000

lần theo cặp. Khoảng tin cậy 95% của hiệu so với mô hình không xử lý:

- huấn luyện nhận biết rung: AP +0,042 [+0,011; +0,074], AP@0,75 +0,108 [+0,010; +0,211];
- ràng buộc nhất quán: AP +0,051 [+0,012; +0,087], AP@0,75 +0,110 [+0,012; +0,214], AP@0,5 +0,119 [+0,071; +0,173].

Mọi khoảng tin cậy trên đều không chứa 0, nên cả hai cách xử lý thực sự tốt hơn mô hình không xử lý trên dữ liệu này. Hai cách xử lý chỉ tách nhau ở AP@0,5 (+0,106 [+0,058; +0,161] nghiêng về ràng buộc nhất quán), còn hiệu AP và AP@0,75 giữa chúng đều chứa 0. Thứ tự ở AP@0,5 ngược với thứ tự trên VisDrone-Shake. Với một người, một cảnh và một độ cao, dữ liệu này không đủ để quyết định thứ tự nào tổng quát hơn; câu trả lời cần một tập bay lớn hơn.

4.4.2 Chọn dữ liệu theo miền triển khai, không theo benchmark

Thêm COCO-person làm AP@0,5 trên dữ liệu bay thật tăng 5,3 lần (0,1435 lên 0,7613), trong khi trên VisDrone-val cùng thay đổi này lại *làm giảm* AP@0,5 (0,5127 xuống 0,5016). Phân tích chi tiết giải thích cơ chế: mô hình chỉ học VisDrone vẫn có recall khá (0,4315), tức vẫn thấy người, nhưng định vị sụp đổ (AP@0,75 chỉ 0,0166) vì chưa từng được học trên người chiếm phần lớn khung hình. Nếu chọn dữ liệu huấn luyện dựa trên benchmark, nhóm sẽ chọn đúng mô hình sai. Với bài toán bám theo người ở cự ly gần, dữ liệu huấn luyện phải khớp với dải độ cao triển khai.

Trong nhóm chỉ VisDrone, thứ tự đảo ngược (AP@0,5 giảm từ 0,1435 xuống 0,0909 và 0,0579). Cả ba mô hình này đều thất bại ở miền bay thấp, nên so sánh giữa chúng là so sánh giữa các cách thất bại: làm cho biểu diễn bền trước nhoè chỉ có ích khi biểu diễn đó vốn đã phù hợp. Huấn luyện nhận biết rung là phương pháp tăng độ bền, không phải phương pháp thích nghi miền.

4.5 Thảo luận

Bốn kết luận chính rút ra từ chương này:

1. Rung là điều kiện luôn có khi bay, và dưới một giao thức đo thống nhất, mọi bộ phát hiện nhỏ gọn đều mất phần lớn độ chính xác. Vấn đề thuộc về điều kiện vận hành chứ không thuộc về một kiến trúc cụ thể.

2. Huấn luyện nhận biết rung tăng retain của SteadyDet-N từ 16,8% lên 36,2% và AP ở mức rung 2 lên 4,2 lần, không tốn chi phí suy luận; đối chứng xác nhận tác dụng không đến từ việc huấn luyện thêm.
3. Kiến trúc và phương pháp huấn luyện đều cần thiết: SteadyDet-N không xử lý đứng cuối bảng về mCP, còn phương pháp áp dụng lên YOLO11-n không có tác dụng. Chỉ khi kết hợp cả hai, mô hình nhỏ nhất và rẻ nhất mới vượt cả bốn mô hình YOLO bản nano khi rung.
4. Trên dữ liệu bay thật, xử lý rung làm hộp khít hơn rõ rệt, và dữ liệu huấn luyện phù hợp miền triển khai quan trọng hơn thứ hạng trên benchmark.

Các giới hạn cần nêu rõ: toán tử rung là mô hình mô phỏng, tham số được chọn để bao quát điều kiện bay chứ chưa hiệu chuẩn theo phổ gia tốc đo trên chính khung máy bay; mỗi cấu hình chỉ được huấn luyện một lần nên thanh sai số phản ánh biến động của phép đo chứ chưa phản ánh biến động của huấn luyện; tập bay thật mới có một người, một cảnh, một độ cao. Các giới hạn này được đưa vào phần kiến nghị.

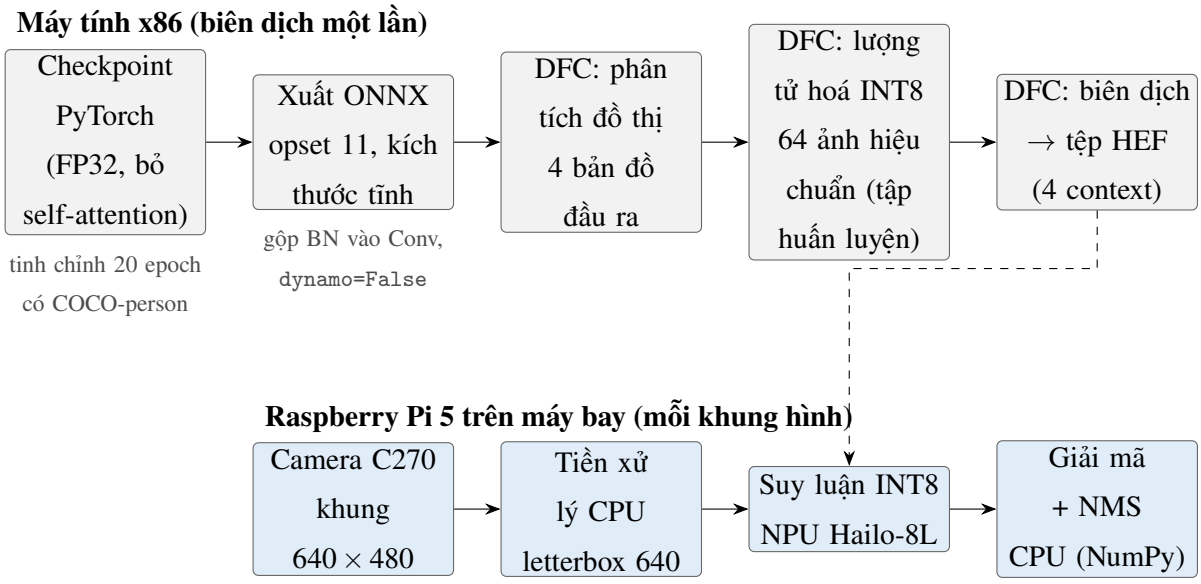
CHƯƠNG 5

TRIỂN KHAI MÔ HÌNH TRÊN THIẾT BỊ BIÊN

Một mô hình chính xác trên máy tính huấn luyện chưa chắc dùng được trên máy bay. Chương này trình bày quá trình đưa SteadyDet-N lên Raspberry Pi 5 gắn NPU Hailo-8L: chuẩn bị mô hình cho trình biên dịch, lượng tử hoá INT8, đo độ chính xác sau lượng tử hoá trên hai miền dữ liệu, đo tốc độ và độ trễ đầu-cuối với camera thật. Mọi con số trong chương đều được đo trực tiếp trên phần cứng đích.

5.1 Nền tảng phần cứng

Máy tính đồng hành là Raspberry Pi 5 chạy Debian 12. NPU Hailo-8L gắn qua khe M.2 và đường PCIe; trong cấu hình đo, liên kết hoạt động ở tốc độ Gen2 x1 (5 GT/s). Camera là Logitech C270 HD cắm cổng USB, dùng định dạng nén MJPG. Bộ điều khiển bay Pixhawk 6C nối với Pi qua USB. Phần suy luận chạy trên NPU; các phần còn lại (đọc camera, tiền xử lý, giải mã, NMS, theo vết, điều khiển, bảng giám sát web) chạy trên CPU của Pi.



Hình 5.1. Quy trình đưa mô hình lên NPU: biên dịch một lần trên máy tính x86 (trên) và chuỗi xử lý mỗi khung hình trên Raspberry Pi 5 (dưới)

5.2 Chuẩn bị mô hình cho trình biên dịch

5.2.1 Loại bỏ khối self-attention

Trình biên dịch Hailo DFC không hỗ trợ các phép biến đổi hình dạng tensor mà khối self-attention theo cửa sổ sử dụng. Đề tài xử lý bằng cách tạo một cấu hình SteadyDet-N không có khối này (thuần tích chập, 0,875 triệu tham số) và tinh chỉnh 20 epoch từ checkpoint của nhánh triển khai với learning rate 10^{-3}. Việc kiểm tra tệp ONNX sau khi xuất xác nhận đồ thị không còn các phép toán MatMul, Softmax hay ReduceMean. Việc chuyển đổi này có ảnh hưởng tới độ chính xác, được định lượng ở mục 5.4.1.

5.2.2 Một lỗi cấu hình và cách phát hiện

Lần tinh chỉnh đầu tiên cho kết quả kém bất thường trên dữ liệu bay thật: F1 của bản FP32 chỉ đạt 0,768 so với 0,882 của checkpoint trước khi tinh chỉnh. Khi phân tích nhật ký huấn luyện, nhóm nhận thấy toàn bộ mẫu dương đều rơi vào hai mức P_2 và P_3 (số mẫu dương ở P_4 và P_5 bằng 0), tức mô hình không hề thấy người kích thước lớn trong suốt quá trình tinh chỉnh. Nguyên nhân là tệp cấu hình đặt epoch bắt đầu trộn COCO-person bằng 100 trong khi toàn bộ quá trình chỉ có 20 epoch, nên dữ liệu người cự ly gần chưa bao giờ được bật. Sau khi sửa để trộn COCO ngay từ epoch đầu, nhật ký xác nhận có mẫu dương ở cả P_4 và P_5, và F1 trên dữ liệu bay thật tăng từ 0,768 lên 0,836, lấy lại 60% khoảng cách. Phần chênh lệch còn lại (0,046) là cái giá của việc bỏ khối self-attention trên miền bay thấp. Bài học rút ra là: kiểm tra phân bố mẫu dương theo mức là một cách rẻ để phát hiện dữ liệu huấn luyện không đúng như ý định.

5.2.3 Xuất ONNX

Mô hình được gộp các lớp batch norm vào tích chập, xuất ONNX với kích thước đầu vào tĩnh 640×640 và opset 11. Đầu ra là bốn bản đồ dự đoán thô; phần giải mã theo phương trình (3.1) và NMS chạy trên CPU. Nhóm gặp một lỗi khi xuất: từ PyTorch 2.6, hàm xuất ONNX mặc định dùng bộ xuất mới, bộ này bỏ thuộc tính `kernel_shape` của nút tích chập (vẫn hợp lệ theo chuẩn ONNX) nhưng trình phân tích của Hailo lại đọc trực tiếp thuộc tính này và dừng với lỗi chỉ số. Buộc dùng bộ xuất cũ (`dynamo=False`) cho tệp opset 11 với đủ 82/82 nút tích chập có thuộc tính này. Để chắc chắn tệp ONNX tương đương PyTorch, nhóm chạy toàn bộ phép quét ngưỡng trên dữ liệu bay thật bằng cả hai: F1 đạt 0,831 (ONNX) so với 0,836 (PyTorch), số dự đoán đúng giống hệt ở mọi

ngưỡng.

5.3 Lượng tử hoá và biên dịch

Tệp ONNX được DFC lượng tử hoá INT8 với mức tối ưu 2 và biên dịch thành tệp HEF gồm 4 context. Tập ảnh hiệu chuẩn được lấy từ *tập huấn luyện*, trộn đúng tỷ lệ dùng khi huấn luyện (75% VisDrone, 25% COCO-person). Tập flight_val không được dùng để hiệu chuẩn vì đó là tập đánh giá. Nhóm biên dịch hai phiên bản: một với 64 ảnh hiệu chuẩn và một với 512 ảnh.

5.4 Độ chính xác sau lượng tử hoá

5.4.1 Chọn tệp HEF triển khai

Ba tệp HEF được đo trên Pi, cùng 92 khung bay thật và cùng một đường giải mã (Bảng 5.1).

Bảng 5.1. So sánh các tệp HEF trên 92 khung bay thật; mỗi tệp dùng ngưỡng tin cậy cho F1 cao nhất của chính nó

Tệp HEF	Ảnh hiệu chuẩn	Ngưỡng	F1	Độ phủ	Độ chuẩn xác	FPS
Có COCO, 64 ảnh (triển khai)	64	0,25	**0,880**	**0,837**	0,928	56,9
Có COCO, 512 ảnh	512	0,25	0,812	0,750	0,885	62,9
Bản trước (chưa trộn COCO)	–	0,30	0,745	0,620	0,934	63,7

Tệp dùng 64 ảnh hiệu chuẩn cho F1 cao nhất và recall 0,837, tức bắt được người ở 84% số khung thay vì 62% như bản trước. Nó chậm hơn khoảng 2 ms mỗi khung (17,6 ms so với 15,7 ms) nhưng vẫn dư xa ngân sách 33,3 ms, nên được chọn để triển khai. Một kết quả trái trực giác: 512 ảnh hiệu chuẩn cho kết quả kém hơn 64 ảnh, dù trình biên dịch khuyến nghị nhiều ảnh hơn. Nhóm chưa xác định được cơ chế; bài học vận hành là phải đo trên dữ liệu miền triển khai thay vì dựa vào khuyến nghị chung.

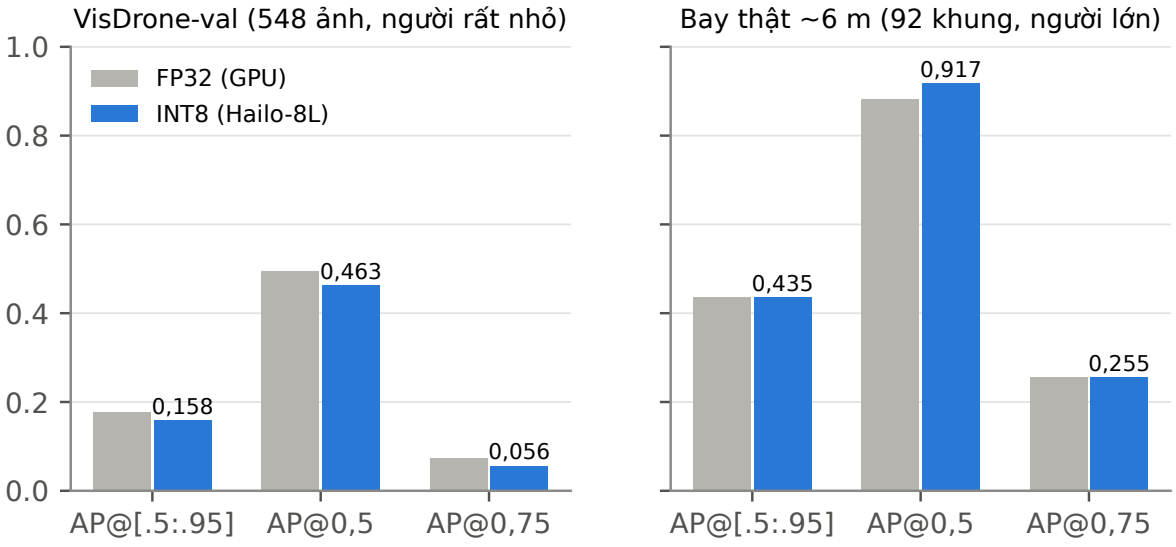
5.4.2 Tổn thất INT8 phụ thuộc kích thước đối tượng

Để đo đúng tổn thất do lượng tử hoá, dự đoán FP32 (trên GPU) và INT8 (trên NPU) của cùng một mô hình được xuất ra cùng một định dạng và chấm bằng cùng một hàm, trên

cả hai miền dữ liệu (Bảng 5.2, Hình 5.2).

Bảng 5.2. Độ chính xác FP32 và INT8 của mô hình triển khai, cùng một chương trình chấm điểm

	VisDrone-val (548 ảnh, người rất nhỏ)			Bay thật ≈6 m (92 khung)		
Chỉ số	**FP32**	**INT8**	**Thay đổi**	**FP32**	**INT8**	**Thay đổi**
AP@[.50:.95]	0,1775	0,1584	−10,8%	0,4356	0,4349	−0,2%
AP@0,50	0,4946	0,4625	−6,5%	0,8822	**0,9171**	+4,0%
AP@0,75	0,0742	0,0555	−25,2%	0,2561	0,2552	−0,4%
AR@100	0,3041	0,2869	−5,7%	0,5489	0,5337	−2,8%



Hình 5.2. Độ chính xác FP32 và INT8 trên hai miền dữ liệu. Trên người rất nhỏ, INT8 làm hộp kém khít (AP@0,75 giảm 25%); trên người ở độ cao bay thực tế, hai bản gần như trùng nhau

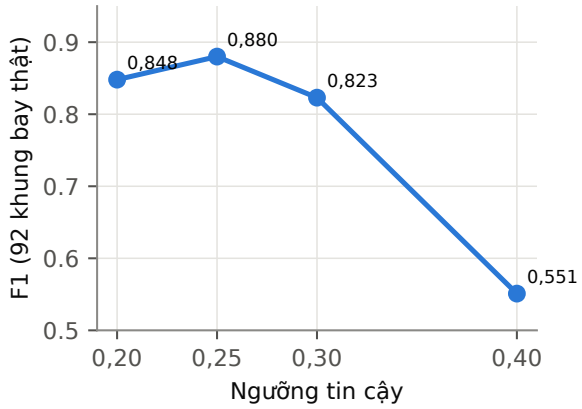
Trên VisDrone-val, AP@0,5 chỉ giảm 6,5% còn AP@0,75 giảm 25,2%: lượng tử hoá không làm mất đối tượng mà làm hộp kém khít. Điều này phù hợp với phân tích ở Chương 1: điểm tin cậy chịu được lượng tử hoá, còn toạ độ hộp mất độ phân giải, và với người chỉ vài điểm ảnh, sai lệch nhỏ đó đủ làm IoU tụt dưới 0,75. Trên dữ liệu bay thật, người chiếm hàng nghìn điểm ảnh nên cùng sai lệch gần như không đáng kể: hai bản khác nhau không quá 0,035 ở mọi chỉ số, nằm trong sai số lấy mẫu của 92 khung. Phát biểu chính xác là: **INT8 tốn khoảng 6,5% AP@0,5 với người rất nhỏ và gần như không tốn gì với người ở độ cao bay 3–6 m**. Kết quả này cũng điều chỉnh một kết luận

bi quan trước đó của nhóm, khi lượng tử hoá sau huấn luyện đơn giản trên ONNX từng làm AP sụp từ 0,18 xuống 0,009; hiệu chuẩn theo từng lớp của DFC tốt hơn nhiều.

Chỉ tiêu độ chính xác của đề tài. Mô hình đang chạy trên máy bay đạt AP@0,5 = 0,917 trên dữ liệu bay thật, vượt chỉ tiêu 70% trong thuyết minh. Con số này là của đúng mô hình INT8 trên NPU chứ không phải của một checkpoint trên máy tính huấn luyện. Lưu ý không so sánh trực tiếp con số 0,917 với 0,7613 ở Bảng 4.3: hai con số thuộc hai checkpoint khác nhau và được tính bằng hai đường giải mã khác nhau; Bảng 4.3 dùng để so sánh *dữ liệu và cách xử lý rung*, còn Bảng 5.2 dùng để đánh giá *hệ thống triển khai*.

5.4.3 Chọn ngưỡng tin cậy vận hành

Ngưỡng tin cậy quyết định trực tiếp hành vi trên máy bay: quá cao thì bỏ sót người, quá thấp thì bám nhầm vào hộp sai. Với bản HEF trước, ngưỡng 0,40 chỉ bắt được người ở 19,6% số khung, còn ngưỡng 0,20 sinh trung bình 1,30 hộp sai mỗi khung, khớp đúng hiện tượng người dùng quan sát được khi thử ("chĩa vào người không thấy gì" và "hạ ngưỡng thì nhiễu"). Hình 5.3 là kết quả quét ngưỡng cho bản HEF triển khai: F1 đạt cực đại 0,880 tại ngưỡng 0,25 và giảm còn 0,551 ở ngưỡng 0,40. Ngưỡng tối ưu thay đổi theo từng tệp HEF (bản trước là 0,30), nên mỗi lần đổi mô hình phải quét lại.



Hình 5.3. F1 của mô hình triển khai theo ngưỡng tin cậy trên 92 khung bay thật

5.5 Tốc độ suy luận trên NPU

5.5.1 Tốc độ theo từng khung và theo lô

Tốc độ được đo bằng công cụ chuẩn `hailortcli benchmark` của Hailo và kiểm tra chéo bằng lời gọi đồng bộ từ Python; hai cách cho kết quả khớp nhau. Với batch size 1, SteadyDet-N đạt **112,55 FPS (8,40 ms)** ở 320 × 320 và **63,66 FPS (15,26 ms)** ở 640 ×

640. Bản HEF triển khai đạt 56,9 FPS, độ trễ suy luận trung vị 17,32 ms và p99 17,52 ms đo trong lượt đánh giá 548 ảnh.

Bảng 5.3. Thông lượng và độ trễ theo batch size trên Hailo-8L (mô hình 320×320)

Kích thước lô	Thông lượng (FPS)	Độ trễ phần cứng (ms)
1	112,55	8,40
4	188,78	17,05
8	214,63	28,35
16	231,82	50,48

Bảng 5.3 cho thấy tăng batch size làm tăng thông lượng nhưng cũng làm tăng độ trễ mỗi khung. Hệ thống bám người dùng lô 1 vì hai lý do: độ trễ phần cứng tăng từ 8,40 lên 17,05 ms ở lô 4, và với camera 30 khung/giây, riêng việc chờ gom đủ 4 khung đã mất thêm 133 ms, một độ trễ mà vòng điều khiển bám người không chấp nhận được. Thông lượng và độ trễ là hai đại lượng khác nhau và không được nhầm lẫn khi báo cáo.

5.5.2 So sánh với YOLO26-n trên cùng một chip

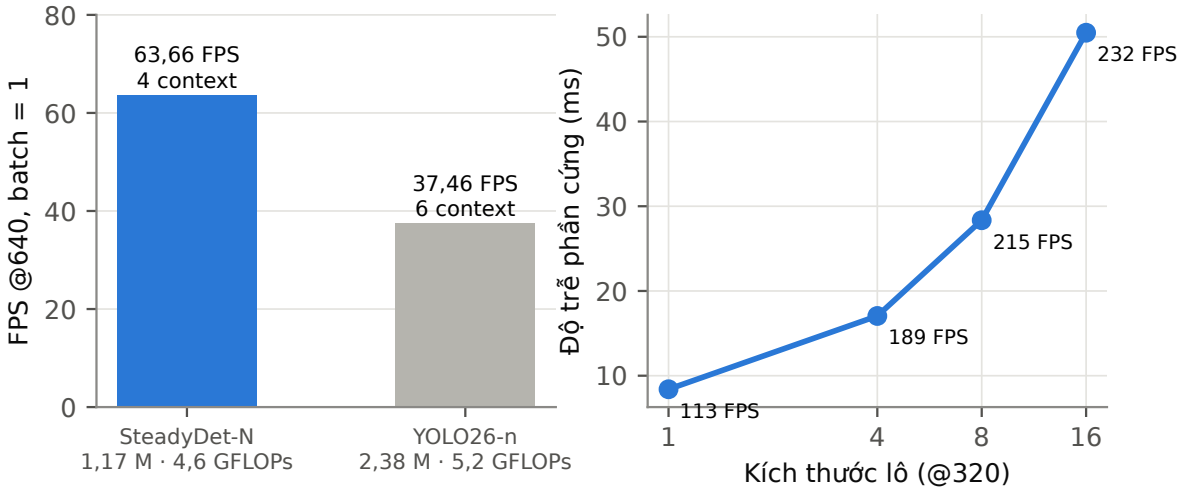
Để kiểm tra luận điểm hiệu quả tính toán trên phần cứng thật, đề tài biên dịch YOLO26-n (chính checkpoint dùng trong Bảng 4.2) bằng đúng cùng lệnh và tham số, cắt đồ thị tại các tích chập đầu ra để phần giải mã và NMS của cả hai mô hình đều chạy trên CPU, rồi đo trên cùng Pi.

Bảng 5.4. SteadyDet-N và YOLO26-n trên cùng Hailo-8L, ảnh 640×640, lô 1

Mô hình	Tham số (M)	GFLOPs	Số context	FPS	Độ trễ (ms)
SteadyDet-N	**1,17**	**4,6**	**4**	**63,66**	**15,26**
YOLO26-n	2,38	5,2	6	37,46	24,79

SteadyDet-N **nhanh hơn 1,70 lần** với 49% số tham số và 88% chi phí tính toán (Bảng 5.4, Hình 5.4). Mức chênh lệch tốc độ lớn hơn nhiều so với mức chênh lệch GFLOPs; cơ chế nằm ở số context: SteadyDet-N vừa 4 context còn YOLO26-n cần 6, và mỗi lần chuyển context NPU phải nạp lại trọng số qua PCIe. Trên loại chip này, giảm kích thước mô hình để giảm số context quan trọng không kém giảm số phép tính. Hai

tệp HEF trong phép so sánh này được hiệu chuẩn bằng ảnh kiểm định, nên chỉ dùng để so tốc độ, không dùng để lấy độ chính xác. Số tham số và GFLOPs của YOLO26-n ở đây được tính trên đồ thị đã xuất để biên dịch, nên khác một chút so với Bảng 4.2.



Hình 5.4. Trái: tốc độ SteadyDet-N và YOLO26-n trên cùng Hailo-8L. Phải: độ trễ phần cứng tăng theo batch size dù thông lượng tăng

5.6 Độ trễ đầu-cuối với camera thật

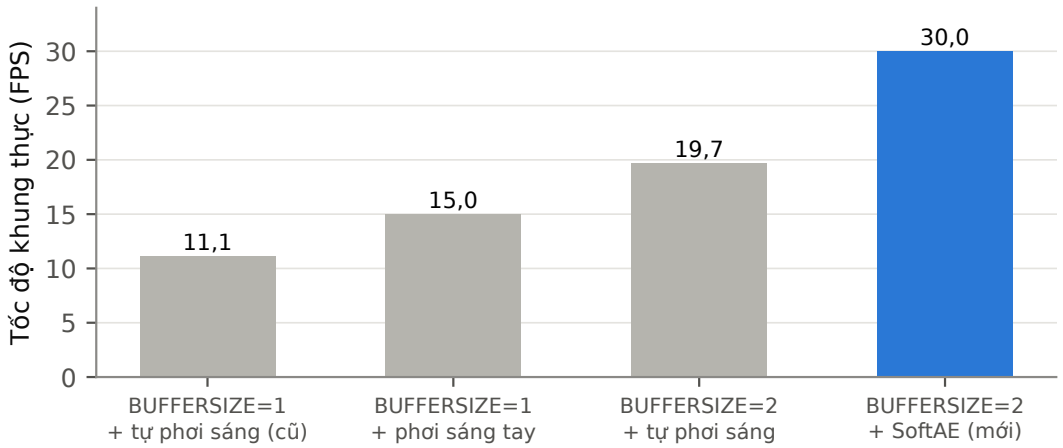
5.6.1 Hai lỗi phía camera làm hệ thống kẹt ở 15 khung hình/giây

Lần đo đầu tiên với camera thật chỉ đạt 15,4 khung hình/giây, trong khi NPU mới dùng khoảng 17,8 ms trong ngân sách 33,3 ms. Đo tách riêng từng chặng cho thấy nút cổ chai không nằm ở mô hình mà ở camera, do hai lỗi độc lập nhân với nhau (Hình 5.5):

1. **Bộ đệm camera bằng 1.** Tham số `CAP_PROP_BUFFERSIZE = 1` vốn được đặt để luôn lấy khung mới nhất, nhưng khi ứng dụng đang giữ bộ đệm duy nhất, trình điều khiển không có chỗ ghi khung kế tiếp nên bỏ luôn khung đó: 30 khung/giây thành đúng 15,0. Tăng lên 2 cho 30,0 khung/giây, còn độ cũ của khung đo được chỉ trung vị 15,7 ms, bằng nửa chu kỳ khung như lý thuyết.
2. **Tự phơi sáng tự hạ tốc độ khung.** Khi thiếu sáng, firmware của camera kéo dài thời gian phơi sáng quá 1/30 s và tự giảm tốc độ khung (đo được 24,9 khung/giây ở thời gian phơi sáng 1/25 s). Cờ UVC dùng để cấm hành vi này bị firmware bỏ qua. Đề tài giải quyết bằng cách khoá phơi sáng thủ công và tự viết bộ điều chỉnh độ sáng bằng phần mềm (SoftAE) có trần thời gian phơi sáng cứng: đo độ sáng trung bình mỗi khung rồi điều chỉnh thời gian phơi sáng và độ khuếch đại. Kết quả: độ sáng

bám giá trị đặt, tốc độ khung giữ 30,1.

Sau khi sửa cả hai, tốc độ tăng từ 11,1 lên 30,0 khung hình/giây mà không thay đổi dòng nào của mô hình. Tần số lưới điện của camera cũng được đặt về 50 Hz theo điện lưới Việt Nam để tránh sọc nhấp nháy dưới đèn huỳnh quang.



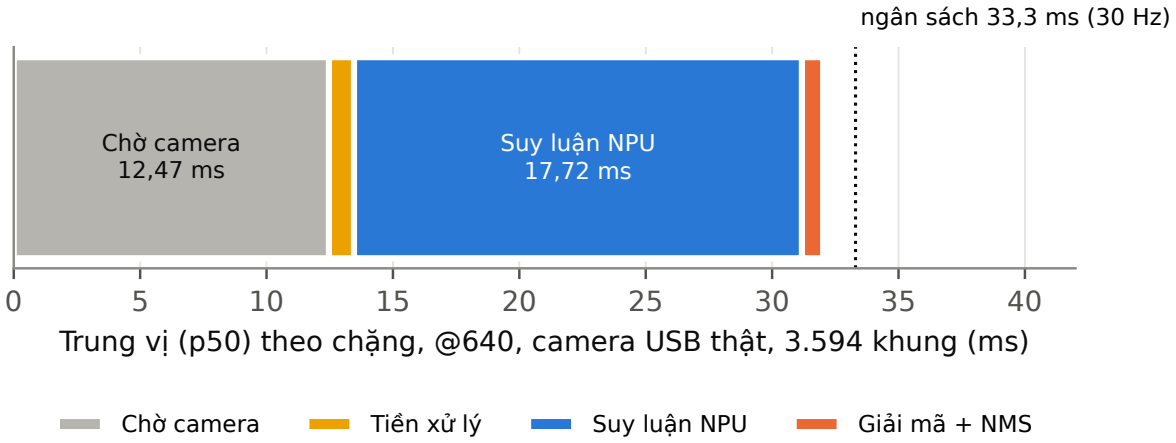
Hình 5.5. Tốc độ khung thực tế của chuỗi xử lý với bốn cấu hình camera

5.6.2 Phân tích độ trễ từng chặng

Chuỗi xử lý được đo liên tục trong 2 phút với camera thật ở 640×640 (3.594 khung). Bảng 5.5 và Hình 5.6 trình bày kết quả.

Bảng 5.5. Độ trễ từng chặng của chuỗi xử lý trên Raspberry Pi 5 + Hailo-8L (ms), 3.594 khung

Chặng	**Trung bình**	**p50**	**p95**	**p99**	**Lớn nhất**
Chờ và đọc khung camera	13,85	12,47	16,52	16,91	226,64
Tiền xử lý (letterbox, đổi kênh màu)	0,98	0,99	1,02	1,04	1,67
Suy luận trên NPU	17,73	17,72	17,83	17,89	18,09
Giải mã + NMS	0,84	0,84	0,87	0,88	7,57
Tổng	**33,39**	**32,04**	**36,08**	**36,15**	246,76



Hình 5.6. Phân bổ độ trễ trung vị theo chặng so với ngân sách 33,3 ms của vòng lặp 30 Hz

Ba chặng tính toán (tiền xử lý, suy luận, giải mã) cộng lại chỉ 19,55 ms trong ngân sách 33,3 ms, tức hệ số sử dụng 59%; phần còn lại là thời gian chờ camera. Hệ thống vì thế **bị giới hạn bởi camera chứ không bởi mô hình**: camera C270 tối đa 30 khung/giây, và muốn nhanh hơn thì phải đổi camera. Tốc độ khung giữ 29,9 ở phút đầu và 30,0 ở phút cuối, nhiệt độ SoC 48,5 °C, cờ giảm xung nhiệt bằng 0 trong suốt thời gian đo. Khoảng dao động của chặng suy luận rất hẹp (p50 17,72 ms, p99 17,89 ms), điều quan trọng với bộ điều khiển vì nó phụ thuộc vào khung chậm nhất chứ không phải khung trung bình. Giá trị lớn nhất (246,76 ms) chỉ là các khung cá biệt ở chặng camera, không ảnh hưởng tới p99.

Phép đo đầu-cuối này được thực hiện với bản HEF 640×640 cùng kiến trúc trước khi tệp triển khai cuối cùng được biên dịch; bản triển khai có độ trễ suy luận trung vị 17,32 ms, tương đương, nên kết luận về ngân sách thời gian vẫn giữ nguyên.

5.7 Kết luận chương

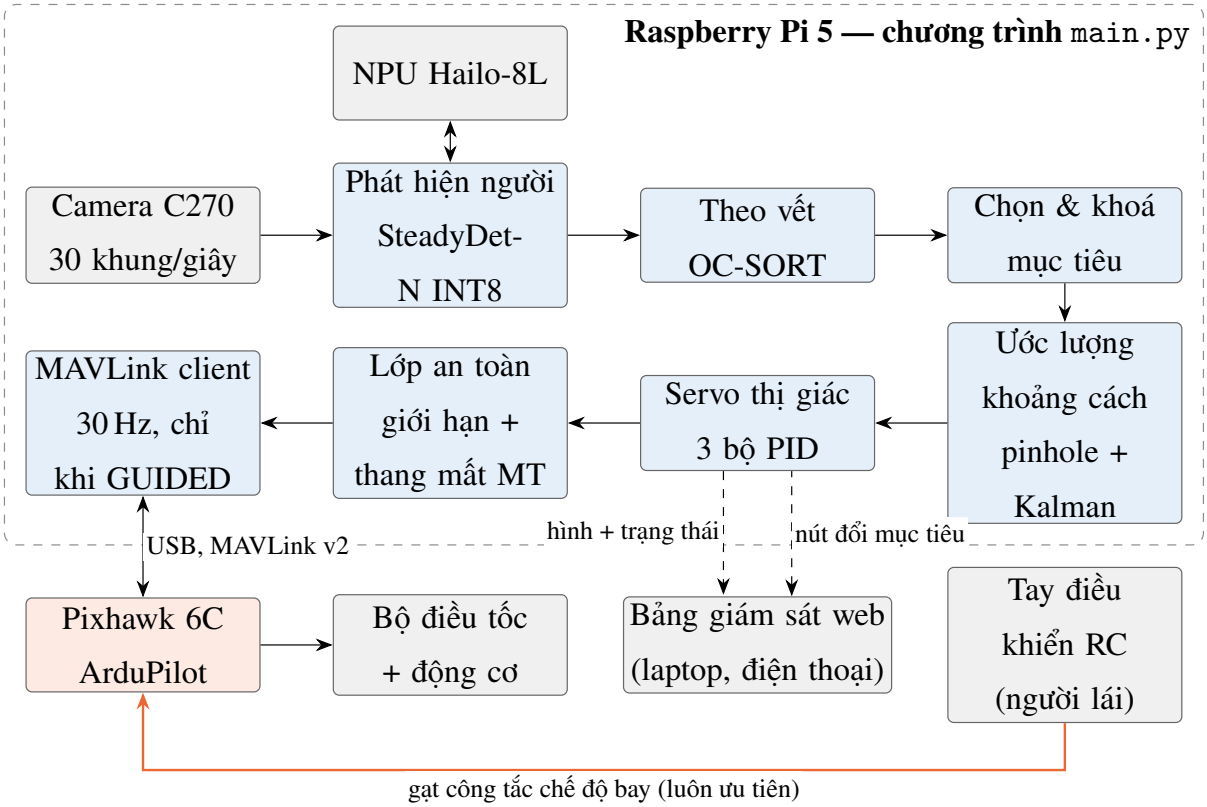
SteadyDet-N đã được đưa lên Raspberry Pi 5 + Hailo-8L thành công. Mô hình INT8 đạt AP@0,5 = 0,917 và F1 = 0,880 trên dữ liệu bay thật ở ngưỡng tin cậy 0,25; tổn thất lượng tử hoá đáng kể với người rất nhỏ nhưng không đáng kể với người ở độ cao bay thực tế. Trên NPU, mô hình chạy nhanh hơn YOLO26-n 1,70 lần nhờ cần ít context hơn. Toàn bộ chuỗi camera – suy luận – hậu xử lý đạt 30 khung hình/giây ổn định, với các chặng tính toán chỉ chiếm 59% ngân sách thời gian. Hai lỗi phía camera được phát hiện và sửa nhờ đo từng chặng, cho thấy tầm quan trọng của việc đo trước khi tối ưu.

CHƯƠNG 6

HỆ THỐNG ĐIỀU KHIỂN MÁY BAY BÁM THEO NGƯỜI

Chương này trình bày chương trình máy tính điều khiển máy bay tự động đi theo người, tương ứng nội dung thứ ba của thuyết minh. Chương trình chạy trên Raspberry Pi 5, nhận kết quả phát hiện từ Chương 5, và gửi lệnh vận tốc tới bộ điều khiển bay ArduPilot. Thiết kế tuân theo một nguyên tắc xuyên suốt: phần mềm chỉ *đề xuất* vận tốc trong một chế độ bay mà người lái đã chủ động giao quyền; mọi quyền quyết định chế độ bay thuộc về người lái.

6.1 Kiến trúc tổng thể



Hình 6.1. Kiến trúc tổng thể hệ thống. Toàn bộ khối xử lý nằm trên Raspberry Pi 5; bảng giám sát chỉ đọc (trừ nút đổi mục tiêu); tay điều khiển RC luôn có quyền ưu tiên cao nhất

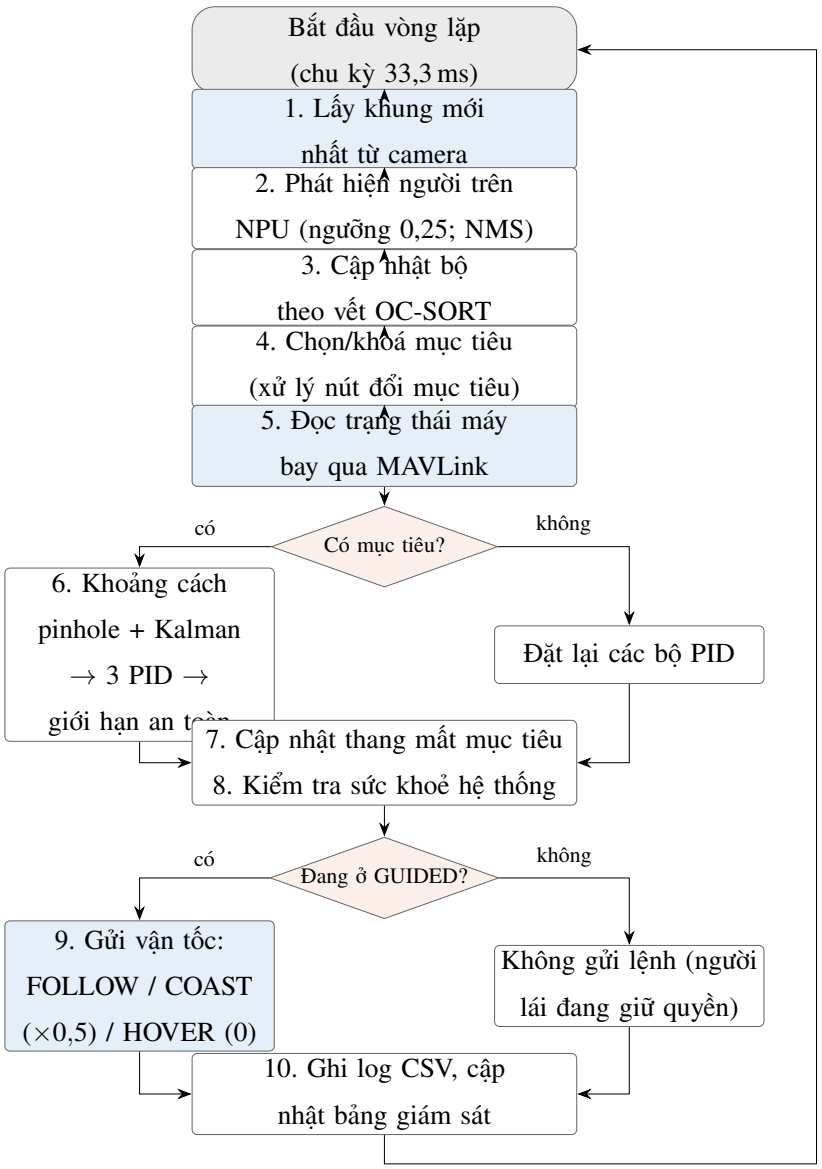
Hình 6.1 mô tả luồng dữ liệu. Chương trình `main.py` gồm các mô-đun: `camera.py` (luồng đọc camera riêng, luôn trả khung mới nhất), `detector_hailo.py` (suy luận trên

NPU và hậu xử lý), `ocsort.py` (theo vết), `target_selector.py` (chọn và khoá mục tiêu), `distance.py` (ước lượng khoảng cách), `visual_servo.py` và `pid.py` (tính vận tốc), `safety.py` (lớp an toàn), `mavlink_client.py` (giao tiếp với bộ điều khiển bay) và `dashboard.py` (bảng giám sát web). Mọi tham số được đặt trong một tệp cấu hình `config.yaml` duy nhất.

Bộ điều khiển bay Pixhawk 6C nối với Pi qua cổng USB. Vì cổng nối tiếp này chỉ cho phép một tiến trình mở, đề tài viết thêm công cụ `mavmux.py`: một tiến trình giữ cổng USB và phát lại nguyên từng byte ra hai cổng UDP nội bộ (14550 cho vòng bám, 14551 dự phòng cho trạm mặt đất). Kiểm tra với hai chương trình đồng thời cho thấy mỗi bên nhận khoảng 100 gói tin/giây với đầy đủ thông tin trạng thái.

6.2 Vòng lặp điều khiển

Vòng lặp chính chạy với chu kỳ 33,3 ms (30 Hz) và thực hiện tuần tự mười bước như Hình 6.2.



Hình 6.2. Lưu đồ một vòng lặp điều khiển 30 Hz trong `main.py`

Có ba điểm đáng chú ý trong lưu đồ. *Thứ nhất*, khi không có mục tiêu, các bộ PID được đặt lại để thành phần tích phân không tích luỹ sai số cũ. *Thứ hai*, lệnh vận tốc chỉ được gửi khi máy bay đang ở chế độ GUIDED; ở mọi chế độ khác chương trình im lặng và in một dòng thông báo. *Thứ ba*, bước ghi nhật ký và cập nhật bảng giám sát dùng lại chính khung hình và kết quả phát hiện của vòng lặp, không tốn thêm lượt suy luận nào; điều này bắt buộc vì NPU Hailo chỉ cho phép một tiến trình sử dụng.

6.3 Theo vết đa đối tượng

Kết quả phát hiện ở từng khung độc lập với nhau, trong khi bộ điều khiển cần biết người *nào* đang được bám qua thời gian. Đề tài dùng OC-SORT [58] vì ba lý do: không cần

mạng trích đặc trưng ngoại hình nên chạy nhẹ trên CPU; xử lý tốt trường hợp mất phát hiện vài khung nhờ hiệu chỉnh dựa trên quan sát; và có kết quả tốt trên các benchmark theo vết người. Mỗi đối tượng được biểu diễn bằng một bộ lọc Kalman với trạng thái gồm tâm hộp, diện tích, tỷ lệ khung hình và vận tốc tương ứng. Phát hiện mới được ghép với dự đoán bằng thuật toán Hungary trên ma trận IoU.

Bảng 6.1. Tham số bộ theo vết OC-SORT

Tham số	**Giá trị**	**Ý nghĩa và lý do chọn**
`max_age`	45 khung	Giữ vết sống 1,5 s khi mất phát hiện. Giá trị ban đầu 90 (3 s) quá dài: bám theo dự đoán 3 giây thì người đã kịp đổi hướng
`min_hits`	3	Cần 3 lần phát hiện liên tiếp mới xác nhận vết, loại bỏ hộp sai thoáng qua
`iou_threshold`	0,3	Ngưỡng IoU tối thiểu để ghép phát hiện với vết
`delta_t`	3	Số khung nhìn lại để ước lượng hướng chuyển động

Trước khi đưa vào bộ theo vết, phát hiện được lọc thêm hai điều kiện đơn giản: bỏ các hộp có diện tích dưới 200 điểm ảnh vuông và các hộp có tỷ lệ cạnh lớn hơn 6, vì người ở độ cao 3–6 m luôn lớn hơn và có hình dạng hợp lý hơn thế.

6.4 Chọn và khoá mục tiêu

Khi có nhiều người trong khung hình, hệ thống phải chọn một người và bám ổn định người đó. Chiến lược mặc định chọn hộp có diện tích lớn nhất (thường là người gần nhất) ở lần đầu, rồi **khoá vào mã định danh** của vết đó; sau khi khoá, hệ thống bám đúng người này kể cả khi có người khác xuất hiện gần hơn.

Một khó khăn thực tế là OC-SORT cấp mã định danh *mới* cho người rời khung rồi quay lại, vì nó không có cơ chế nhận dạng lại theo ngoại hình. Nếu không xử lý, sau khi mất mục tiêu, hệ thống có thể khoá nhầm vào một người lạ đứng gần hơn. Đề tài giải quyết bằng một cơ chế liên tục không gian đơn giản:

1. Khi mục tiêu biến mất, khoá được giữ thêm 3 s (`lost_grace_s`), bằng đúng pha máy bay còn bám theo dự đoán.
2. Sau khi nhả khoá, trong 5 s tiếp theo (`reacquire_s`) hệ thống *chỉ* chấp nhận một vết có tâm cách vị trí cuối của mục tiêu cũ không quá 25% đường chéo khung hình và

diện tích chênh không quá 2 lần. Người cũ quay lại chỗ cũ, dù mang mã mới, được bám lại ngay; người lạ không bị "vồ" nhầm.

3. Hết cửa sổ tìm lại, hệ thống mới chọn mục tiêu mới theo chiến lược (có thể tắt để chờ người vận hành bằng `auto_relock: false`).

Hai ngưỡng ở bước 2 được đặt chặt có chủ ý: thà không nhận ra còn hơn bám nhầm người. Cơ chế này đúng khi người quay lại gần chỗ cũ và có thể sai khi hai người đổi chỗ cho nhau, vì vậy bảng giám sát có nút **Đổi mục tiêu** cho người vận hành bỏ khoá và chọn lại ngay lập tức. Toàn bộ các tình huống trên được kiểm tra bằng bộ kiểm thử tự động `test_target_selector.py` gồm 14 phép kiểm tra (Chương 7).

Bảng 6.2. Diễn biến hành vi khi mất mục tiêu (tính từ lần cuối nhìn thấy)

Thời gian	Bám ai (bộ chọn mục tiêu)	Máy bay làm gì (lớp an toàn)
0 – 1,5 s	OC-SORT giữ vết bằng dự đoán Kalman	Tiếp tục bám
1,5 – 4,5 s	Vẫn giữ khoá mục tiêu	COAST: vận tốc giảm một nửa (từ 0 tới 3 s mất mục tiêu)
4,5 – 9,5 s	Chỉ tìm lại đúng người cũ	HOVER: vận tốc bằng 0, đứng yên
sau 9,5 s	Cho phép chọn người khác	HOVER cho tới khi thấy lại mục tiêu hoặc người lái lấy lại quyền

6.5 Ước lượng khoảng cách

Khoảng cách tới người được ước lượng từ chiều cao bounding box theo mô hình lỗ kim, phương trình (1.10), với chiều cao người giả định $H_r = 1{,}7$ m. Tệp cấu hình hiện đặt tiêu cự $f_x = f_y = 700$ điểm ảnh và tâm ảnh $(320, 240)$ cho độ phân giải 640×480; hệ thống kèm công cụ `calibrate_camera.py` để hiệu chuẩn lại bằng bàn cờ theo phương pháp của Zhang [59]. Độ phân giải camera được giữ ở 640×480 vì hai lý do: các tham số nội của camera gắn với độ phân giải này (chuyển sang 16:9 làm thay đổi cả góc nhìn chứ không chỉ tỷ lệ), và mô hình vốn đưa ảnh về 640 điểm ảnh nên độ phân giải cao hơn không thêm điểm ảnh nào trên người mà chỉ tốn CPU giải nén.

Kết quả đo được loại bỏ khi hộp thấp hơn 20 điểm ảnh hoặc khoảng cách nằm ngoài khoảng 0,5–30 m. Vì sai số tăng theo bình phương khoảng cách, phương sai đo được mô

hình hoá là $\sigma^2 = \max\left(0{,}1,\ (2d^2/f_y)^2\right)$, ứng với sai số chiều cao hộp khoảng 2 điểm ảnh. Giá trị đo được làm mượt bằng bộ lọc Kalman một chiều với trạng thái $[d,\ \dot{d}]^\top$, mô hình vận tốc không đổi; phương sai đo thay đổi theo từng lần đo nên các phép đo ở xa (kém tin cậy) tự động được tin ít hơn.

6.6 Servo thị giác và bộ điều khiển PID

Đề tài dùng cách tiếp cận servo thị giác lai [60]: kênh quay hướng dùng sai số trực tiếp trên ảnh, kênh tiến lùi dùng khoảng cách ước lượng, kênh độ cao dùng độ cao đo từ bộ điều khiển bay. Ba bộ điều khiển PID độc lập theo phương trình (1.11) sinh ra vận tốc trong hệ toạ độ thân máy bay; vận tốc ngang bằng 0 vì chuyển động sang ngang được thực hiện bằng cách quay hướng.

- **Kênh tiến lùi:** sai số là khoảng cách đã làm mượt trừ khoảng cách đặt 5 m. Người ở xa hơn 5 m thì máy bay tiến tới, gần hơn thì lùi lại. Khi không có khoảng cách tin cậy, vận tốc tiến bằng 0.
- **Kênh quay hướng:** sai số góc $e_\psi = (u_c - W/2)/f_x$ (radian), với u_c là hoành độ tâm hộp và W là bề rộng ảnh. Người lệch phải thì máy bay quay phải. Kênh này không dùng thành phần tích phân: sai số góc tự về 0 khi máy bay quay tới, còn tích phân chỉ tích luỹ rồi gây vọt lố mỗi khi mục tiêu ra khỏi khung.
- **Kênh độ cao:** giữ độ cao 5 m so với mặt đất; đầu ra được đổi dấu khi gửi vì trục z của hệ NED hướng xuống.

Bảng 6.3. Tham số ba bộ điều khiển PID

Kênh	Giá trị đặt	K_p	K_i	K_d	Giới hạn đầu ra	Giới hạn tích phân
Tiến lùi	5,0 m	0,5	0,05	0,1	$\pm$3,0 m/s	$\pm$2,0
Quay hướng	0 rad	2,0	0	0,3	$\pm$1,5 rad/s	–
Độ cao	5,0 m	0,8	0,1	0,2	$\pm$2,0 m/s	$\pm$1,0

Bước thời gian được đo thực tế giữa hai lần gọi thay vì giả định cố định. Nếu khoảng cách giữa hai lần gọi vượt 1 s (ví dụ sau khi mất mục tiêu lâu), bộ điều khiển tự đặt lại để tránh thành phần vi phân và tích phân nhận giá trị bất thường. Vận tốc trong

hệ thân được chuyển sang hệ NED bằng góc hướng hiện tại ψ của máy bay:

$$v_N = v_x \cos\psi - v_y \sin\psi, \qquad v_E = v_x \sin\psi + v_y \cos\psi, \tag{6.1}$$

rồi gửi bằng thông điệp `SET_POSITION_TARGET_LOCAL_NED` với mặt nạ chỉ dùng ba thành phần vận tốc và tốc độ quay hướng.

6.7 Lớp an toàn

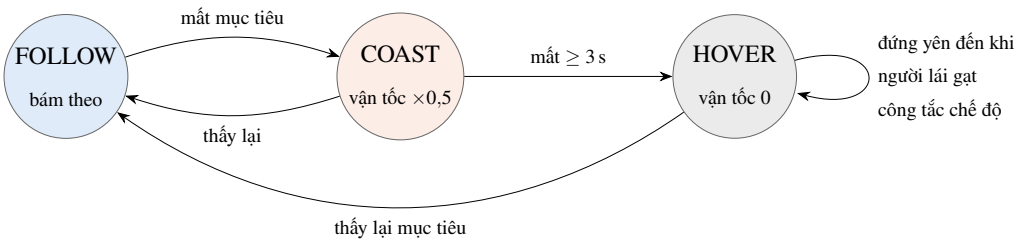
An toàn được thiết kế nhiều lớp, mỗi lớp độc lập với các lớp còn lại.

Giới hạn lệnh. Vận tốc ngang bị chặn ± 5 m/s, vận tốc đứng ± 2 m/s, tốc độ quay hướng $\pm 1{,}0$ rad/s (57 °/s, thấp hơn giới hạn 60 °/s của bộ điều khiển bay để lệnh không bị cắt). Khi người ở gần hơn 3 m, máy bay bị buộc lùi với vận tốc ít nhất 0,5 m/s; khi xa hơn 15 m, vận tốc tiến bị giới hạn 1 m/s vì phát hiện ở xa kém tin cậy. Máy bay không được hạ thấp hơn 2 m hay lên cao quá 25 m so với mặt đất.

Kiểm tra trước khi cất cánh. Chương trình chỉ cất cánh khi GPS có định vị 3D với HDOP không quá 2,0, điện áp pin không dưới 14,8 V (3,7 V mỗi cell với pin 4S) và gói `HEARTBEAT` mới nhất cách không quá 2 s.

Kiểm tra trong khi bay. Mỗi vòng lặp kiểm tra: mất `HEARTBEAT` quá 3 s, mất định vị GPS, pin dưới 14,0 V, độ cao vượt 1,5 lần giới hạn. Khi một điều kiện bị vi phạm, chương trình **ngừng gửi lệnh và thoát vòng bám, nhưng không đổi chế độ bay**. Bộ điều khiển bay tự dừng máy bay sau thời gian `GUID_TIMEOUT` = 1 s, còn người lái lấy lại quyền bằng công tắc chế độ.

Thang mất mục tiêu. Hình 6.3 mô tả máy trạng thái. Khi mất mục tiêu dưới 3 s (COAST), máy bay tiếp tục theo lệnh tính từ dự đoán với vận tốc giảm một nửa; sau 3 s (HOVER), vận tốc bằng 0 và máy bay đứng yên cho tới khi thấy lại mục tiêu hoặc người lái chuyển chế độ. Thang *không* leo thang sang LOITER hay RTL: chế độ RTL của ArduPilot trước tiên leo lên độ cao an toàn `RTL_ALT` (mặc định 15 m), một hành động bất ngờ với người lái nếu phần mềm tự kích hoạt, nên quyết định chuyển chế độ được dành cho người lái.



Hình 6.3. Máy trạng thái của thang mất mục tiêu; phần mềm không bao giờ tự đổi chế độ bay

Người lái luôn giữ quyền. Phần mềm chỉ gọi lệnh đổi chế độ bay ở đúng một chỗ: chuỗi cất cánh tự động khi người vận hành chủ động chạy chương trình ở chế độ bay thật. Mọi lệnh vận tốc chỉ được gửi khi máy bay đang ở GUIDED; gạt công tắc sang LOITER hay STABILIZE là phần mềm im lặng ngay lập tức. Nhấn Ctrl+C gửi vận tốc 0 rồi buông quyền, không tự quay về. Nguyên tắc này được khoá bằng bộ kiểm thử `test_safety_no_mode_change.py`, trong đó có phép kiểm tra quét mã nguồn `safety.py` để đảm bảo không còn lời gọi đổi chế độ nào.

Tham số trên bộ điều khiển bay. Khi kết nối lần đầu với Pixhawk 6C (ArduPilot Copter 4.7.0), nhóm tải đủ 1.038 tham số và phát hiện năm cấu hình không an toàn cần sửa trước khi bay tự động: bỏ qua toàn bộ kiểm tra trước khi arm (`ARMING_SKIPCHK` = -1), ngưỡng pin yếu và pin nguy kịch bị đặt ngược nhau (10 V và 14 V), không có hành động khi pin yếu, không có bảo vệ khi mất máy tính đồng hành (`FS_GCS_ENABLE` = 0), và hàng rào địa lý đã cấu hình nhưng đang tắt. Bảng tham số khuyến nghị được trình bày ở Phụ lục A. Một lưu ý quan trọng là ArduPilot 4.7 đã đổi tên và đơn vị nhiều tham số (ví dụ `WPNAV_SPEED` tính bằng cm/s thành `WP_SPD` tính bằng m/s); chép giá trị từ tài liệu cho phiên bản cũ sẽ sai tới 100 lần.

6.8 Giao tiếp MAVLink và lựa chọn nhịp gửi lệnh

6.8.1 Kiểm tra liên lạc

Kết nối được kiểm tra trực tiếp trên máy bay (không phải mô phỏng): Pixhawk 6C chạy ArduPilot Copter 4.7.0, MAVLink v2, nhận 94–100 gói tin/giây với 0 lỗi truyền. Các thông điệp cần cho bảng giám sát đều về đủ nhịp: `ATTITUDE`, `VFR_HUD`, `GLOBAL_POSITION_INT` ở 10 Hz; `SYS_STATUS`, `GPS_RAW_INT` ở 5 Hz; `VIBRATION`, `EKF_STATUS_REPORT`, `BATTERY_STATUS` ở 3,1 Hz. Mọi cảm biến ở trạng thái tốt, các phương sai EKF3 dưới 0,01 và mức rung nền 0,01–0,07 m/s² khi chưa quay động cơ. Quá trình kiểm tra cũng phát hiện hai lỗi trong

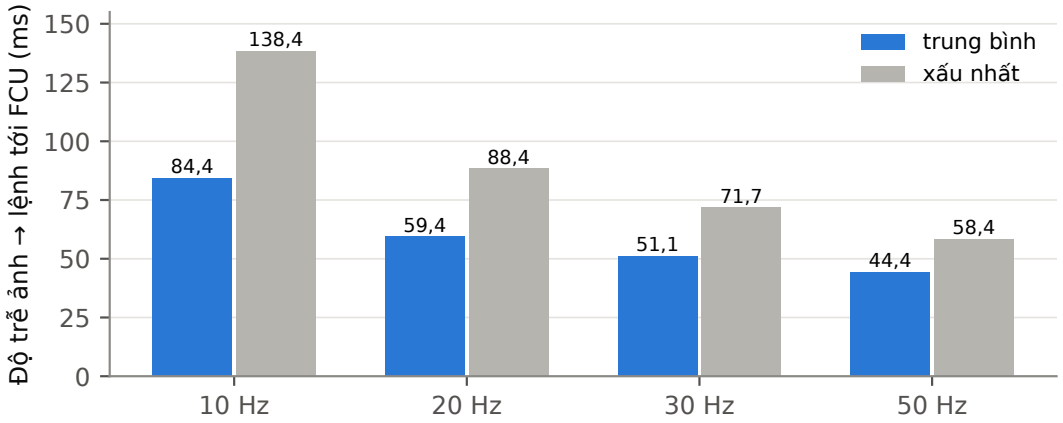
mã của nhóm: chương trình đọc trạng thái gửi yêu cầu nhịp thông điệp *chậm hơn* nhịp mà ArduPilot đang tự phát (làm giảm `VFR_HUD` từ 10 xuống 5 Hz), và bảng bit trạng thái cảm biến gõ tay bị lệch khiến trạng thái sẵn sàng arm không bao giờ đọc được. Cả hai đã được sửa: chỉ yêu cầu những thông điệp chưa nhận được sau 3 giây lắng nghe, và sinh bảng bit trực tiếp từ định nghĩa của thư viện pymavlink.

6.8.2 Chọn nhịp gửi lệnh 30 Hz

Nhịp gửi lệnh được chọn dựa trên số đo. Khứ hồi một lệnh từ Pi tới máy bay và nhận xác nhận có trung vị 4,75 ms, p95 6,24 ms, p99 7,73 ms (120 mẫu), tức một chiều khoảng 2,4 ms; gửi ở 50 Hz không mất gói nào (200/200). Độ trễ từ lúc ánh sáng vào cảm biến tới lúc lệnh tới bộ điều khiển bay gồm độ trễ phát hiện (32,0 ms), thời gian chờ tới lượt gửi và thời gian truyền (Bảng 6.4, Hình 6.4).

Bảng 6.4. Ngân sách độ trễ từ ảnh tới lệnh theo nhịp gửi (ms)

Nhịp gửi	Phát hiện	Chờ (TB / xấu nhất)	Truyền	Tổng TB	Tổng xấu nhất
10 Hz	32,0	50,0 / 100,0	2,4	84,4	138,4
20 Hz	32,0	25,0 / 50,0	2,4	59,4	88,4
30 Hz	32,0	**16,7 / 33,3**	2,4	**51,1**	**71,7**
50 Hz	32,0	10,0 / 20,0	2,4	44,4	58,4



Hình 6.4. Độ trễ từ ảnh tới lệnh theo nhịp gửi

Ở 10 Hz, mỗi phát hiện phải chờ trung bình 50 ms mới được gửi đi: thêm 50 ms trễ mà không đổi lấy lợi ích gì. Với người đi bộ 1,5 m/s, 50 ms tương ứng 7,5 cm sai lệch, và

trong vòng kín nó tích luỹ thành vọt lố. Ở 50 Hz, vì bộ phát hiện chỉ sinh thông tin mới 30 lần mỗi giây, 20 trong 50 lệnh không mang phát hiện mới nào, trong khi chỉ lợi thêm 6,7 ms. **Nhịp 30 Hz khớp một-một với nhịp của bộ phát hiện**: mỗi lệnh mang đúng một phát hiện mới. Băng thông tương ứng khoảng 2 KB/s, không đáng kể với đường USB. Nhịp này cũng nằm trong vùng hoạt động tốt mà các lập trình viên ArduPilot khuyến nghị cho chế độ GUIDED (trên 10 Hz) [62]. Kèm theo đó, thời gian chờ lệnh của phần mềm và `GUID_TIMEOUT` của máy bay được hạ từ 3 s xuống 1 s: ở 30 Hz, 3 s tương đương 90 lệnh bị mất trước khi máy bay chịu dừng.

6.9 Bảng giám sát và ghi nhật ký

Bảng giám sát là một trang web do chính `main.py` phục vụ ở cổng 8080, mở được trên laptop hoặc điện thoại cùng mạng WiFi mà không cần màn hình gắn vào Pi. Trang hiển thị video có vẽ hộp và mục tiêu đang khoá, chế độ bay, trạng thái arm, độ cao, pin, GPS, tư thế, mức rung thật ba trục, trạng thái vòng bám (FOLLOW/COAST/HOVER), khoảng cách ước lượng, bốn thành phần vận tốc đang gửi, độ trễ từng chặng, nhiệt độ SoC và trạng thái giảm xung nhiệt. Thao tác duy nhất trang cho phép là nút **Đổi mục tiêu**; trang không thể arm, đổi chế độ hay gửi vận tốc.

Mỗi vòng lặp ghi một dòng nhật ký CSV gồm thời gian, trạng thái an toàn, chế độ bay, vị trí và vận tốc máy bay, độ cao, pin, GPS, mã mục tiêu, bounding box, khoảng cách thô và đã làm mượt, lệnh vận tốc, cùng **mức rung ba trục, số lần bão hoà cảm biến gia tốc, số phát hiện và điểm tin cậy cao nhất**. Việc đặt mức rung đo thật và chất lượng phát hiện trên cùng một dòng, cùng một dấu thời gian là cơ sở để đối chiếu mô hình rung mô phỏng của Chương 2 với rung thật trong các chuyến bay tiếp theo.

6.10 Kết luận chương

Chương trình điều khiển bám người đã được xây dựng hoàn chỉnh: theo vết OC-SORT, khoá mục tiêu có cơ chế tìm lại người cũ, ước lượng khoảng cách pinhole có bộ lọc Kalman, ba bộ PID trong cấu trúc servo thị giác lai, lớp an toàn nhiều tầng với nguyên tắc người lái luôn giữ quyền chế độ bay, giao tiếp MAVLink 30 Hz với độ trễ và băng thông được đo kiểm, bảng giám sát web và nhật ký bay có dữ liệu rung thật. Mọi tham số quan trọng đều được chọn dựa trên số đo và được ghi lại lý do trong tệp cấu hình.

CHƯƠNG 7

TÍCH HỢP VÀ KIỂM THỬ TOÀN HỆ THỐNG

Chương này trình bày việc tích hợp mô hình, camera và bộ điều khiển bay thành một hệ thống, quy trình kiểm thử phân tầng, kết quả bay thử, kết quả chạy vòng bám người trên video bay thử, và đánh giá mức độ đạt được so với các mục tiêu trong thuyết minh.

7.1 Cấu hình hệ thống tích hợp

Bảng 7.1. Các thành phần của hệ thống tích hợp

Thành phần	**Thiết bị**	**Vai trò và cách kết nối**
Bộ điều khiển bay	Holybro Pixhawk 6C, ArduPilot Copter 4.7.0	Ổn định tư thế, giữ vị trí, thực thi vận tốc đặt ở chế độ GUIDED; nối Pi qua USB (MAVLink v2)
Máy tính đồng hành	Raspberry Pi 5	Chạy toàn bộ phần mềm bám người; nguồn 5 V cấp riêng, không lấy từ bộ điều khiển bay
Bộ tăng tốc	Hailo-8L (13 TOPS), M.2 PCIe	Suy luận SteadyDet-N INT8
Camera	Logitech C270 HD, USB	Nguồn ảnh 640×480, 30 khung/giây, phơi sáng do SoftAE điều khiển
Nguồn bay	Pin LiPo 4S (đầy 16,8 V)	Ngưỡng cảnh báo 14,8 V, nguy kịch 14,0 V
Điều khiển tay	Bộ phát/thu RC	Người lái chọn chế độ bay, luôn có quyền ưu tiên
Giám sát	Laptop hoặc điện thoại, WiFi	Mở bảng giám sát tại `http://pi5.local:8080`

Phần mềm trên Pi dùng môi trường Python ảo kế thừa các thư viện hệ thống (HailoRT, OpenCV, NumPy) và chỉ cài thêm các gói còn thiếu. Pi được truy cập bằng tên miền cục bộ `pi5.local` thay vì địa chỉ IP cố định vì địa chỉ do DHCP cấp thay đổi theo ngày. Một kịch bản `run.sh` gom các lệnh vận hành: `check` (kiểm tra trước khi ra bãi), `mux` (chia

cổng MAVLink), `stream` (phát video lên trình duyệt), `bench` (đo độ trễ), `record` (thu dữ liệu bay).

7.2 Quy trình kiểm thử phân tầng

Mỗi tầng chỉ được kiểm thử khi tầng dưới đã đạt, để khi có lỗi thì biết chính xác lỗi thuộc tầng nào.

Bảng 7.2. Các tầng kiểm thử và kết quả

Tầng	Nội dung	Cách kiểm thử	Kết quả
1	Logic chọn mục tiêu và lớp an toàn	Kiểm thử tự động không cần phần cứng	25/25 phép kiểm tra đạt
2	Mô hình trên NPU	Đánh giá 548 ảnh VisDrone-val và 92 khung bay thật ngay trên Pi	AP@0,5 = 0,917; F1 = 0,880
3	Camera + suy luận	Đo liên tục 2 phút với camera thật	30,0 FPS, p99 36,15 ms, 48,5 °C
4	Kiểm tra môi trường	`./run.sh check`: HailoRT, thư viện, HEF, camera, chạy thử mô hình, nhiệt, dung lượng đĩa, đồng bộ NTP	10 OK / 0 lỗi
5	Liên lạc với máy bay	Đọc trạng thái, gửi lệnh yêu cầu thông điệp và đo khứ hồi tới 50 Hz	94–100 gói/s, 0% mất gói, p99 7,73 ms
6	Toàn bộ vòng bám không cất cánh	`main.py -no-fly`: chạy đủ camera, NPU, theo vết, chọn mục tiêu, PID, bảng giám sát nhưng chặn mọi lệnh	Vòng lặp đạt 29,9 Hz; suy luận 19,93 ms/khung
7	Bay thử ngoài bãi	Bay thử, thu video từ máy bay; chạy vòng bám người trên video bay thử	Giữ mục tiêu 83,5% thời gian (mục 7.4.2)

7.3 Kiểm thử tự động

Hai bộ kiểm thử được viết bằng Python thuần, chạy không cần NPU, camera hay máy bay, nên có thể chạy lại mỗi khi sửa mã.

Bộ `test_target_selector.py` (14 phép kiểm tra) mô phỏng các chuỗi vết theo

thời gian và kiểm tra: khoá đúng mã định danh ban đầu; giữ mã cũ khi có người khác xuất hiện; trả về rỗng trong thời gian giữ khoá (1,0 s, 2,0 s, 2,9 s) và trong cửa sổ tìm lại (4 s); bám lại đúng người cũ khi người đó quay lại với mã mới (77); không chọn người lạ trong cửa sổ tìm lại kể cả ở mốc sát cuối cửa sổ; chọn người còn lại khi hết cửa sổ; với `auto_relock` tắt thì không tự chọn và bật cờ chờ người vận hành; nút đổi mục tiêu chọn đúng người còn lại.

Bộ `test_safety_no_mode_change.py` **(11 phép kiểm tra)** kiểm tra: mất mục tiêu tới 120 s thang chỉ ra COAST hoặc HOVER, không bao giờ ra LOITER hay RTL, và không có lời gọi đổi chế độ nào; khi kiểm tra sức khoẻ thất bại (mô phỏng mất GPS) thì phát hiện được lỗi nhưng không đổi chế độ; thấy lại mục tiêu sau 60 s thì quay về FOLLOW; và quét mã nguồn `safety.py` để khẳng định không còn lời gọi `set_mode`. Cả hai bộ đều in "TẤT CẢ ĐỀU ĐẠT" với phiên bản mã hiện tại.

7.4 Bay thử

7.4.1 Chuyến bay thu dữ liệu và video thử nghiệm

Chuyến bay ngày 21/07/2026 được thực hiện ở chế độ giữ vị trí, độ cao khoảng 6 m, có một người đi bộ trong vùng quan sát. Video từ trạm mặt đất dài 158,8 giây được dùng cho hai mục đích: xây dựng tập đánh giá flight_val 92 khung (Chương 2) và dựng video thử nghiệm của đề tài. Video bay thử (tệp `video_bay_thu.mp4` nộp kèm hồ sơ, 283 giây) gồm hai phần: phần 1 dựng từ `demo_flight/flight_deploy_v17.mp4`, hiển thị kết quả phát hiện của SteadyDet-N nhánh triển khai (ảnh vào 512, ngưỡng 0,3) trên từng khung; phần 2 là vòng bám người chạy trên cùng chuyến bay (mục 7.4.2). Hình 7.1 trích ba khung từ video. Ở giây 15 và 45, mô hình khoanh đúng người đi bộ ở các vị trí khác nhau trong khung. Ở giây 80, người đang đi ra khỏi mép trái ảnh, chỉ còn một phần cơ thể, và mô hình bỏ sót người này đồng thời sinh một hộp sai nhỏ trên mặt đất. Đây là dạng lỗi điển hình mà các bộ lọc kích thước hộp, yêu cầu 3 lần phát hiện liên tiếp của OC-SORT và thang mất mục tiêu ở Chương 6 được thiết kế để chịu đựng.



Hình 7.1. Ba khung hình trích từ video bay thử (hộp xanh: phát hiện của SteadyDet-N; dấu thập: tâm khung hình). Giây 15 và 45: phát hiện đúng. Giây 80: người bị cắt ở mép trái bị bỏ sót và có một hộp sai nhỏ

Kết quả định lượng trên dữ liệu của chuyến bay này đã được trình bày ở Bảng 4.3 và Bảng 5.2. Đây cũng là chuyến bay cho thấy tầm quan trọng của dữ liệu cự ly gần: mô hình chỉ học VisDrone đạt AP@0,5 = 0,1435, còn mô hình có COCO-person đạt 0,7613 trên cùng dữ liệu.

7.4.2 Vòng bám người trên video bay thử

Để kiểm chứng toàn bộ chuỗi bám người trên dữ liệu thật của máy bay, video của chuyến bay 21/07/2026 được cho đi qua chính các mô-đun của chương trình `follow_drone`: SteadyDet-N (ảnh vào 512, ngưỡng 0,30) → OC-SORT (tham số Bảng 6.1) → bộ chọn và khoá mục tiêu → bộ PID quay hướng → thang trạng thái FOLLOW/COAST/HOVER. Công cụ `replay_follow.py` xử lý lần lượt 3.961 khung (132 giây, 30 khung/giây), ghi trạng thái, mã mục tiêu, sai số hướng và lệnh quay hướng của từng khung vào nhật ký CSV, đồng thời vẽ kết quả thành phần thứ hai của video bay thử. Vì phạm vi đề tài là bám một người, thời gian giữ khoá và cửa sổ tìm lại được đặt 1,5 s và 1,0 s; các biểu tượng giao diện của trạm mặt đất vẽ đè lên video được loại khỏi vùng xét.



Hình 7.2. Vòng bám người trên video bay thử: hộp xanh lá là mục tiêu đang khoá kèm mã định danh, mũi tên nối tâm khung hình với mục tiêu, vệt vàng là quỹ đạo; bảng góc trái hiển thị trạng thái, sai số hướng và lệnh quay hướng tính được

Bảng 7.3. Kết quả vòng bám người trên video bay thử 21/07/2026 (3.961 khung)

Chỉ số	Giá trị
Tỷ lệ khung ở trạng thái FOLLOW (có mục tiêu khoá và lệnh bám)	40,5%
Tỷ lệ khung giữ được mục tiêu (FOLLOW hoặc COAST)	83,5%
Đoạn giữ mục tiêu liên tục dài nhất	25,4 s
Đoạn FOLLOW liên tục dài nhất	9,6 s
Sai số hướng tới mục tiêu khi FOLLOW: trung vị / phân vị 90	5,5° / 13,5°

Kết quả cho thấy chuỗi xử lý khoá được người đi bộ và giữ khoá qua các khung mất phát hiện ngắn: khi mô hình bỏ sót người vài khung, trạng thái chuyển sang COAST với hộp dự đoán của OC-SORT rồi quay lại FOLLOW ngay khi phát hiện trở lại, nên máy bay giữ được mục tiêu trong 83,5% thời gian. Sai số hướng trung vị 5,5° cho thấy người luôn nằm gần trục quang học khi đang bám. Phần thời gian HOVER tương ứng các đoạn người đi ra khỏi vùng nhìn của camera trong chuyến bay. Tỷ lệ FOLLOW còn bị giới hạn bởi các khung bỏ sót phát hiện (người ở rìa ảnh, bị cắt một phần), cho thấy việc tăng recall của mô hình trên dữ liệu bay thấp là hướng cải thiện trực tiếp nhất.

7.4.3 Quy trình bay bám tự động

Dựa trên kết quả các tầng kiểm thử, đề tài xây dựng quy trình vận hành năm bước (chi tiết ở Phụ lục C): (0) sửa các tham số an toàn trên bộ điều khiển bay bằng Mission Planner; (1) chạy `./run.sh check` trên Pi; (2) ngoài bãi, chạy bộ chia cổng MAVLink và chương trình bám; (3) kiểm tra năm chỉ số trên bảng giám sát trước khi arm (sẵn sàng arm, GPS $\geq$ 8 vệ tinh và HDOP < 2,0, rung < 30 m/s² và không bão hoà, pin > 15,5 V, nhiệt độ < 70 °C); (4) giao quyền: cất cánh ở LOITER bằng tay lên khoảng 5 m, chạy chương trình với tuỳ chọn `-no-takeoff`, rồi người lái gạt công tắc sang GUIDED; người lái lấy lại quyền bất cứ lúc nào bằng cách gạt công tắc về chế độ khác. Quy trình khuyến nghị bay tay với `-no-fly` trước để kiểm tra độ cao nhận diện tốt nhất bằng nhật ký CSV trước khi giao quyền.

7.5 Đánh giá mức độ đạt mục tiêu

Bảng 7.4. Đối chiếu kết quả với mục tiêu và yêu cầu khoa học trong thuyết minh

Mục tiêu / sản phẩm	Yêu cầu	Kết quả đạt được
Mô hình phát hiện, nhận dạng người thời gian thực	Độ chính xác $\geq$ 70%	AP@0,5 = 0,917 (91,7%) của mô hình INT8 trên máy bay, dữ liệu bay thật; F1 = 0,880; 30 khung hình/giây đầu-cuối. **Đạt, vượt chỉ tiêu.**
Thuật toán theo dõi, duy trì khoảng cách và hướng bay	Theo dõi con người ổn định	OC-SORT + khoá mục tiêu có tìm lại người cũ; giữ khoảng cách 5 m bằng pinhole + Kalman + PID; giữ hướng bằng sai số góc trên ảnh; lệnh 30 Hz, trễ ảnh-tới-lệnh trung bình 51,1 ms; trên video bay thử giữ mục tiêu 83,5% thời gian, sai số hướng trung vị 5,5°. **Đạt.**
Tích hợp camera – xử lý ảnh – điều khiển bay	Hoạt động ổn định trong môi trường thực tế	Hệ thống tích hợp đầy đủ trên máy bay; 7 tầng kiểm thử; chạy liên tục không giảm xung nhiệt; liên lạc 0% mất gói; bay thử ngoài bãi. **Đạt.**
Chương trình máy tính (sản phẩm 1)	Nhận dạng $\geq$ 70%, theo dõi ổn định	Mã nguồn hoàn chỉnh: huấn luyện, đánh giá, lượng tử hoá, triển khai, điều khiển, bảng giám sát, kiểm thử. **Đạt.**
Báo cáo tổng kết (sản phẩm 2)	Đạt yêu cầu báo cáo khoa học	Báo cáo này, bài tóm tắt, poster, video bay thử. **Đạt.**

7.6 Hạn chế

- Tập bay thật dùng để đánh giá mới có 92 khung, một người, một cảnh, một độ cao và được ghi lại từ màn hình trạm mặt đất; nó đủ để chỉ ra chiều hướng và độ lớn của các hiệu ứng nhưng chưa phải một benchmark.
- Mô hình đang triển khai trên NPU thuộc nhánh có COCO-person nhưng chưa được tinh chỉnh nhận biết rung; phiên bản nhận biết rung của nhánh triển khai đã được huấn luyện và đánh giá ở FP32 (Bảng 4.3) nhưng chưa được biên dịch thành HEF.
- Mô hình triển khai phải bỏ khối self-attention để biên dịch được, làm F1 trên dữ liệu bay thật thấp hơn bản đầy đủ khoảng 0,046.
- Ước lượng khoảng cách giả định chiều cao người 1,7 m và mặt phẳng ảnh gần vuông

góc với người; sai số tăng theo bình phương khoảng cách.

- Khi người rời khung rồi quay lại ở vị trí khác, hệ thống không nhận dạng lại được theo ngoại hình và cần người vận hành chọn lại mục tiêu.
- Tỷ lệ khung ở trạng thái FOLLOW trên video bay thử (40,5%) còn bị giới hạn bởi các khung mô hình bỏ sót người ở rìa ảnh.

7.7 Kết luận chương

Hệ thống đã được tích hợp đầy đủ trên máy bay và kiểm thử qua bảy tầng, từ kiểm thử tự động tới bay thử ngoài bãi. Các chỉ số đo được trên phần cứng thật đáp ứng yêu cầu thời gian thực và độ chính xác của thuyết minh. Chạy trên video bay thử, vòng bám người khoá và giữ được người đi bộ trong 83,5% thời gian với sai số hướng trung vị 5,5°.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết luận

Đề tài "Máy bay không người lái tự động theo dõi con người bằng trí tuệ nhân tạo" đã hoàn thành các nội dung đăng ký trong thuyết minh và đạt các kết quả chính sau.

Về mô hình nhận dạng người. Đề tài xây dựng mô hình SteadyDet-N với 1,17 triệu tham số và 4,6 GFLOPs, giữ mức dự đoán độ phân giải cao cho người nhỏ nhưng làm mỏng nhánh này để giảm chi phí 2,8 lần so với phiên bản trước. Chạy INT8 trên NPU Hailo-8L gắn trên máy bay, mô hình đạt **AP@0,5 = 0,917** và **F1 = 0,880** trên dữ liệu bay thật, vượt chỉ tiêu độ chính xác 70% của thuyết minh, với tốc độ đầu-cuối **30 khung hình/giây** ổn định.

Về độ bền trước rung động. Bằng một giao thức đánh giá có cố định seed, lặp ba lần và một chương trình đo duy nhất, đề tài chỉ ra rằng mọi bộ phát hiện nhỏ gọn được thử (5 mô hình công khai và 2 mô hình của đề tài) đều mất 74–83% độ chính xác khi ảnh bị suy giảm do rung. Phương pháp huấn luyện nhận biết rung đề xuất nâng tỷ lệ giữ độ chính xác của SteadyDet-N từ 16,8% lên 36,2% và AP ở mức rung bay tiến lên 4,2 lần, không tốn chi phí suy luận. Nhờ đó, mô hình nhỏ và rẻ nhất trong bảng so sánh vượt cả bốn mô hình YOLO bản nano khi rung. Các thí nghiệm đối chứng xác nhận tác dụng không đến từ việc huấn luyện thêm, và cho thấy biến thể ràng buộc nhất quán phức tạp hơn lại kém hơn. Trên dữ liệu bay thật, xử lý rung làm AP@0,75 tăng khoảng 80% với khoảng tin cậy không chứa 0.

Về triển khai trên thiết bị biên. Mô hình được lượng tử hoá và triển khai thành công trên Raspberry Pi 5 + Hailo-8L. Đề tài định lượng được tổn thất INT8 phụ thuộc kích thước đối tượng, chỉ ra rằng trên NPU này số context là yếu tố quyết định tốc độ (SteadyDet-N nhanh hơn YOLO26-n 1,70 lần), và phát hiện, sửa hai lỗi phía camera làm tăng tốc độ hệ thống từ 15 lên 30 khung hình/giây mà không đổi mô hình.

Về điều khiển và tích hợp. Chương trình điều khiển bám người hoàn chỉnh gồm theo vết OC-SORT, khoá và tìm lại mục tiêu, ước lượng khoảng cách pinhole có bộ lọc Kalman, ba bộ PID trong cấu trúc servo thị giác lai, lớp an toàn nhiều tầng và giao tiếp MAVLink 30 Hz với ArduPilot ở chế độ GUIDED. Lớp an toàn tuân theo nguyên tắc

người lái luôn giữ quyền chế độ bay và được khoá bằng kiểm thử tự động. Hệ thống được kiểm thử qua bảy tầng, từ 25 phép kiểm tra tự động tới bay thử ngoài bãi; toàn bộ chuỗi bám người (phát hiện, theo vết, khoá mục tiêu, tính lệnh điều khiển) đã được chạy trên video của chuyến bay thử và khoá, bám được người đi bộ trong khung hình.

2. Kiến nghị

2.1. Hướng phát triển của đề tài

1. **Hiệu chuẩn mô hình rung bằng dữ liệu thật.** Nhật ký bay của hệ thống đã ghi mức rung đo từ bộ điều khiển bay cùng dòng với chất lượng phát hiện. Cần bay có chủ đích ở các mức rung khác nhau, trích phổ gia tốc từ nhật ký của ArduPilot để hiệu chuẩn tham số toán tử rung, thay cho các giá trị hiện được chọn để bao quát điều kiện bay.
2. **Mở rộng tập đánh giá bay thật.** Thu video gốc từ camera (không qua màn hình trạm mặt đất) ở nhiều độ cao (3, 5, 8, 12 m), nhiều người, nhiều bối cảnh và điều kiện ánh sáng, gán nhãn thành một benchmark đủ lớn cho miền bay thấp.
3. **Đưa mô hình nhận biết rung lên NPU.** Biên dịch phiên bản nhánh triển khai đã tinh chỉnh nhận biết rung thành HEF, và thử khối chú ý thuần tích chập để lấy lại phần độ chính xác mất khi bỏ khối self-attention.
4. **Mở rộng thử nghiệm bay bám.** Bay theo quy trình ở Phụ lục C với nhiều kịch bản chuyển động của người, đo sai số giữ khoảng cách và thời gian bám liên tục; tinh chỉnh các hệ số PID dựa trên nhật ký; bổ sung gửi đồng thời vị trí, vận tốc và gia tốc đặt.
5. **Nhận dạng lại theo ngoại hình.** Bổ sung một mô hình trích đặc trưng nhỏ để tìm lại đúng người sau khi bị che khuất hoặc rời khung lâu.
6. **Xác định ranh giới áp dụng của phương pháp huấn luyện nhận biết rung** bằng cách thử trên nhiều họ kiến trúc và hàm mất mát khác nhau, nhằm tìm ra yếu tố quyết định việc mô hình có phản hồi với phương pháp hay không.

2.2. Lĩnh vực nên ứng dụng kết quả nghiên cứu

- **Tìm kiếm cứu nạn:** hệ thống xử lý hoàn toàn trên máy bay nên hoạt động được ở vùng không có sóng di động sau thiên tai, lũ lụt.
- **Giám sát an ninh, tuần tra:** bám theo đối tượng di chuyển trong khuôn viên nhà

máy, khu công nghiệp, khu vực biên giới.

- **Quay phim tự động:** thể thao ngoài trời, sự kiện, sản xuất nội dung.
- **Giáo dục và đào tạo:** quy trình trọn vẹn từ dữ liệu tới điều khiển bay, cùng mã nguồn và các bộ kiểm thử, dùng làm học liệu thực hành cho các học phần thị giác máy tính, trí tuệ nhân tạo nhúng và điều khiển tự động tại Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh.
- **Các thiết bị thị giác trên nền rung:** phương pháp huấn luyện nhận biết rung chỉ thay đổi dữ liệu huấn luyện nên có thể áp dụng cho camera trên robot di động, xe tự hành, máy nông nghiệp.

TÀI LIỆU THAM KHẢO

[1] X. Wu, W. Li, D. Hong, R. Tao, and Q. Du, "Deep learning for unmanned aerial vehicle-based object detection and tracking: A survey," *IEEE Geoscience and Remote Sensing Magazine*, vol. 10, no. 1, pp. 91–124, 2022.

[2] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, "Detection and tracking meet drones challenge," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7380–7399, 2022.

[3] D. Du, Y. Qi, H. Yu, Y. Yang, K. Duan, G. Li, W. Zhang, Q. Huang, and Q. Tian, "The unmanned aerial vehicle benchmark: Object detection and tracking," in *Proc. European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, 2018, pp. 375–391.

[4] S.-Y. Yang, H.-Y. Cheng, and C.-C. Yu, "Real-time object detection and tracking for unmanned aerial vehicles based on convolutional neural networks," *Electronics*, vol. 12, no. 24, p. 4928, 2023.

[5] L.-Y. Lo, C. H. Yiu, Y. Tang, A.-S. Yang, B. Li, and C.-Y. Wen, "Dynamic object tracking on autonomous UAV system for surveillance applications," *Sensors*, vol. 21, no. 23, p. 7888, 2021.

[6] C. Liu and T. Szirányi, "Real-time human detection and gesture recognition for on-board UAV rescue," *Sensors*, vol. 21, no. 6, p. 2180, 2021.

[7] K. Kang and W. jong Kim, "A human-following drone providing gesture recognition to control IoT devices based on 3D body-landmark detection," *International Journal of Control, Automation and Systems*, vol. 23, no. 9, pp. 2718–2727, 2025.

[8] DJI, "ActiveTrack: intelligent subject tracking," Tài liệu sản phẩm, 2025, truy cập 09/2026. [Online]. Available: https://www.dji.com

[9] A. Loquercio, A. I. Maqueda, C. R. del Blanco, and D. Scaramuzza, "DroNet: Learning to fly by driving," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 1088–1095, 2018.

[10] D. Palossi, A. Loquercio, F. Conti, E. Flamand, D. Scaramuzza, and L. Benini, "A 64-mW DNN-based visual navigation engine for autonomous nano-drones," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8357–8371, 2019.

[11] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, 2023.

[12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[13] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.

[14] A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin, J. Han, and G. Ding, "YOLOv10: Real-time end-to-end object detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 107 984–108 011.

[15] G. Jocher, J. Qiu, and A. Chaurasia, "Ultralytics YOLO (YOLOv8, YOLO11, YOLO26)," Phần mềm mã nguồn mở, 2023, phiên bản 8.4, truy cập 09/2026. [Online]. Available: https://github.com/ultralytics/ultralytics

[16] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16 965–16 974.

[17] C. Li, R. Zhao, Z. Wang, H. Xu, and X. Zhu, "RemDet: Rethinking efficient model design for UAV object detection," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 39, no. 5, 2025, pp. 4643–4651.

[18] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[19] C. Michaelis, B. Mitzkus, R. Geirhos, E. Rusak, O. Bringmann, A. S. Ecker, M. Bethge, and W. Brendel, "Benchmarking robustness in object detection: Autonomous driving when winter is coming," arXiv:1907.07484, 2019.

[20] S. Dodge and L. Karam, "Understanding how image quality affects deep neural networks," in *Proc. Eighth International Conference on Quality of Multimedia Experience (QoMEX)*, 2016, pp. 1–6.

[21] K.-P. Phung, T.-H. Lu, T.-T. Nguyen, N.-L. Le, H.-H. Nguyen, and V.-P. Hoang, "Multi-model deep learning drone detection and tracking in complex background conditions," in *Proc. International Conference on Advanced Technologies for Communications (ATC)*, 2021, pp. 189–194.

[22] L.-A. Tran, T.-D. Do, D.-C. Park, and M.-H. Le, "Enhancement of robustness in object detection module for advanced driver assistance systems," in *Proc. International Conference on System Science and Engineering (ICSSE)*, 2021, pp. 158–163.

[23] T. B. Ngo, L. Ngo, D. T. Nguyen, A. V. Phi, A. Perera, and A. Nguyen, "UAV-based visual detection and tracking of drowning victims in maritime rescue operations," *Drones*, vol. 10, no. 2, p. 146, 2026.

[24] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, "Distance-IoU loss: Faster and better learning for bounding box regression," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 34, no. 7, 2020, pp. 12 993–13 000.

[25] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[26] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[28] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.

[29] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, 2016, pp. 21–37.

[30] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2999–3007.

[31] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 936–944.

[32] M. Tan, R. Pang, and Q. V. Le, "EfficientDet: Scalable and efficient object detection," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 778–10 787.

[33] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 9626–9635.

[34] C. Feng, Y. Zhong, Y. Gao, M. R. Scott, and W. Huang, "TOOD: Task-aligned one-stage object detection," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 3490–3499.

[35] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, 2020, pp. 213–229.

[36] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[37] D. Qin, C. Leichner, M. Delakis, M. Fornoni, S. Luo, F. Yang, W. Wang, C. Banbury *et al.*, "MobileNetV4: Universal models for the mobile ecosystem," in *Proc. European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, 2024, pp. 78–96.

[38] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, "CSPNet: A new backbone that can enhance learning capability of CNN," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 1571–1580.

[39] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets great again," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 13 728–13 737.

[40] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 9992–10 002.

[41] Y. Li, Q. Hou, Z. Zheng, M.-M. Cheng, J. Yang, and X. Li, "Large selective kernel network for remote sensing object detection," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 16 748–16 759.

[42] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Proc. European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, 2014, pp. 740–755.

[43] G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng, X. Xie, and J. Han, "Towards large-scale small object detection: Survey and benchmarks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.

[44] X. Yu, Y. Gong, N. Jiang, Q. Ye, and Z. Han, "Scale match for tiny person detection," in *Proc. IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1246–1254.

[45] J. Wang, C. Xu, W. Yang, and L. Yu, "A normalized Gaussian Wasserstein distance for tiny object detection," arXiv:2110.13389, 2021.

[46] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[47] S. Nah, T. H. Kim, and K. M. Lee, "Deep multi-scale convolutional neural network for dynamic scene deblurring," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 257–265.

[48] O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin, and J. Matas, "DeblurGAN: Blind motion deblurring using conditional adversarial networks," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8183–8192.

[49] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, p. 60, 2019.

[50] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[51] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," arXiv:1806.08342, 2018.

[52] M. Nagel, R. A. Amjad, M. van Baalen, C. Louizos, and T. Blankevoort, "Up or down? Adaptive rounding for post-training quantization," in *Proc. International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 119, 2020, pp. 7197–7206.

[53] Raspberry Pi Ltd, "Raspberry pi AI kit: Hailo-8l M.2 accelerator module," Tài liệu kỹ thuật, 2024, truy cập 09/2026. [Online]. Available: https://www.raspberrypi.com/documentation/accessories/ai-kit.html

[54] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[55] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3464–3468.

[56] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3645–3649.

[57] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. European Conference on Computer Vision (ECCV)*, ser. Lecture Notes in Computer Science, 2022, pp. 1–21.

[58] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 9686–9696.

[59] Z. Zhang, "A flexible new technique for camera calibration," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.

[60] F. Chaumette and S. Hutchinson, "Visual servo control. I. Basic approaches," *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, 2006.

[61] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ: Princeton University Press, 2008.

[62] ArduPilot Dev Team, "ArduPilot copter documentation: Guided mode and companion computers," Tài liệu trực tuyến, 2026, truy cập 09/2026. [Online]. Available: https://ardupilot.org/copter/

[63] A. Koubaa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, and M. Khalgui, "Micro air vehicle link (MAVLink) in a nutshell: A survey," *IEEE Access*, vol. 7, pp. 87 658–87 680, 2019.

[64] R. Padilla, W. L. Passos, T. L. B. Dias, S. L. Netto, and E. A. B. da Silva, "A comparative analysis of object detection metrics with a companion open-source toolkit," *Electronics*, vol. 10, no. 3, p. 279, 2021.

[65] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[66] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein, "Muon: An optimizer for hidden layers in neural networks," Bài viết kỹ thuật, 2024. [Online]. Available: https://kellerjordan.github.io/posts/muon/

PHỤ LỤC A

CẤU HÌNH PHẦN CỨNG VÀ THAM SỐ HỆ THỐNG

A.1 Tham số an toàn trên bộ điều khiển bay (ArduPilot Copter 4.7.0)

Giá trị "đọc từ máy bay" được tải trực tiếp từ Pixhawk 6C ngày 27/08/2026 (1.038 tham số). Giá trị khuyến nghị được đặt bằng Mission Planner trước mọi chuyến bay tự động.

Bảng A.1. Tham số ArduPilot cần chỉnh trước khi bay tự động

Tham số	Đọc từ máy bay	Khuyến nghị	Lý do
ARMING_SKIPCHK	−1	0	−1 bỏ qua toàn bộ kiểm tra trước khi arm
BATT_LOW_VOLT	10	14,0 V	Pin 4S không bao giờ tụt tới 10 V
BATT_CRT_VOLT	14	13,5 V	Phải thấp hơn ngưỡng pin yếu (đang bị ngược)
BATT_FS_LOW_ACT	0	2 (RTL)	0 là không có hành động nào
BATT_FS_CRT_ACT	0	1 (Land)	0 là không có hành động nào
FS_GCS_ENABLE	0	1	Có phản ứng khi mất liên lạc máy tính đồng hành
FENCE_ENABLE	0	1	Hàng rào bán kính 300 m, trần 100 m đã cấu hình nhưng đang tắt
GUID_TIMEOUT	3	1,0 s	Khớp thời gian chờ lệnh của phần mềm; dừng sau 1 s nếu Pi treo
WP_SPD	10 m/s	4,0 m/s	Phòng thủ nhiều lớp (phần mềm đã giới hạn 3–5 m/s)
INS_LOG_BAT_MASK	1	7	Ghi phổ rung của cả ba IMU để phân tích rung

Các tham số đã hợp lý và giữ nguyên: WP_ACC = 2,5 m/s², PSC_JERK_NE = 5, PSC_NE_VEL_P = 1,80, FS_THR_ENABLE = 3, FS_EKF_ACTION = 1, FS_CRASH_CHECK = 1, INS_HNTCH_ENABLE = 1 (bộ lọc chặn hài theo tốc độ động cơ, có lợi cho giảm

rung), ATC_RATE_WPY_MAX = 60 °/s.

Bảng A.2. Một số tham số ArduPilot đã đổi tên và đơn vị ở phiên bản 4.7

Tên trong tài liệu cũ	Tên ở 4.7	Đơn vị ở 4.7
WPNAV_SPEED	WP_SPD	m/s (cũ: cm/s)
WPNAV_ACCEL	WP_ACC	m/s² (cũ: cm/s²)
ANGLE_MAX	ATC_ANGLE_MAX	độ (cũ: phần trăm độ)
PSC_VELXY_P	PSC_NE_VEL_P	–
RTL_ALT	RTL_ALT_M	m (cũ: cm)
ARMING_CHECK	ARMING_SKIPCHK	mặt nạ bit, ý nghĩa ngược lại

A.2 Tham số chính của chương trình bám người (`config.yaml`)

Bảng A.3. Tham số cấu hình chính của chương trình

Khoá	Giá trị	Ghi chú
`model.backend`	`hailo`	Suy luận trên NPU
`model.hef`	`pfdet_v17_640_coco_-hailo8l.hef`	Mô hình triển khai
`model.conf_threshold`	0,25	F1 cao nhất trên dữ liệu bay thật
`model.nms_iou`	0,45	
`model.min_box_px`, `max_aspect`	200 px², 6,0	Lọc hộp bất thường
`camera.width/height/fps`	640×480, 30	$f_x = f_y = 700$, $c_x = 320$, $c_y = 240$
`tracking.max_age/min_hits`	45 / 3	1,5 s ở 30 Hz
`target_selection`	largest, lock, 3,0 s, 5,0 s	Chiến lược, khoá, giữ khoá, cửa sổ tìm lại
`distance.person_height_m`	1,7	Chiều cao người giả định
`control.follow_distance_m`	5,0	Khoảng cách đặt
`control.follow_altitude_m`	5,0	Độ cao đặt
`control.command_rate_hz`	30	Khớp nhịp bộ phát hiện
`safety.d_min / d_max`	3,0 / 15,0 m	
`safety.v_max_horizontal`	5,0 m/s	
`safety.yaw_rate_max`	1,0 rad/s	Dưới 60 °/s của máy bay
`safety.altitude_min/-max_agl`	2,0 / 25,0 m	
`safety.command_timeout_s`	1,0 s	Khớp `GUID_TIMEOUT`
`safety.battery_warn/-critical_v`	14,8 / 14,0 V	Pin 4S
`mavlink.connection`	`udpin:127.0.0.1:14550`	Qua `mavmux.py`
`web.port`, `allow_retarget`	8080, bật	Bảng giám sát

A.3 Siêu tham số huấn luyện

Bảng A.4. Siêu tham số huấn luyện SteadyDet-N

Siêu tham số	**Huấn luyện gốc**	**Tinh chỉnh rung**	**Tinh chỉnh cho NPU**
Dữ liệu	VisDrone	VisDrone	VisDrone + 25% COCO
Khởi tạo	ngẫu nhiên	checkpoint gốc	checkpoint nhánh triển khai
Số epoch	300	120	20
Kích thước lô $\times$ tích luỹ	16×6	8×12	12
Tốc độ học (cosin)	$10^{-2} \rightarrow 10^{-4}$	$2 \cdot 10^{-3} \rightarrow 10^{-4}$	10^{-3}
Khởi động	10 epoch	3 epoch	3 epoch
Bộ tối ưu	MuSGD	MuSGD	MuSGD
Suy giảm trọng số / quán tính	$5 \cdot 10^{-4}$ / 0,9	$5 \cdot 10^{-4}$ / 0,9	$5 \cdot 10^{-4}$ / 0,9
EMA	0,9998	0,9998	0,9998
Toán tử rung	tắt	$p = 0{,}5$, mức 1–3	tắt
Self-attention P_5	có	có	bỏ

PHỤ LỤC B

CHƯƠNG TRÌNH MÁY TÍNH

B.1 Cấu trúc mã nguồn

```
pfdet_drone_follow/
|-- models/pfdet_nano_v17.py     # kien truc SteadyDet-N (v15: SteadyDet-W)
|-- utils/losses_v15.py          # ham mat mat QFL + NWD/CIoU + gan nhan
|-- utils/musgd.py               # bo toi uu MuSGD
|-- vib_consistency.py           # toan tu rung + bien the rang buoc nhat quan
|-- train_v3.py                  # chuong trinh huan luyen (doc file cau hinh)
|-- configs/                     # cau hinh huan luyen tung thi nghiem
|-- paper_eval_unified.py        # MOT chuong trinh do cho moi mo hinh, 3 hat
   giong
|-- flight_eval_unified.py       # danh gia 92 khung bay that + bootstrap
|-- train_yolo_baselines.py      # huan luyen 5 mo hinh doi chung
|-- export_onnx_hailo.py         # xuat ONNX cho Hailo (opset 11, dynamo=False)
|-- compile_hef.py               # ONNX -> HEF (Hailo DFC)
|-- hailo_eval_val.py            # do do chinh xac INT8 tren Pi
|-- hailo_bench_latency.py       # do do tre tung chang p50/p95/p99
|-- cam_ctl.py                   # SoftAE: tu phoi sang bang phan mem
|-- mavmux.py                    # chia 1 cong MAVLink cho nhieu chuong trinh
|-- nckh_SV2026-474/video/replay_follow.py  # chay vong bam nguoi tren video bay
    thu
‘-- follow_drone/                # CHUONG TRINH BAM NGUOI
    |-- main.py                  # vong lap 30 Hz + bang giam sat
    |-- config.yaml              # toan bo tham so
    |-- RUNBOOK.md               # quy trinh ra bai bay
    |-- follow/
    |   |-- camera.py            # luong doc camera
    |   |-- detector_hailo.py    # suy luan NPU + giai ma + NMS
    |   |-- ocsort.py            # theo vet OC-SORT
    |   |-- target_selector.py   # chon, khoa, tim lai muc tieu
    |   |-- distance.py          # pinhole + Kalman 1 chieu
    |   |-- visual_servo.py      # servo thi giac lai
    |   |-- pid.py               # 3 bo PID
```

```
|   |-- safety.py            # lop an toan
|   |-- mavlink_client.py    # giao tiep ArduPilot
|   `-- dashboard.py         # web giam sat
|-- scripts/                 # calibrate_camera, test_mavlink, arm_test...
`-- tests/                   # 2 bo kiem thu tu dong (25 phep kiem tra)
```

B.2 Các lệnh tái lập kết quả

Huấn luyện và đánh giá (máy tính có GPU):

```
# Huan luyen goc, roi tinh chinh nhan biet rung
python train_v3.py --config configs/train_config_v17_light.yaml
python train_v3.py --config configs/train_config_v17_vib_augonly.yaml
# Do moi mo hinh bang MOT chuong trinh, 3 hat giong -> UNIFIED.json
python paper_eval_unified.py --seeds 0 1 2 --out runs/paper_queue/UNIFIED.json
```

Triển khai lên NPU:

```
python export_onnx_hailo.py --weights runs/train_v17_hailo_coco/best.pt --img-
    size 640
python compile_hef.py --onnx <tep>.onnx --hw-arch hailo8l --img-size 640 \
       --calib-dir <anh hieu chuan tu tap huan luyen> --calib-n 64
bash scripts/deploy_to_pi.sh      # day HEF sang Pi, quet nguong tren 92 khung
```

Trên Raspberry Pi 5:

```
./run.sh check                      # kiem tra truoc khi ra bai
./run.sh bench --minutes 2 --source cam   # do do tre tung chang
./run.sh mux                        # cua so 1: chia cong MAVLink
cd follow_drone
../venv/bin/python3 -u main.py --no-fly       # thu tren ban, khong gui lenh
../venv/bin/python3 -u main.py --no-takeoff   # bay: cat canh tay roi gat GUIDED
```

Kiểm thử tự động:

```
cd follow_drone
python3 tests/test_target_selector.py         # 14 phep kiem tra
python3 tests/test_safety_no_mode_change.py   # 11 phep kiem tra
```

B.3 Trích đoạn mã nguồn tiêu biểu

Toán tử rung (`vib_consistency.py`):

```
_PRESETS = {1: (5, 0.02, 0.10), 2: (9, 0.04, 0.20), 3: (15, 0.06, 0.30)}

def corrupt_batch(imgs, severity=None, angle=None):
    if severity is None:
        severity = random.choice([1, 2, 3])
    L, ns, br = _PRESETS.get(int(severity), _PRESETS[2])
    ang = random.uniform(0, 180) if angle is None else angle
    k = motion_blur_kernel(L, ang, imgs.device, imgs.dtype)   # (L, L)
    w = k.view(1, 1, *k.shape).repeat(imgs.shape[1], 1, 1, 1)
    out = F.conv2d(imgs, w, padding=k.shape[0] // 2, groups=imgs.shape[1])
    out = out + torch.randn_like(out) * ns                     # nhieu cam bien
    out = out * (1.0 + random.uniform(-br, br))                # dao dong sang
    return out
```

Thang mất mục tiêu không đổi chế độ bay (`follow_drone/follow/safety.py`):

```
def update_target_status(self, target_present):
    now = time.time()
    if target_present:
        self.target_lost_since = None
        self.current_state = SafetyState.FOLLOW
        return SafetyState.FOLLOW
    if self.target_lost_since is None:
        self.target_lost_since = now
        return SafetyState.COAST
    elapsed = now - self.target_lost_since
    # Nguoi lai so huu che do bay: KHONG goi set_mode o bat cu dau.
    new_state = SafetyState.COAST if elapsed < self.t_coast_end else SafetyState
    .HOVER
    self.current_state = new_state
    return new_state
```

PHỤ LỤC C

QUY TRÌNH VẬN HÀNH BAY BÁM NGƯỜI

Bước 0 – Chuẩn bị tham số (thực hiện trước, trong nhà)

Đặt đủ các tham số ở Bảng A.1 bằng Mission Planner, ghi và khởi động lại bộ điều khiển bay. Lưu ý ArduPilot 4.7 dùng tên và đơn vị mới (Bảng A.2). Chưa sửa xong năm tham số đầu thì không bay tự động.

Bước 1 – Kiểm tra trên Pi trước khi ra bãi

Chạy `./run.sh check`, yêu cầu 0 lỗi: HailoRT, thư viện, tệp HEF, camera, chạy thử mô hình, nhiệt độ, dung lượng đĩa và đồng bộ đồng hồ NTP (cần để căn thời gian ảnh với nhật ký bay).

Bước 2 – Khởi động phần mềm ngoài bãi

1. Cửa sổ 1: `./run.sh mux` để chia cổng MAVLink.
2. Cửa sổ 2: `main.py` (thêm `-no-fly` khi chỉ quan sát).
3. Mở `http://pi5.local:8080` trên laptop hoặc điện thoại cùng mạng.

Không chạy song song chương trình trạm mặt đất riêng vì NPU chỉ phục vụ một tiến trình.

Bước 3 – Kiểm tra trước khi arm (trên bảng giám sát)

Bảng C.1. Điều kiện bắt buộc trước khi arm

Ô trên bảng giám sát	Yêu cầu
Sẵn sàng bay	"SẴN SÀNG ARM" màu xanh
GPS	Định vị 3D trở lên, $\geq$ 8 vệ tinh, HDOP < 2,0
Rung	Cả ba trục < 30 m/s², bão hoà 0/0/0
Pin	> 15,5 V khi chưa quay động cơ
Nhiệt Pi	< 70 °C, cờ giảm xung 0x0
Cần ga	Ở vị trí thấp nhất

Bước 3b – Bay tay để khảo sát (khuyến nghị trước lần giao quyền đầu)

Chạy `main.py -no-fly` và bay tay. Chương trình chạy đủ chuỗi xử lý và đẩy lên bảng giám sát nhưng không gửi lệnh nào. Dùng nhật ký CSV (độ cao, số phát hiện, điểm tin cậy trên cùng dòng) để chọn độ cao nhận diện tốt.

Bước 4 – Giao quyền bay bám

1. Cất cánh ở LOITER bằng tay, lên khoảng 5 m, bay treo ổn định.
2. Trên Pi chạy `main.py -no-takeoff`.
3. Người lái gạt công tắc sang GUIDED; kiểm tra ô "chế độ bay" trên bảng giám sát (tuỳ chọn này không tự chuyển GUIDED).
4. Máy bay khoá người có hộp lớn nhất; dùng nút "Đổi mục tiêu" nếu khoá nhầm.
5. Lấy lại quyền bất cứ lúc nào bằng cách gạt công tắc sang chế độ khác; phần mềm im lặng ngay. Ctrl+C trên Pi gửi vận tốc 0 và không đổi chế độ, người lái tự hạ cánh.

Bước 5 – Sau chuyến bay

Kéo nhật ký bám người (`logs/flight_*.csv`), nhật ký ArduPilot (`.bin` trên thẻ nhớ Pixhawk) và video thu riêng về máy tính bằng `pull_flight_data.sh` để phân tích và bổ sung dữ liệu đánh giá.

Xử lý sự cố thường gặp

Bảng C.2. Sự cố thường gặp và cách xử lý

Hiện tượng	**Nguyên nhân và cách xử lý**
`HAILO_OUT_OF_PHYSICAL_DEVICES`	Tiến trình khác đang giữ NPU; tắt tiến trình đó rồi chạy lại
Cổng serial "returned no data"	Hai tiến trình cùng mở `/dev/ttyACM0`; dùng `mavmux.py`
`PreArm: Need Position Estimate`	Chưa có GPS; ra chỗ thoáng, chờ ≥ 8 vệ tinh
Camera chỉ 15 khung/giây	Bộ đệm bằng 1 hoặc tự phơi sáng; dùng cấu hình `cam_ctl.py`
Máy bay đứng yên khi đã chạy chương trình	Chưa gạt sang GUIDED; kiểm tra ô chế độ bay
Không khoá được người	Người quá xa (trên khoảng 8 m) hoặc ngưỡng quá cao; ngưỡng đang dùng 0,25

PHỤ LỤC D

MINH CHỨNG SẢN PHẨM ĐỀ TÀI

D.1 Danh mục sản phẩm nộp kèm

Bảng D.1. Danh mục sản phẩm nộp kèm hồ sơ nghiệm thu

STT	Sản phẩm	Tệp / nội dung
1	Chương trình máy tính	Thư mục mã nguồn `pfdet_drone_follow` (Phụ lục B)
2	Mô hình triển khai	`pfdet_v17_640_coco_hailo8l.hef` (INT8, Hailo-8L)
3	Video bay thử	`video/video_bay_thu.mp4`: phần 1 phát hiện người trên video bay thử; phần 2 vòng bám người trên cùng chuyến bay
4	Nhật ký vòng bám người	`video/replay_log.csv`: trạng thái, mục tiêu khoá, sai số hướng, lệnh quay hướng của từng khung
5	Báo cáo tổng kết	Tài liệu này
6	Bài tóm tắt đề tài	`tomtat/tomtat.docx` (2 trang, 2 cột)
7	Poster (mẫu BM07A)	`poster_a3/poster_a3.pdf` (A3 ngang, Phụ lục E) và `poster_a3/poster_a1.pdf` (A1)
8	Standee (mẫu BM07B)	`standee/standee.pdf` (0,6 $\times$ 1,6 m)
9	Bài trình bày trước hội đồng	`slides/slides.pdf`

D.2 Hình ảnh sản phẩm



Hình D.1. Phần 1 của video bay thử: SteadyDet-N phát hiện người trên các khung hình của chuyến bay 21/07/2026



Hình D.2. Phần 2 của video bay thử: vòng bám người khoá mục tiêu, hiển thị trạng thái, sai số hướng và lệnh quay hướng

PHỤ LỤC E

POSTER VÀ STANDEE CỦA ĐỀ TÀI

Poster của đề tài được trình bày theo mẫu BM07A/QT-PKHCN-NCKHSV/00: khổ A3 nằm ngang, bốn cột, gồm phần đầu (tên đề tài, mã số, giảng viên hướng dẫn, sinh viên thực hiện, email, khoa, logo trường và logo đơn vị), phần tóm tắt kèm từ khoá, và bốn mục I–IV cùng tài liệu tham khảo. Bản in triển lãm khổ A1 dùng cùng nội dung, phóng theo tỷ lệ 2:1 (tệp `poster_a3/poster_a1.pdf`).

Standee của đề tài (tệp `standee/standee.pdf`) trình bày theo mẫu BM07B/QT-PKHCN-NCKHSV/00, khổ 0,6 $\times$ 1,6 m, gồm năm mục I–V.

Trang sau là bản poster khổ A3 đóng kèm theo quy định.

TÊN ĐỀ TÀI: MÁY BAY KHÔNG NGƯỜI LÁI TỰ ĐỘNG THEO DÕI CON NGƯỜI BẰNG TRÍ TUỆ NHÂN TẠO

Mã số đề tài: SV2026-474

Giảng viên hướng dẫn: PGS.TS. Lê Mỹ Hà

Sinh viên thực hiện: Huỳnh Lê Thành Nhân (chủ nhiệm đề tài), Trịnh Trần Quốc Bảo

Email: 24119068@student.hcmute.edu.vn **Khoa:** Đào tạo tiên tiến

Tóm tắt

Đề tài xây dựng máy bay không người lái tự phát hiện và bám theo người, toàn bộ khối thị giác máy tính chạy ngay trên máy bay. **Mục tiêu:** mô hình nhận dạng người thời gian thực đạt độ chính xác từ 70% trở lên, thuật toán theo dõi giữ khoảng cách và hướng bay, tích hợp camera – xử lý ảnh – điều khiển bay hoạt động ổn định ngoài thực tế. **Phương pháp:** đề xuất mạng phát hiện người **SteadyDet-N** (1,17 triệu tham số; 4,6 GFLOPs) giữ nhánh độ phân giải cao cho người nhỏ, kết hợp **huấn luyện nhận biết rung** – đưa mô hình vật lý của nhoè, nhiễu và dao động sáng do cánh quạt vào dữ liệu huấn luyện, không đổi nhãn và không tốn chi phí khi suy luận; triển khai INT8 trên NPU Hailo-8L gắn Raspberry Pi 5, ghép với theo vết OC-SORT, khoá mục tiêu, ước lượng khoảng cách pinhole và ba bộ điều khiển PID gửi lệnh MAVLink 30 Hz tới ArduPilot. **Kết quả:** mô hình trên máy bay đạt **AP@0,5 = 0,917** trên dữ liệu bay thật (vượt chỉ tiêu 70%), chuỗi xử lý đạt **30 khung hình/giây**; huấn luyện nhận biết rung nâng tỷ lệ giữ độ chính xác khi rung từ **16,8% lên 36,2%**, giúp mô hình nhỏ nhất trong bảng vượt cả bốn mô hình YOLO bản nano khi rung; vòng bám người chạy trên video bay thử giữ mục tiêu 83,5% thời gian. **Bình luận:** kết quả cho thấy với thiết bị bay, thứ quyết định không phải điểm số trên benchmark ảnh sạch mà là độ chính xác còn giữ được khi khung máy bay rung, và độ bền này phải được đo và tối ưu riêng.

Keywords: UAV, person detection, vibration robustness, edge AI, Hailo NPU, OC-SORT, visual servoing, ArduPilot.

I. ĐẶT VẤN ĐỀ

UAV nhiều cánh quạt ngày càng phổ biến trong giám sát, cứu hộ, quay phim. Người vận hành vừa phải lái vừa phải giữ đối tượng trong khung hình, nên một UAV **tự nhận dạng và bám theo người** làm tăng tính linh hoạt, an toàn và tiện ích.

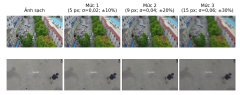
Trong 5 năm gần đây nhiều hệ thống bám người đã được công bố [1–3]; trong nước có các nghiên cứu phát hiện và theo vết UAV trong nền phức tạp [6] và nâng độ bền khối phát hiện cho hệ hỗ trợ lái xe [7]. Hầu hết đánh giá mô hình trên ảnh **sạch, tĩnh**.

Thực tế khung máy bay **luôn rung** khi động cơ quay: ảnh bị nhoè theo hướng, nhiễu tăng, độ sáng dao động. Mạng học sâu rất nhạy với các suy giảm này [4], nhưng chưa có giao thức nào mô phỏng riêng rung của UAV để đánh giá các bộ phát hiện nhỏ gọn.

Mục tiêu của nghiên cứu:

- Mô hình nhận dạng người thời gian thực, độ chính xác $\geq$ **70%**, chạy ngay trên máy bay.
- Thuật toán theo dõi giữ khoảng cách và hướng bay với người được bám.
- Tích hợp camera – xử lý ảnh – điều khiển bay thành hệ thống ổn định ngoài thực tế.

Đối tượng: bộ phát hiện người từ UAV, ảnh hưởng của rung, thuật toán bám người trên nền ArduPilot. **Phạm vi:** một người, không gian thoáng, bay thấp 3–6 m.



Hình 1. Ảnh sạch và ba mức rung của giao thức VisDrone-Shake

II. PHƯƠNG PHÁP VÀ NỘI DUNG NGHIÊN CỨU

1. Mô hình SteadyDet-N. Mạng một giai đoạn anchor-free, 1,17 M tham số, 4,6 GFLOPs. Giữ mức dự đoán bước 4 (nơi người ở xa còn tín hiệu) nhưng làm nhánh này mỏng: 1 khối, nửa số kênh neck, head depthwise separable; tính toán dồn về các mức có bản đồ nhỏ. Chi phí giảm 2,8 lần so với bản trước. Hàm mất mát: QFL cho phân loại, NWD kết hợp CIoU theo diện tích cho hồi quy hộp.

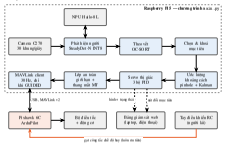
2. Huấn luyện nhận biết rung. Mỗi batch ảnh được thay bằng bản đã làm rung với xác suất $p = 0{,}5$:

$$\hat{x} = \gamma(x * k_{L,\theta}) + n, \quad \mathscr{L} = \mathscr{L}_{\text{det}}\big(f_\theta(\hat{x}),\, y\big)$$

Nhoè, nhiễu và độ sáng không dịch chuyển vật nên **nhãn giữ nguyên**; toán tử chỉ tồn tại khi huấn luyện nên **không tốn chi phí suy luận**.

3. Đo công bằng. Một chương trình đánh giá duy nhất cho mọi mô hình, cố định seed ngẫu nhiên và lặp 3 lần, báo cáo trung bình $\pm$ độ lệch chuẩn; 5 mô hình đối chứng YOLOv8/v10/11/26-n và RT-DETR-l.

4. Triển khai và điều khiển. Mô hình lượng tử hoá INT8, chạy trên NPU Hailo-8L gắn Raspberry Pi 5; theo vết OC-SORT, khoá và tìm lại mục tiêu, ước lượng khoảng cách pinhole với bộ lọc Kalman, ba bộ PID, lớp an toàn nhiều tầng, gửi lệnh vận tốc MAVLink 30 Hz tới Pixhawk 6C (GUIDED).

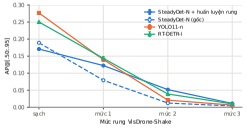


Hình 2. Kiến trúc hệ thống trên máy bay

III. KẾT QUẢ NGHIÊN CỨU

Bảng 1. Kết quả chính đo trên phần cứng thật

Chỉ số	Kết quả
AP@0,5 bay thật (INT8, trên máy bay)	**0,917**
F1 / recall / precision	0,880 / 0,837 / 0,928
Tốc độ đầu-cuối	**30,0 FPS**
Độ trễ p50 / p99	32,04 / 36,15 ms
Suy luận NPU @320 / @640	112,55 / 63,66 FPS
So với YOLO26-n trên Hailo-8L	**nhanh hơn 1,70 lần**
retain khi rung: gốc $\rightarrow$ đề xuất	16,8 $\rightarrow$ **36,2%**
Vòng bám: giữ mục tiêu	**83,5%** thời gian
Sai số hướng khi bám (trung vị)	5,5°



Hình 3. AP theo mức rung: SteadyDet-N có huấn luyện rung vượt YOLO11-n và RT-DETR-l từ mức 2

Khi rung, mọi bộ phát hiện nhỏ gọn mất **74–83%** độ chính xác. Sau huấn luyện nhận biết rung, mô hình **nhỏ và rẻ nhất** vượt cả **bốn mô hình YOLO bản nano** khi bay (mCP 0,0617 so với 0,0557) và vượt RT-DETR-l lớn gấp 28 lần ở mức rung bay tiền; trên dữ liệu bay thật, AP@0,75 tăng khoảng 80%.



Hình 4. Vòng bám người trên video bay thử

IV. KẾT LUẬN

- Hoàn thành UAV tích hợp nhận dạng và bám theo người, xử lý hoàn toàn trên máy bay: **AP@0,5 = 91,7%** (vượt chỉ tiêu 70%), **30 FPS**, độ trễ p99 36,15 ms.
- Chỉ ra bằng đo đạc có đối chứng rằng rung làm mọi bộ phát hiện nhỏ gọn mất 74–83% độ chính xác — điều các benchmark thông thường không đo.
- Huấn luyện nhận biết rung tăng gấp đôi độ bền (16,8 $\rightarrow$ 36,2%) mà không tốn chi phí suy luận, rẻ hơn nhiều so với thêm mạng khử nhoè hay gimbal chống rung.
- Trên NPU, số *context* chứ không phải số phép tính quyết định tốc độ: SteadyDet-N nhanh hơn YOLO26-n 1,70 lần.
- **Hạn chế:** tập bay thật còn nhỏ (92 khung, một người, một độ cao); mô hình rung là mô phỏng.
- **Hướng phát triển:** hiệu chuẩn mô hình rung bằng nhật ký IMU, mở rộng dữ liệu bay thật nhiều độ cao, đưa mô hình nhận biết rung lên NPU, bám nhiều kịch bản và nhận dạng lại theo ngoại hình.

TÀI LIỆU THAM KHẢO

[1] S.-Y. Yang, H.-Y. Cheng, C.-C. Yu, "Real-time object detection and tracking for unmanned aerial vehicles based on convolutional neural networks," *Electronics*, vol. 12, no. 24, p. 4928, 2023.

[2] L.-Y. Lo *et al.*, "Dynamic object tracking on autonomous UAV system for surveillance applications," *Sensors*, vol. 21, no. 23, p. 7888, 2021.

[3] K. Kang, W. Kim, "A human-following drone providing gesture recognition to control IoT devices based on 3D body-landmark detection," *Int. J. Control, Automation and Systems*, vol. 23, no. 9, pp. 2718–2727, 2025.

[4] D. Hendrycks, T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," in *Proc. ICLR*, 2019.

[5] P. Zhu *et al.*, "Detection and tracking meet drones challenge," *IEEE TPAMI*, vol. 44, no. 11, pp. 7380–7399, 2022.

[6] K.-P. Phung *et al.*, "Multi-model deep learning drone detection and tracking in complex background conditions," in *Proc. ATC*, 2021, pp. 189–194.

[7] L.-A. Tran *et al.*, "Enhancement of robustness in object detection module for advanced driver assistance systems," in *Proc. ICSSE*, 2021, pp. 158–163.

[8] J. Cao *et al.*, "Observation-centric SORT," in *Proc. CVPR*, 2023, pp. 9686–9696.

PHỤ LỤC F

BẢN SAO THUYẾT MINH VÀ HỢP ĐỒNG ĐÃ ĐƯỢC PHÊ DUYỆT

Phụ lục này gồm bản sao thuyết minh đề tài nghiên cứu khoa học sinh viên năm 2026 (mẫu BM02/QT-PKHCN-NCKHSV/00, mã số SV2026-474) và bản sao hợp đồng thực hiện đề tài đã được phê duyệt.

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐÀO TẠO TIÊN TIẾN

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

THUYẾT MINH ĐỀ TÀI
NGHIÊN CỨU KHOA HỌC SINH VIÊN NĂM 2026

1. **TÊN ĐỀ TÀI:** Máy bay không người lái tự động theo dõi con người bằng trí tuệ nhân tạo	**2. MÃ SỐ** SV2026-474
3. LĨNH VỰC NGHIÊN CỨU	**4. LOẠI HÌNH NGHIÊN CỨU**

Tự nhiên	Xã hội nhân văn	Giáo dục	Kỹ thuật	Nông Lâm-Ngư	Y dược	Môi trường	Cơ bản	Ứng dụng	Triển khai
☐	☐	☐	☒	☐	☐	☐	☐	☒	☐

5. **THỜI GIAN THỰC HIỆN:** 12 tháng

6. CƠ QUAN CHỦ TRÌ: TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

Khoa: Đào tạo tiên tiến — Bộ môn trực thuộc: Công nghệ kỹ thuật máy tính

7. SINH VIÊN THỰC HIỆN ĐỀ TÀI

Họ và tên: Huỳnh Lê Thành Nhân — Mã số SV: 24119068
Điện thoại: 093795656 — E-mail: 24119068@student.hcmute.edu.vn

8. **GIÁO VIÊN HƯỚNG DẪN**

Họ và tên GVHD: PGS.TS Lê Mỹ Hà
Email: halm@hcmute.edu.vn

9. NHỮNG NGƯỜI THAM GIA THỰC HIỆN ĐỀ TÀI (NẾU CÓ)

Họ và tên	MSSV	Khoa	Điện thoại, email	Ký tên
Trịnh Trần Quốc Bảo	24119016	Đào tạo tiên tiến	0962924006 24119016@student.hcmute.edu.vn	

10. TÌNH HÌNH NGHIÊN CỨU TRONG VÀ NGOÀI NƯỚC

10.1. Tổng quan tình hình nghiên cứu thuộc lĩnh vực của đề tài:

- Trong nước: Trong nước**:** trong những năm gần đây, một số trường đại học và viện nghiên cứu tại Việt Nam như Đại học Bách Khoa Thành phố Hồ Chí Minh, Đại học Công nghệ thuộc Đại học Quốc gia Hà Nội và Học viện Kỹ thuật Quân sự đã triển khai nhiều đề tài liên quan đến điều khiển máy bay không người lái tự hành và xử lý hình ảnh bằng trí tuệ nhân tạo. Tuy nhiên, phần lớn các nghiên cứu này mới chỉ tập trung vào những ứng dụng cơ bản như phát hiện vật cản, xác định vị trí bằng hệ thống định vị toàn cầu hoặc điều khiển ổn định khi bay. Các hướng nghiên cứu về máy bay không người lái có khả năng tự động đi theo con người bằng trí tuệ nhân tạo hiện vẫn đang ở giai đoạn thử nghiệm, chủ yếu sử dụng các phương pháp nhận dạng hình ảnh truyền thống. Việc ứng dụng các mô hình học sâu vào điều khiển bay theo thời gian thực vẫn còn hạn chế do yêu cầu cao về khả năng tính toán và độ ổn định của hệ thống.
- Ngoài nước: Ngoài nước**:** trên thế giới, các nghiên cứu quốc tế đã đạt được nhiều tiến bộ trong việc ứng dụng thị giác máy tính và học sâu cho hệ thống máy bay không người lái tự hành. Các công trình gần đây đã chứng minh rằng việc kết hợp giữa nhận dạng đối tượng và theo dõi mục tiêu bằng mô hình học sâu giúp máy bay không người lái có thể bám theo con người một cách chính xác và ổn định trong thời gian thực [1][2].
  Một số nghiên cứu tại các trường đại học lớn như Viện Công nghệ Massachusetts, Đại học Stanford và Đại học ETH Zurich đã phát triển hệ thống máy bay không người lái có khả năng phát hiện, theo dõi và duy trì khoảng cách an toàn với người di chuyển [3].
  Ngoài ra, các tập đoàn công nghệ hàng đầu như DJI đã thương mại hóa tính năng theo dõi người dùng tự động bằng trí tuệ nhân tạo, điển hình là công nghệ ActiveTrack, cho phép máy bay không người lái nhận dạng và bám theo người sử dụng trong nhiều điều kiện môi trường khác nhau [4].
  Những kết quả nghiên cứu này cho thấy hướng phát triển máy bay không người lái bám theo người bằng trí tuệ nhân tạo có tiềm năng ứng dụng rất lớn trong các lĩnh vực như quay phim, giám sát, cứu hộ và an ninh, tuy nhiên vẫn cần tối ưu hơn về mô hình và phần cứng để đáp ứng yêu cầu về tốc độ xử lý và tiết kiệm năng lượng.

TÀI LIỆU THAM KHẢO

[1] L. Zheng, Y. Lin, and W. Chen, *Vision-Based Human Tracking for Autonomous UAVs Using Deep Learning*, *IEEE Access*, vol. 10, pp. 65431–65440, 2022.

[2] J. Mao, X. Zhao, and Y. Liu, *Human-Following Drone System Based on Real-Time Object Detection and Tracking*, *Sensors*, vol. 21, no. 5, pp. 1–14, 2021.

[3] S.-Y. Yang, H.-Y. Cheng, and C.-C. Yu, *Real-Time Object Detection and Tracking for Unmanned Aerial Vehicles Based on Convolutional Neural Networks*, *Electronics*, vol. 12, no. 24, p. 4928, 2023.

[4] K. Kang, et al., *A Human-Following Drone Providing Gesture Recognition*, *Springer*, 2025.

10.2. Danh mục các công trình liên quan: (Liệt kê các công trình do chủ nhiệm đề tài đã công bố có liên quan đến đề tài này: Tên tác giả; Tên bài báo, tên ấn phẩm; Các yếu tố về xuất bản khác)

11. TÍNH CẤP THIẾT ĐỀ TÀI

Trong bối cảnh máy bay không người lái ngày càng được ứng dụng rộng rãi trong giám sát, cứu hộ, quay phim, và giao hàng, khả năng nhận dạng và đi theo người tự động giúp tăng tính linh hoạt, an toàn và tiện ích. Việc nghiên cứu đề tài này có ý nghĩa trong việc tự động hóa hệ thống bay, tăng khả năng tương tác giữa con người và máy bay không người lái.

12. MỤC TIÊU ĐỀ TÀI

- Xây dựng mô hình máy bay không người lái có khả năng phát hiện và nhận dạng con người theo thời gian thực bằng trí tuệ nhân tạo.
- Thiết kế thuật toán theo dõi chuyển động để máy bay không người lái duy trì khoảng cách và hướng bay phù hợp với người được theo.
- Tích hợp hệ thống máy ảnh – xử lý ảnh – điều khiển bay tự động thành một hệ thống hoạt động ổn định trong môi trường thực tế.

13. CÁCH TIẾP CẬN, PHƯƠNG PHÁP NGHIÊN CỨU, PHẠM VI NGHIÊN CỨU

- Cách tiếp cận: Ứng dụng học sâu và thị giác máy tính để nhận dạng người, kết hợp điều khiển tự động để di chuyển máy bay không người lái theo mục tiêu.
- Phương pháp nghiên cứu: Thu thập và huấn luyện mô hình nhận dạng người và thiết kế thuật toán theo dõi và điều khiển vị trí máy bay không người lái dựa trên thông tin hình ảnh.
- Phạm vi nghiên cứu: Giới hạn trong môi trường không gian hẹp, tầm nhìn rõ, mục tiêu duy nhất là con người di chuyển với tốc độ trung bình. Không xét đến nhiều đối tượng hoặc điều kiện môi trường phức tạp ví dụ: ánh sáng mạnh, gió lớn, che khuất.

14. NỘI DUNG NGHIÊN CỨU VÀ TIẾN ĐỘ THỰC HIỆN

STT	Các nội dung, công việc thực hiện chủ yếu	Sản phẩm phải đạt	Thời gian (số tháng)	Người thực hiện
1	Nghiên cứu tổng quan và thu thập dữ liệu huấn luyện: tìm hiểu các mô hình phát hiện người, thu thập và xử lý tập dữ liệu ảnh người ở nhiều góc độ.	Báo cáo tổng quan và bộ dữ liệu huấn luyện chuẩn hóa.	2	Trịnh Trần Quốc Bảo

2	Huấn luyện và đánh giá mô hình nhận dạng người: huấn luyện mô hình phát hiện người theo thời gian thực, đánh giá độ chính xác và tốc độ khung hình.	Mô hình trí tuệ nhân tạo nhận dạng người đạt độ chính xác ≥70%.	2	Huỳnh Lê Thành Nhân
3	Thiết kế hệ thống điều khiển máy bay không người lái tự động đi theo người: lập trình thuật toán theo dõi vị trí và điều khiển máy bay không người lái duy trì khoảng cách ổn định.	Hệ thống điều khiển máy bay không người lái hoạt động ổn định.	3	Trịnh Trần Quốc Bảo
4	Tích hợp và kiểm thử toàn hệ thống: kết hợp mô hình trí tuệ nhân tạo, máy ảnh và bộ điều khiển máy bay không người lái; chạy thử nghiệm trong môi trường thực tế.	Máy bay không người lái hoàn chỉnh có khả năng nhận dạng và đi theo người tự động.	3	Huỳnh Lê Thành Nhân
5	Viết báo cáo tổng kết	Báo cáo tổng kết	2	Trịnh Trần Quốc Bảo

15. SẢN PHẨM VÀ ĐỊA CHỈ ỨNG DỤNG

Loại sản phẩm:

Mẫu ☐	Vật liệu ☐	Thiết bị máy móc ☐	Dây chuyền công nghệ ☐
Giống cây trồng ☐	Giống gia súc ☐	Qui trình công nghệ ☐	Phương pháp ☐
Tiêu chuẩn ☐	Qui phạm ☐	Sơ đồ ☐	Báo cáo phân tích ☒
Tài liệu dự báo ☐	Đề án ☐	Luận chứng kinh tế ☐	Chương trình máy tính ☒
Bản kiến nghị ☐	Sản phẩm khác:		

Tên sản phẩm, số lượng và yêu cầu khoa học đối với sản phẩm

STT	Tên sản phẩm	Số lượng	Yêu cầu khoa học
1	Chương trình máy tính cho máy bay không người lái tự động theo dõi con người	01	Nhận dạng chính xác ≥70%, theo dõi con người ổn định

2	Báo cáo tổng kết	01	Đạt yêu cầu báo cáo khoa học

Địa chỉ có thể ứng dụng (tên địa phương, đơn vị ứng dụng): Sản phẩm được áp dụng tại trường đại học Công Nghệ Kỹ Thuật TP. Hồ Chí Minh, để thu thập dữ liệu, tiến hành phân tích và đánh giá tính khả thi của sản phẩm.

16. KINH PHÍ THỰC HIỆN ĐỀ TÀI

Tổng kinh phí: 5.000.000 đồng

Dự trù kinh phí theo các mục chi:

STT	Nội dung công việc	Thành tiền	Ghi chú
1	**Chi công thực hiện đề tài của chủ nhiệm và thành viên**	**4.900.000**	Bảng ký nhận
	- Nghiên cứu tổng quan và thu thập dữ liệu huấn luyện.	**300.000**	
	- Huấn luyện và đánh giá mô hình nhận dạng người.	**300.000**	
	- Thiết kế hệ thống điều khiển máy bay không người lái tự động đi theo người.	**3.146.000**	
	- Tích hợp và kiểm thử toàn hệ thống.	**1.000.000**	
	- Viết báo cáo tổng kết	**154.000**	
2	**Chi in ấn, photo**	**100.000**	Hóa đơn in ấn
	Tổng cộng	**5.000.000**	

TP. HCM, ngày 7 tháng 1 năm 2026

Trưởng Khoa	GV hướng dẫn	Sinh viên thực hiện đề tài
(*Ký và ghi rõ họ tên*)	(*Ký và ghi rõ họ tên*)	(*Ký và ghi rõ họ tên*)

TP. HCM, ngày 15 tháng 1 năm 2026

TUQ. HIỆU TRƯỞNG

Q. TRƯỞNG PHÒNG KHCN

(*Ký và ghi rõ họ tên*)

PGS.TS. Vũ Văn Phong